Republic of Tunisia
Ministry of Higher Education and Scientific Research

**University of Carthage — École Polytechnique de Tunisie**

**MASTER'S THESIS**

Submitted in partial fulfillment of the requirements for the degree of

**MASTER'S IN COMPLEX AND INTELLIGENT SYSTEMS**

# Learning Motor Control Strategies in Musculoskeletal Systems Using Reinforcement Learning

**Presented by:**

M. *Mohamed Naceur Hammami*

**Supervised by:**

Prof. *Taysir Rezgui*

**Academic Year 2025–2026**

*To my parents, an inexhaustible source of love and support,*
*whose sacrifices made this work possible.*

*To my brother and sisters, faithful companions at every moment.*

*To my friends and colleagues, for their constant kindness.*

*To all those affected by neurological injury, who remind us*
*that scientific research must serve human dignity.*

***I humbly dedicate this modest work to you.***

# Acknowledgements

At the completion of this work, I would like to express my deep gratitude to all the people who contributed, directly or indirectly, to its realisation.

First and foremost, I extend my most sincere thanks to my thesis supervisor, Professor Taysir Rezgui, whose availability, wise advice, and rigorous guidance were invaluable throughout this project. Their expertise in computational biomechanics has been an intellectual model that I will strive to draw inspiration from for years to come.

I also wish to express my gratitude to the entire teaching staff of the Computer Engineering and Cyber-Physical Systems Department at the École Polytechnique de Tunisie, who provided me with the theoretical and methodological foundations necessary for the successful completion of this program.

I warmly thank the members of the jury for the interest they have shown in this work and for agreeing to evaluate it, thereby devoting a valuable part of their time to its critical reading.

My thanks also go to the *open-source* scientific community — the developers of MuJoCo, Gymnasium, Stable-Baselines3, PyTorch, Optuna, and the Python libraries used throughout this work — whose efforts greatly facilitated my experiments, and without whom reproducible reinforcement-learning research would be far more difficult.

Finally, I thank my family and loved ones for their unwavering moral support and their patience throughout this academic journey.

**To all of you, thank you.**

# Executive Summary

Because nine muscles actuate four degrees of freedom, infinitely many force distributions produce any given arm movement, and the movement itself can never reveal which one the controller used. Biomechanics resolves this indeterminacy with a minimum-effort criterion; this thesis asks whether a *Deep Reinforcement Learning* (DRL) controller, trained only on a reaching reward, arrives at the same per-muscle force distribution.

The work began from a different question — whether a trained policy could replace the iterative Newton–Raphson muscle solver cheaply at inference time — and that premise did not survive measurement. A directly solved classical load-sharing problem runs in 23,9 µs against the network's 288,8 µs, so no speed advantage is claimed anywhere in this work.

A four-degree-of-freedom articulated multi-body dynamic model driven by nine muscles was developed in MuJoCo [1]. The model specifies a Hill-type muscle representation [2, 3], including the contractile element with its force-length (FL) and force-velocity (FV) relationships, the series elastic element (SEE) modelling the tendon, the parallel elastic element (PEE), pennation-angle dynamics, and neural activation dynamics [4]. Numerical integration of the tendon-fibre equilibrium equation is handled with an iterative Newton–Raphson method. The resulting system is wrapped as a Gymnasium simulation environment.

Four DRL algorithms were implemented with Stable-Baselines3 [5] and compared: DDPG [6], TD3 [7], SAC [8], and PPO [9], alongside a random policy and a PD impedance controller as baselines. Five configurations were trained for $10^5$ steps with three seeds each; the best configuration was additionally trained to $3\times10^5$ steps and evaluated over fifty episodes.

The central analysis compares the learned muscle forces against those prescribed by classical static optimisation on the identical movements. The learned controller reproduces the coordination pattern of the classical solution — correlation $0{,}81 \pm 0{,}04$ over five independent training runs; pattern cosine 0,813 against a shuffled null of 0,372 — while expending $1{,}91 \pm 0{,}15$ times the minimum effort required to generate the same joint torques, the excess concentrated in co-contraction of the elbow antagonists. A properly tuned conventional controller solving the same load-sharing problem explicitly attains 1,263, so the learned policy uses about 51 % more effort than is achievable rather than 91 % more than an unattainable ideal. Raising the reward's effort weight closes

most of that gap, to $1{,}304 \pm 0{,}025$ against that floor.

Two secondary results follow. An error plateau at 0,10 m survived three reward redesigns and a fourfold correction to the training budget; one-at-a-time ablation over five hyperparameters identifies the discount factor as its sole cause, recovering 106 % of the default-to-tuned interval where the other four recover nothing. Its default horizon of 100 steps spanned the whole episode against a reach completing in 25. And configuration proved to dominate algorithm choice: with every algorithm given a correctly matched discount factor, 0,060 m separates the best off-policy method from the second, while correcting the discount factor within a single algorithm is worth 0,165 m — about three times as much.

The results rest on three seeds for the algorithm comparison and five for the principal analysis. No significance testing is reported. The strict success criterion — one that a privileged controller satisfies only 7 % of the time — is met in 0–20 % of episodes depending on seed, a twentyfold spread under identical settings that identifies it as noise. Limitations are set out in full in the general conclusion.

**Keywords:** Musculoskeletal model, MuJoCo, Hill model, Static optimisation, Muscle redundancy, Deep Reinforcement Learning, Motor control, PPO, SAC, TD3, DDPG, Co-contraction, Discount factor.

# Abstract

Nine muscles actuate four degrees of freedom in the model studied here, so infinitely many force distributions produce any given movement and the movement cannot reveal which was used. Biomechanics resolves this indeterminacy by a minimum-effort criterion. This thesis asks whether a *Deep Reinforcement Learning* (DRL) controller, trained only on a reaching reward, recovers the same per-muscle force distribution, and under what conditions on the training objective.

The work was begun on a different premise — that a trained policy might replace the iterative Newton–Raphson muscle solver cheaply at inference time. That premise did not survive measurement: a directly solved classical load-sharing problem takes 23,9 µs against the network's 288,8 µs. The latency argument is withdrawn in full.

A four-degree-of-freedom multi-body articulated dynamic model driven by nine muscles was developed using the MuJoCo physics engine [1]. The model specifies a Hill-type muscle model [2, 3] comprising: a contractile element (CE) governed by force-length (FL) and force-velocity (FV) relationships; a series elastic element (SEE) representing the tendon; a parallel elastic element (PEE); pennation-angle dynamics; and first-order activation dynamics [4]. The tendon-fibre equilibrium equation is numerically resolved via Newton-Raphson iteration at each simulation step.

Four DRL algorithms were implemented and benchmarked: DDPG [6], TD3 [7], SAC [8], and PPO [9], alongside two reference baselines: a random policy and a PD impedance controller. Five configurations were trained for $10^5$ steps with three seeds each, and the best configuration additionally to $3{\times}10^5$ steps with fifty-episode evaluation.

The central contribution is a per-muscle load-sharing comparison between the learned solution and classical static optimisation, posed on the identical trajectory with the identical required joint torques and verified to machine precision. It is a conditional comparison — static optimisation never chooses a trajectory of its own — and therefore bears on force distribution, not on the realism of the movement. The controller reproduces the coordination structure of the classical solution (Pearson $r = 0.81 \pm 0.04$ over five seeds; pattern cosine 0.813 against a shuffled null of 0.372) at a fixed, branch-free inference cost, but expends $1.91 \pm 0.15$ times the minimum effort needed for the same joint torques, the excess concentrated in elbow antagonist co-contraction. Raising the reward's effort weight reduces that ratio to $1.304 \pm 0.025$ against the 1.263 a properly tuned conventional controller attains, for a mild accuracy

cost. Across all fifteen trained policies the discrepancy is present, ranging from 1.21 to 2.30×, and the ordering by muscular economy is approximately the inverse of the ordering by reaching accuracy.

Two secondary results are reported. An error plateau at 0,10 m survived three complete reward redesigns and a fourfold correction to the effective training budget. One-at-a-time ablation over five hyperparameters, three seeds each, identifies the *discount factor* as its sole cause: it recovers 106 % of the default-to-tuned interval while the target entropy, learning rate, target-network rate and batch size each recover nothing. Its default value implies a 100-step horizon — the entire episode — against a reach that completes in 25 steps. Separately, configuration was found to dominate algorithm choice: with every algorithm given a correctly matched discount factor, 0,060 m separates the best off-policy method from the second, against 0,165 m recovered by correcting that one setting within a single algorithm.

These results rest on three seeds for the algorithm comparison and the ablations, and five for the principal analysis and the effort study; no significance testing is reported. Under the strict success criterion — which a privileged controller with full state access satisfies only 7 % of the time — the best policy scores 0–4 %. Limitations are stated in full in the general conclusion.

**Keywords:** Musculoskeletal model, MuJoCo, Hill muscle model, Static optimisation, Muscle redundancy, Newton–Raphson solver, Deep Reinforcement Learning, Motor control, PPO, SAC, TD3, DDPG, Co-contraction, Discount factor.

# List of Abbreviations

| Abbrev. | Meaning |
|---|---|
| BCa | *Bias-Corrected and accelerated (bootstrap)* |
| CCI | Co-contraction index (Falconer & Winter) |
| CE | *Contractile Element* (Hill model) |
| CNS | Central nervous system |
| DOF | Degrees of freedom |
| DDPG | *Deep Deterministic Policy Gradient* |
| DR | *Domain Randomization* |
| DRL | *Deep Reinforcement Learning* |
| EMA | *Exponential Moving Average* |
| EMG | Electromyography |
| FL | Force-length (muscle relationship) |
| FV | Force-velocity (muscle relationship) |
| GAE | *Generalised Advantage Estimation* |
| GPU | *Graphics Processing Unit* |
| MDP | *Markov Decision Process* |
| MJCF | *MuJoCo XML Format* |
| MLP | *Multi-Layer Perceptron* |
| MuJoCo | *Multi-Joint dynamics with Contact* |
| NMF | *Non-negative Matrix Factorisation* |
| NRMSE | *Normalised Root Mean Squared Error* |
| OFC | *Optimal Feedback Control* |
| SLSQP | *Sequential Least-Squares Quadratic Programming* |
| VAF | Variance accounted for (synergy decomposition) |
| OpenSim | *Open-Source Musculoskeletal Simulator* |
| OU | Ornstein-Uhlenbeck (stochastic process) |
| PD | *Proportional-Derivative* (controller) |
| PEE | *Parallel Elastic Element* (Hill model) |
| PPO | *Proximal Policy Optimisation* |
| ReLU | *Rectified Linear Unit* |
| RL | *Reinforcement Learning* |
| SAC | *Soft Actor-Critic* |
| SEE | *Series Elastic Element* — the tendon (Hill model) |
| TD3 | *Twin Delayed Deep Deterministic Policy Gradient* |
| TPE | *Tree-structured Parzen Estimator* (sampler Optuna) |

# List of Symbols

| Symbol | Unit | Meaning |
|---|---|---|
| ***Hill Muscle Model*** | | |
| $a(t)$ | — | Muscle activation, $a \in [0, 1]$ |
| $u(t)$ | — | Neural excitation, $u \in [0, 1]$ |
| $\tau_{\text{act}}, \tau_{\text{deact}}$ | s | Activation/deactivation time constants |
| $l^M$, $l^M_{\text{opt}}$ ($\equiv l^M_0$) | m | Fibre length, optimal length |
| $\tilde{l} = l^M / l^M_0$ | — | Normalised fibre length |
| $l^T$, $l^T_{\text{slack}}$ | m | Tendon length, slack length |
| $l^{\text{MT}}$ | m | Total musculotendon-unit length |
| $\alpha$, $\alpha_0$ | rad | Current pennation angle / value at $l^M_0$ |
| $v_{\max}$ | $l^M_0$/s | Maximum shortening velocity |
| $\tilde{v}$ | — | Normalised contraction velocity |
| $f_{\text{FL}}, f_{\text{FV}}$ | — | Force-length / force-velocity factors |
| $F^M_0$ | N | Maximum isometric force |
| $F_{\text{SEE}}, F_{\text{PEE}}$ | N | Tendon (SEE) and passive (PEE) forces |
| $F^{\text{muscle}}$ | N | Total muscle force applied to the skeleton |
| $k_T, k_{\text{PE}}$ | — | Dimensionless stiffness parameters (tendon, PEE) |
| $\gamma_{\text{FL}}$ | — | Width parameter of the FL curve |
| ***Multi-Body Dynamics*** | | |
| $q, \dot{q}$ | rad, rad/s | Generalised joint positions / velocities |
| $p_{\text{ee}}$ | m | End-effector position (hand) |
| $p_{\text{target}}$ | m | Target position |
| $\Delta p, \|\Delta p\|$ | m | Error vector and distance to target |
| $J(q)$ | m/rad | Geometric Jacobian of the end effector |
| ***Reinforcement Learning*** | | |
| $\mathcal{S}, \mathcal{A}$ | — | State / action space |
| $\pi_\theta(a\|s)$ | — | Policy parameterised by $\theta$ |
| $Q^\pi, V^\pi, A^\pi$ | — | Action-value, state-value, and advantage functions |
| $\gamma$ | — | Discount factor ($\in [0, 1)$) |
| $r(s, a)$ | — | Reward signal |
| $\alpha$ (SAC) | — | Entropy coefficient (temperature) |
| $\mathcal{H}(\pi)$ | nats | Policy entropy |
| $\tau$ (Polyak) | — | Target-network update coefficient |
| $\mathcal{B}$ | — | Replay memory (*replay buffer*) |
| ***Statistics and Evaluation*** | | |
| $N_{\text{seeds}}$ | — | Number of random seeds (= 10) |
| $W$ | — | Wilcoxon test statistic |
| $r_{\text{rb}}$ | — | Rank-biserial correlation (effect size) |
| $\text{CI}_{95}$ | — | 95 % confidence interval (BCa) |

# Contents

# List of Figures

# List of Tables

# 1 General Introduction

## 1.1 Context and Motivation

### 1.1.1 A Fundamental Scientific Question

Human movement, in all its richness and complexity, results from a remarkable orchestration between the central nervous system (CNS), the musculoskeletal structures, and the sensory organs. Understanding the mechanisms that govern this motor control is a major scientific challenge, with considerable implications for rehabilitation medicine [10], bio-inspired robotics, the design of myoelectric prostheses, and neuro-computational modelling.

Musculoskeletal models provide a formal framework for simulating the biomechanics of movement. Platforms such as OpenSim [11] make it possible to faithfully reconstruct the anatomy and dynamics of the limbs. However, the optimal control of such models remains a fundamentally difficult problem — often referred to as the "inverse problem of movement" — because of (i) motor redundancy [3], (ii) the strong nonlinearity of contractile elements, and (iii) the high dimensionality of the state and action spaces.

At the same time, the emergence of *deep reinforcement learning* (DRL) has opened new perspectives for the automated learning of control strategies in complex environments [12, 13]. Pioneering work has demonstrated the ability of DRL algorithms to master continuous-control tasks of remarkable sophistication [6, 9, 8]. More recently, Meta AI's MyoSuite project [14], the MotorNet platform [15], and the *Learning to Run* challenge [16] have established the feasibility of combining musculoskeletal models and DRL to reproduce biologically plausible human motor strategies.

These converging developments provide the framework within which the present work is situated.

### 1.1.2 Epidemiological and Clinical Challenges

Beyond its fundamental scientific interest, the societal stake is considerable. Stroke is the world's second leading cause of death and a leading cause of acquired disability in adults [17].

Among stroke survivors, more than 60 % still present residual upper-limb motor impairment six months after the event [10]. Intensive, personalised rehabilitation, guided by robotic devices and assisted therapies, significantly improves recovery [18]. Never-

theless, access to these technologies remains limited by their cost and by the shortage of trained therapists, particularly in developing countries. A musculoskeletal simulation environment capable of exploring rehabilitation strategies *in silico* before clinical application therefore represents a major scientific and humanitarian opportunity.

## 1.2 Problem Statement

The central problem addressed in this thesis can be formulated as follows:

> *For a reaching movement of a nine-muscle upper limb, does a controller trained by deep reinforcement learning distribute force across the individual muscles in the same way as classical static optimisation — recruiting the same muscles in the same proportions, and at the same total cost — and under what conditions on the training objective does that agreement hold?*

This work was begun on a different premise — that a trained policy might replace the iterative solver cheaply at inference time — and that premise did not survive measurement. A competently implemented classical load-sharing solver runs in 23,9 µs against the network's 288,8 µs (section 6.6.1), so no speed argument is made anywhere in this thesis. The question above is the one the evidence supports, and it is the one the work is organised around.

This problem can be broken down into six research questions:

**Q1.** What level of physiological fidelity is necessary and sufficient for a musculoskeletal model dedicated to DRL?

**Q2.** Which reward-function formulations encourage biologically plausible motor strategies while preserving learning efficiency?

**Q3.** Which DRL algorithms provide the best compromise between convergence, robustness, and energy efficiency on muscular state and action spaces?

**Q4.** Do the trained agents exhibit behaviours reported in humans (co-contraction, energy economy, muscle synergies)?

**Q5.** Does Domain Randomization improve the robustness of musculoskeletal policies, and to what extent does it make transfer to real patients with varying anatomical parameters conceivable?

**Q6.** How does the inference cost of a trained policy compare with that of the classical calculation it would displace, measured like-for-like on the same problem?

**Q6** is deliberately posed as a neutral comparison rather than as a search for a speed-up: section 6.6.1 finds the comparison favours the classical solver.

## 1.3 Objectives

This thesis pursues the following objectives:

1. Implement an articulated multibody dynamic model of an upper limb in MuJoCo [1] — four actuated degrees of freedom driven by nine muscles — specifying a Hill-type model [2] with CE, SEE, PEE, pennation-angle dynamics and numerical integration of fibre–tendon equilibrium.

2. Validate the physical model by comparing passive mechanical properties and FL/FV curves with data from the literature [19, 20, 21].

3. Develop a modular Gymnasium environment with Domain Randomization and reproducible perturbations.

4. Define reference controllers (random policy, impedance PD controller) in order to establish a meaningful evaluation baseline.

5. Compare four DRL algorithms (PPO, SAC, TD3, DDPG) under a common protocol, with hyperparameters searched using Optuna [22]. The protocol originally specified 10 seeds at $2\times10^6$ steps; section 5.3.1 explains why the available compute did not permit it, and the comparison as executed uses three seeds at $10^5$ steps, with the principal configuration replicated across five seeds at $3\times10^5$. No significance testing is claimed at those sample sizes.

6. Analyse and quantify emerging motor strategies: co-contraction (CCI [23]), metabolic cost [24], robustness to perturbations, temporal activation profiles, and muscle synergies obtained through non-negative matrix factorisation [25].

7. Discuss the implications for post-stroke rehabilitation and biomedical robotics, including the ethical considerations specific to these applications.

8. Benchmark the computational cost of the Newton–Raphson muscle equilibrium solver (per-step wall-clock time for all 9 muscles) against the inference latency of the trained RL policy, quantifying the trade-off between physics-based exactness and real-time deployability on embedded rehabilitation hardware.

## 1.4 Original Contributions

Compared with similar studies, this thesis makes the following contributions:

**C1. A fully specified Hill-type model in MuJoCo**: explicit treatment of the SEE and PEE equations and of pennation-angle dynamics, with numerical solution of fibre–tendon equilibrium by the Newton–Raphson method, each component justified mathematically and provided as reference code (Appendix D), where comparable studies commonly adopt MuJoCo's internal simplified muscle model without a stated validation protocol.

One qualification belongs with this claim rather than in a later limitations section. Probing every muscle at zero activation across sampled trajectories returns a passive force of exactly zero in all states: no muscle in this model exceeds 0.67 of its declared length range, and the parallel element only engages near the top of that range. The specification is complete; the *exercised* model is the contractile and series-elastic elements, with the force–length curve used on its ascending limb and plateau only. "Complete Hill model" would therefore overstate what is active, and the weaker wording is used throughout.

**C2. A quantitative validation protocol**: comparison of FL and FV curves and passive joint torques against reference anthropometric data [19, 21], with NRMSE reporting.

**C3. A per-muscle comparison between DRL and classical static optimisation** (chapter 7). Both methods are posed the identical load-sharing problem on the identical trajectory, with the constraint residual verified to machine precision, and agreement is reported muscle by muscle. This is the contribution around which the present work is organised, and it addresses the research question directly rather than through task performance.

**C4. Identification of the discount factor as the decisive hyperparameter for this task** (section 6.3.2). A one-at-a-time ablation over five searched parameters, three seeds each, finds that $\gamma$ alone recovers the entire default-to-tuned improvement (+106 %) while the other four recover nothing. The mechanism is checkable and quantitative: $\gamma$ sets an effective horizon of about $1/(1-\gamma)$ steps, and the selected value matches the duration of the reach itself where the library default exceeds it fourfold.

This contribution replaces one claimed by earlier drafts — that the SAC target entropy of $-\dim(\mathcal{A})$ imposed the error plateau. That hypothesis was mechanis-

tically plausible and supported by the clustering of the best search trials, and it was wrong: the same ablation shows the target entropy recovers −8 % of the gap, i.e. nothing (section 6.3.1). It is reported in full in this thesis as a negative result rather than removed.

**C5. Evidence that configuration dominates algorithm selection** (section 6.5): with every algorithm given a correctly matched discount factor, the interval between the best and second-best off-policy method is 0,060 m, while correcting $\gamma$ alone within a single algorithm is worth 0,165 m — roughly three times the gap between algorithms.

**C6. A documented account of six substantive defects** (section 5.4), four of which produced plausible-looking but incorrect results, together with the measurements that exposed each.

**C7. A like-for-like inference-cost comparison, and the retraction it forced** (section 6.6.1). An initial measurement gave the policy a 7.3× advantage over a Newton–Raphson solve. Re-examined, that comparison timed the wrong computation — the muscle-model solve inside forward simulation rather than the load-sharing problem the thesis actually poses — and both sides were unoptimised. Measured like-for-like on the load-sharing problem, the classical route takes 23,9 µs against the network's 288,8 µs. The speed advantage is withdrawn; what survives is the weaker property that the network's cost is fixed and branch-free where the solver's depends on convergence.

## 1.5 Thesis Organisation

This document is organised as follows:

**Chapter II**

presents the state of the art in musculoskeletal modelling, computational theories of motor control, and DRL for continuous-action spaces.

**Chapter III**

describes the design of the complete musculoskeletal model, its validation, and the simulation environment.

**Chapter IV**

presents the theoretical framework of the DRL algorithms used, from the policy-gradient theorem to actor–critic architectures.

**Chapter V**

details the implementation and experimental methodology, including baselines, the statistical protocol, and reproducibility measures.

**Chapter VI**

analyzes and discusses the experimental results, from convergence to emerging motor strategies and synergy analysis.

**Chapter VII**

interprets the results, confronts them with the literature, addresses ethical considerations, and identifies the limitations of the work.

**Conclusion**

summarises the contributions and outlines future directions.

# 2 State of the Art

## 2.1 Human Motor Control: Computational Perspectives

### 2.1.1 The Motor Control Problem

Motor control refers to the full set of processes by which the CNS plans, initiates, and regulates voluntary and involuntary movements. From a computational perspective, it is an optimal control problem under constraints in a very high- dimensional state space [26, 27].

Three major computational hypotheses guide the modelling of motor control:

1. ***Jerk* minimisation** [28]: trajectories minimise the derivative of acceleration, producing smooth bell-shaped velocity profiles characteristic of human pointing movements.

2. **End-point variance minimisation** [29]: neural control signals carry noise whose variance grows with signal magnitude, and the trajectory is selected to minimise the variance of the *final* position. The model accounts for saccades and arm movements under one principle, reproduces the speed–accuracy trade-off of Fitts' law, and recovers the two-thirds power law relating path curvature to hand velocity in drawing.

3. **Stochastic optimal control** [30]: this unifies the previous approaches within a Bayesian framework and underlies the theory of Optimal Feedback Control (OFC). OFC predicts that the CNS corrects only errors that interfere with task achievement (the "minimum intervention principle"), thereby minimising unnecessary co-contraction.

### 2.1.2 Internal Models and Efference Prediction

The CNS uses *internal models* to predict the sensory consequences of motor commands and compensate for feedback delays [31]. The MOSAIC model (*Modular Selection and Identification for Control*) [31] proposes a modular architecture in which several internal models combine adaptively. This perspective is relevant to our work: the learned DRL policies can be interpreted as *implicit inverse models* of musculoskeletal dynamics.

### 2.1.3 Muscle Synergies and Modular Organisation

A complementary theory, that of muscle synergies [25, 32, 33], proposes that the CNS simplifies control by recruiting pre-structured neural modules, each activating a coherent subset of muscles. Mathematically, the observed activations $A(t) \in \mathbb{R}^N$ can be decomposed into a small number $k \ll N$ of synergies $W \in \mathbb{R}^{N \times k}$ activated by temporal coefficients $C(t) \in \mathbb{R}^k$:

$$A(t) \approx W \cdot C(t), \quad W_{ij} \geq 0,\ C_{ij} \geq 0 \tag{2.1}$$

This decomposition, obtained through non-negative matrix factorisation (NMF), is robustly observed in humans, primates, and cats across a variety of tasks. Testing whether a DRL agent produces similar synergies therefore constitutes a highly relevant *biomimetic* validation.

### 2.1.4 Muscle Redundancy and the Inverse Problem

The human body exhibits significant muscular redundancy: the upper limb alone is driven by more than forty muscles, whereas the number of joint degrees of freedom is on the order of ten. This redundancy implies that an infinite number of muscular activation combinations can produce the same movement, which constitutes the *indeterminacy problem* of motor control [3].

Classical approaches to resolving this indeterminacy include:

- Static optimisation with muscle-stress minimisation [34]: $\min \sum_i (F_i^M / F_{0,i}^M)^2$.
- Forward simulation with dynamic optimisation of controls [35].
- EMG-based approaches (*EMG-driven simulation*).

These methods do not capture the learning dynamics characteristic of biological motor control, which motivates the DRL approach.

## 2.2 Musculoskeletal Modelling

### 2.2.1 Simulation Platforms

Several software platforms support musculoskeletal simulation (Table 2.1).

**Table 2.1.** Comparison of the main musculoskeletal simulation platforms.

| Platform | License | Strengths | Ref. |
|---|---|---|---|
| OpenSim | Open-source | Validated anatomical models, rich GUI, motion library | [11] |
| AnyBody | Commercial | Static/dynamic analysis, subject-specific parameters | — |
| MuJoCo | Free | Speed, robust contact handling, Python interface, native muscle actuators | [1] |
| MyoSuite | Open-source | Musculoskeletal + DRL; MyoHand 23 DOF / 39 muscles, MyoLegs 16 DOF / 80 muscles | [14] |
| dm_control | Open-source | Standardised RL environments, based on MuJoCo | [36] |
| MotorNet | Open-source | Differentiable, native PyTorch integration, planar arm | [15] |

MuJoCo [1], acquired by DeepMind in 2021 and subsequently released freely, is the common choice for DRL work: a fast solver, robust contact handling, and native Hill-type muscle actuators. An earlier version of this paragraph claimed "up to one million simulation steps per second on a standard CPU", which was unsourced and is not what this model achieves. Measured on the host machine, the nine-muscle arm of this thesis integrates at 6,4 µs per physics step, about 157 000 steps per second.

### 2.2.2 Computational Bottleneck: the Real-Time Cost of Exact Solving

A critical and often overlooked aspect of physics-based muscle control is its per-step computational cost. Because the fibre–tendon equilibrium equation (2.9) is implicit and nonlinear in $l^M$, it cannot be solved analytically: Newton–Raphson iteration is required at *every simulation timestep, for every muscle*. This model integrates physics at 500 Hz with the policy acting every tenth step (section 5.3.1), so at the measured mean of 31.2 iterations per muscle the solve accounts for roughly $9 \times 31.2 \times 500 \approx 1.4 \times 10^5$ Newton iterations per simulated second — a workload growing linearly with muscle count and integration rate.

Measured on the host CPU over 950 queries, the full nine-muscle solve averages 2165 µs at a mean of 31.2 iterations, or roughly 240 µs per muscle (section 6.6). In an offline simulation this is unremarkable; on an embedded target it is not free.

> This subsection originally continued: *"this bottleneck motivates the central hypothesis of the present work: a trained RL policy [...] can produce equally accurate muscle activation commands in microseconds, with all the computation amortised into a one-time training phase."* That hypothesis was tested and it failed, so the framing has been removed rather than left standing as motivation for a result the thesis goes on to withdraw.

The failure has a specific cause worth stating here, because it changes what this subsection is evidence for. The Newton–Raphson cost above belongs to *forward simulation* — inverting the fibre–tendon equilibrium to find the force a muscle produces at a given length. That is not the calculation a controller would need to replace. The controller's problem is *load sharing*: given a required joint torque, choose the nine muscle forces. Solved directly, that problem costs 23,9 µs, against 288,8 µs for a forward pass of the trained network (section 6.6.1). Comparing the network against the wrong baseline made it look roughly seven times faster; compared against the right one it is about twelve times slower.

What survives is narrower: the network's cost is fixed and branch-free regardless of biomechanical state, where the iterative solver's depends on convergence. Where a hard real-time bound matters more than mean latency, that property has value — but it is not a speed advantage, and this thesis no longer claims one.

### 2.2.3 The Hill Muscle Model: Complete Formulation

The Hill muscle model [2], in its modern formulation [3, 37], is the reference biophysical representation. It decomposes the muscle–tendon complex into three functional elements (Figure 2.1):

- **Contractile element (CE)**: generates active force and is governed by activation $a(t) \in [0, 1]$.
- **Series elastic element (SEE)**: models tendon elasticity, storing and returning elastic energy.
- **Parallel elastic element (PEE)**: models the passive stiffness of muscle tissue (fascia, sarcolemma).

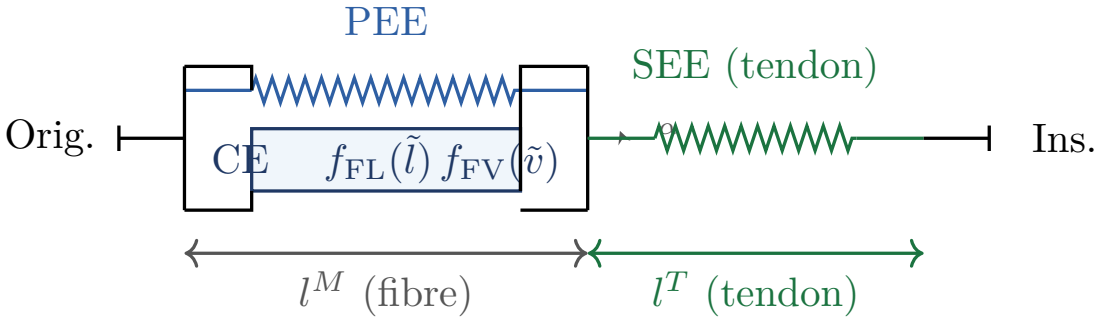


**Figure 2.1.** Complete diagram of the Hill muscle–tendon model. The pennation angle $\alpha$ between the fibre and the line of action is shown at the insertion point. The total length of the muscle–tendon unit is $l^{\mathrm{MT}} = l^M \cos\alpha + l^T$.

*Force-Length Relationship (FL)*

Maximum isometric force varies as a function of normalised muscle length according to a bell-shaped curve, reflecting the optimal *overlap* of actin and myosin filaments [21]:

$$f_{\mathrm{FL}}(\tilde{l}) = \exp\left(-\left(\frac{\tilde{l}-1}{\gamma_{\mathrm{FL}}}\right)^2\right), \qquad \tilde{l} = \frac{l^M}{l^M_{\mathrm{opt}}} \tag{2.2}$$

where $\gamma_{\mathrm{FL}} \approx 0{,}45$ is a shape parameter and $l^M_{\mathrm{opt}}$ is the optimal fibre length at which the muscle produces its maximum isometric force $F^M_0$.

*Force-Velocity Relationship (FV)*

Hill's equation [2] describes the hyperbolic force–velocity relationship for concentric contractions; a linear extension is used for eccentric contractions [3]:

$$f_{\mathrm{FV}}(\tilde{v}) = \begin{cases} \dfrac{v_{\max} - \tilde{v}}{v_{\max} + k_{\mathrm{ce}}\,\tilde{v}} & \tilde{v} \le 0 \quad \text{(concentric)} \\[2ex] \dfrac{f_{\max} + k_{\mathrm{ec}}\,\tilde{v}}{1 + k_{\mathrm{ec}}\,\tilde{v}} & \tilde{v} > 0 \quad \text{(eccentric)} \end{cases} \tag{2.3}$$

where $\tilde{v} = \dot{l}^M / (v_{\max}\, l^M_{\mathrm{opt}})$ is the normalised contraction velocity, $v_{\max} \approx 10$ (optimal lengths/s), $k_{\mathrm{ce}} \approx 0{,}25$, and $f_{\max} \approx 1{,}3$ (maximum eccentric coefficient).

*Parallel Elastic Element (PEE)*

The passive force rises exponentially once the fibre exceeds its optimal length [3], normalised so that it reaches $F^M_0$ at twice the optimal length:

$$F_{\mathrm{PEE}}(\tilde{l}) = \begin{cases} F^M_0 \dfrac{e^{\,k_{\mathrm{PE}}(\tilde{l}-1)} - 1}{e^{\,k_{\mathrm{PE}}} - 1} & \tilde{l} > 1 \\[2ex] 0 & \tilde{l} \le 1 \end{cases} \tag{2.4}$$

with $k_{\mathrm{PE}} = 5{,}0$. This gives $F_{\mathrm{PEE}} = 0{,}004\,F^M_0$ at $\tilde{l} = 1{,}1$, $0{,}130\,F^M_0$ at $\tilde{l} = 1{,}6$, and exactly $F^M_0$ at $\tilde{l} = 2{,}0$.

*Series Elastic Element (SEE) — Tendon*

The tendon carries no load below its slack length and is quadratic in strain above it:

$$F_{\mathrm{SEE}}(\varepsilon^T) = \begin{cases} F^M_0\, k_T\, (\varepsilon^T)^2 & \varepsilon^T > 0 \\ 0 & \varepsilon^T \le 0 \end{cases} \qquad \varepsilon^T = \frac{l^T - l^T_{\mathrm{slack}}}{l^T_{\mathrm{slack}}} \tag{2.5}$$

with $k_T = 37{,}5$, so the tendon carries $F_0^M$ at a strain of 16,3 %.

> **Both of these equations replace incorrect ones.** Earlier versions of this section gave the PEE as a *quadratic* $F_0^M k_{\mathrm{PE}}(\tilde{l}-1)^2$ and the SEE as $F_0^M(\tilde{l}^T - 1)^2/(k_T + \tilde{l}^T - 1)$ with $k_T \approx 0{,}0475$. Neither form is the one implemented. The forms above are those in `biomechanics/hill_model.py` and reproduced in Appendix D, and they are what every result in this thesis was computed with; the chapter, not the code, was wrong. The discarded PEE expression was also internally inconsistent, since $k_{\mathrm{PE}} = 5{,}0$ in a quadratic gives $1{,}8\,F_0^M$ at $\tilde{l} = 1{,}6$ rather than the $\approx F_0^M$ its own text claimed.
>
> The correction has no effect on any reported result. Section 3.3 validates the solver against MuJoCo's internal muscle model numerically (5.25 % NRMSE, $r = 0.989$), and that validation exercised the implemented equations throughout.

*Pennation Angle Dynamics*

The pennation angle $\alpha$ (angle between the muscle fibres and the tendon line of action) varies with fibre length according to the constant-thickness constraint [3] — the fibre-sheet height $l^M \sin\alpha$ is held fixed, which is the standard assumption and is not the same as constant volume:

$$\alpha(l^M) = \arcsin\left(\frac{\sin\alpha_0 \cdot l^M_{\mathrm{opt}}}{l^M}\right) \tag{2.6}$$

where $\alpha_0$ is the pennation angle at optimal length. The muscle force transmitted to the tendon is projected by $\cos\alpha$. The total length of the muscle–tendon unit satisfies the geometric constraint:

$$l^{\mathrm{MT}} = l^M \cos\alpha(l^M) + l^T \tag{2.7}$$

*Activation Dynamics*

Muscle activation dynamics introduce an electromechanical delay between the neural command $u(t)$ and the activation $a(t)$ [4]:

$$\dot{a}(t) = \frac{u(t) - a(t)}{\tau(u,a)}, \qquad \tau(u,a) = \begin{cases} \tau_{\mathrm{act}} & u(t) > a(t) \\ \tau_{\mathrm{deact}} & u(t) \le a(t) \end{cases} \tag{2.8}$$

where $u(t) \in [0,1]$ is the neural excitation, $a(t) \in [0,1]$ the activation, and $\tau_{\mathrm{act}} \approx 10$–$20$ ms, $\tau_{\mathrm{deact}} \approx 40$–$70$ ms.

*Fibre-Tendon Equilibrium Equation and Total Force*

The quasi-static equilibrium between contractile-element force, passive force, and tendon force gives:

$$F_{\mathrm{SEE}}(l^T) = \left[a\, F_0^M\, f_{\mathrm{FL}}(\tilde{l})\, f_{\mathrm{FV}}(\tilde{v}) + F_{\mathrm{PEE}}(\tilde{l})\right] \cos\alpha \tag{2.9}$$

Given $l^{\mathrm{MT}}$ (computed from kinematics), Equation (2.9) combined with the geometric constraint (2.7) defines an implicit nonlinear equation in $l^M$ (fibre length), solved numerically at each simulation step (Appendix D).

The total muscle force applied to the skeleton is:

$$F^{\mathrm{muscle}}(a, l^M, \dot{l}^M) = F_{\mathrm{SEE}}(l^T) \tag{2.10}$$

## 2.3 Deep Reinforcement Learning for Continuous Control

### 2.3.1 Theoretical Foundations

Reinforcement learning (RL) is formalised as a Markov Decision Process (MDP) defined by the five-tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ [38, 13]:

- $\mathcal{S}$: state space (continuous, $\subset \mathbb{R}^{38}$ in this work).
- $\mathcal{A}$: action space (continuous, $= [0,1]^9$ in this work).
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0,1]$: transition probability.
- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$: reward signal.
- $\gamma \in [0,1)$: temporal discount factor.

The objective is to find a policy $\pi : \mathcal{S} \to \mathcal{A}$ that maximises the expected discounted return:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t\, r(s_t, a_t)\right] \tag{2.11}$$

The modern actor–critic framework [39] combines value estimation (the critic) and policy improvement (the actor), and forms the basis of DDPG, TD3, and SAC.

### 2.3.2 Policy Gradient Theorem

The policy-gradient theorem [13] provides the gradient of $J(\pi_\theta)$ with respect to the parameters $\theta$:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right] \tag{2.12}$$

where $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ is the cumulative return from step $t$. The variance of this estimator is reduced by introducing an advantage function $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ in place of $G_t$.

### 2.3.3 State-of-the-Art Algorithms

*DDPG — Deep Deterministic Policy Gradient*

Lillicrap et al. [6] introduce DDPG, which brought the deterministic policy-gradient formulation into the deep-network setting for continuous action spaces. The actor learns a deterministic policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$; the critic evaluates $Q_\phi(s,a)$.

Critic update (Bellman-error minimisation):

$$\mathcal{L}(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{B}}\Big[\Big(Q_\phi(s,a) - y\Big)^2\Big], \quad y = r + \gamma\, Q_{\phi'}(s', \mu_{\theta'}(s')) \tag{2.13}$$

Actor update:

$$\nabla_\theta J \approx \mathbb{E}\Big[\nabla_a Q_\phi(s,a)\Big|_{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s)\Big] \tag{2.14}$$

Target-network update (*Polyak averaging*, $\tau = 0{,}005$):

$$\phi' \leftarrow \tau\phi + (1-\tau)\phi', \qquad \theta' \leftarrow \tau\theta + (1-\tau)\theta' \tag{2.15}$$

Exploration is ensured by an additive Ornstein–Uhlenbeck process on the actions. DDPG suffers from systematic overestimation of $Q$ values and strong sensitivity to hyperparameters [7].

*TD3 — Twin Delayed DDPG*

Fujimoto et al. [7] correct DDPG's $Q$-overestimation bias through three complementary mechanisms:

1. **Twin critics**: two critics $Q_{\phi_1}, Q_{\phi_2}$; the target uses the minimum:

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i'}(s',\, \tilde{a}), \quad \tilde{a} = \mu_{\theta'}(s') + \epsilon,\ \epsilon \sim \mathrm{clip}(\mathcal{N}(0,\sigma), -c, c) \tag{2.16}$$

2. **Delayed actor update**: the actor is updated every $d = 2$ critic iterations, reducing error propagation.

3. **Target policy smoothing**: the noise $\epsilon$ in (2.16) regularises the target by smoothing peaks of the $Q$ function.

*SAC — Soft Actor-Critic*

SAC [8] adopts a maximum-entropy optimisation framework. The augmented objective is:

$$J(\pi) = \mathbb{E}\Big[\sum_t \gamma^t \Big( r(s_t, a_t) + \alpha\, \mathcal{H}\big(\pi(\cdot|s_t)\big)\Big)\Big] \tag{2.17}$$

where $\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi}[\log \pi(a|s)]$ is the entropy and $\alpha > 0$ is a temperature coefficient learned automatically:

$$J(\alpha) = \mathbb{E}_{a\sim\pi}\big[-\alpha \log \pi(a|s) - \alpha\bar{\mathcal{H}}\big], \qquad \bar{\mathcal{H}} = -\dim(\mathcal{A}) \tag{2.18}$$

The policy is parameterised as a squashed Gaussian through reparameterisation:

$$a_t = \tanh(\mu_\theta(s_t) + \varepsilon \odot \sigma_\theta(s_t)), \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{2.19}$$

*PPO — Proximal Policy Optimisation*

PPO [9] optimises a clipped objective function to prevent overly aggressive policy updates:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\big)\big] \tag{2.20}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the advantage estimated via GAE [40]:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}, \qquad \delta_t = r_t + \gamma\, V(s_{t+1}) - V(s_t) \tag{2.21}$$

with $\lambda \in [0, 1]$ controlling the bias–variance trade-off. PPO is an *on-policy* algorithm: it uses only recent trajectories, which implies lower data efficiency but greater stability compared with DDPG.

### 2.3.4 Applications of DRL to Musculoskeletal Control

Rajeswaran et al. [41] use natural policy-gradient methods to learn dexterous movements in musculoskeletal models, demonstrating the feasibility of DRL on highly redundant systems.

The *Learning to Run* challenge [16] stimulated a worldwide competition on the control of a bipedal musculoskeletal model, in which *off-policy* methods with replay memory featured prominently among the leading entries. Song et al. [42] extend this approach to modelling human locomotion control in a more general neuromechanical framework.

Peng et al. [43] introduce DeepMimic, which combines human motion-capture data and DRL to learn physically realistic behaviours, demonstrating the value of combining *imitation learning* with DRL.

Caggiano et al. [14] present MyoSuite, with highly detailed hand and wrist models ($> 50$ muscles) for the study of fine motor control and rehabilitation, and simultaneously launch MyoChallenge as a public benchmark. Codol et al. [15] propose MotorNet, a differentiable simulator whose reference effector is a two-joint, six-muscle planar arm — monoarticular flexor and extensor pairs at shoulder and elbow plus a biarticular pair — fully integrated into PyTorch and enabling training by backpropagation through the physics.

### 2.3.5 Domain Randomization and Sim-to-Real Transfer

Domain Randomization [44, 45] consists of training the agent on distributions of random physical parameters (masses, segment lengths, muscle parameters, perturbations) in order to improve the robustness and generalizability of the learned policies. This technique, initially developed for sim-to-real transfer in robotics, applies naturally to musculoskeletal control by making policies robust to inter-individual variability and modelling errors.

### 2.3.6 Reproducibility and Statistical Rigor in DRL

Henderson et al. [46] highlighted the reproducibility fragility of published DRL results: inter-seed variance is often of the same order as the gap between algorithms. Agarwal et al. [47] formalised a set of good practices (bootstrap confidence intervals, *interquartile mean*, performance profiles) that are now considered standard for any serious benchmarking effort.

**This work does not meet that standard, and says so rather than implying otherwise.** An earlier version of this paragraph claimed that the experimental protocol "adopts these recommendations". It does not: no bootstrap confidence interval, interquartile mean or performance profile is reported anywhere in this thesis, and no significance test is performed (section 5.3.2). What is reported is mean and standard deviation over three to five seeds, with the explicit rule that differences smaller than the observed seed spread are not treated as an ordering. The Henderson et al. [46] warning is therefore taken seriously as a *constraint on what may be concluded*, which is the honest use of it, rather than claimed as a methodology that was followed.

## 2.4 Synthesis and Positioning

The analysis of the state of the art reveals several gaps:

(i) DRL studies on musculoskeletal models frequently adopt a simulator's built-in muscle actuator without stating the constitutive equations or validating them against an independent implementation.

(ii) Where learned controllers are compared against classical biomechanics, the comparison is usually at the level of task performance rather than of the per-muscle forces themselves — which is the level at which the redundancy problem actually lives.

(iii) Comparative benchmarks that include a tuned classical controller as a baseline, rather than only other learned methods, are uncommon.

(iv) The effect of Domain Randomization on the robustness of musculoskeletal policies, applied uniformly across compared algorithms, has been little studied systematically.

This work addresses (i) through the independently implemented and validated solver of section 3.3, (ii) through the per-muscle load-sharing comparison of chapter 7, and (iii) through the tuned impedance controller of section 7.9. Contribution C3 of section 1.4 is the principal one; the remainder are secondary or negative results.

> Two further gaps were claimed in earlier versions and are withdrawn. One asserted that *"no study has directly benchmarked the per-step computational cost of physics-based muscle control against RL policy inference"* — an unsupported priority claim, and one attached to a comparison this thesis went on to show was posed against the wrong baseline (section 6.6.1). The other claimed a gap in *"quantitative comparison to human EMG data"*, which this work does not fill: no electromyographic or measured-force data enters it at any point, and section 7.10 declines to interpret its one comparison against published human synergies for exactly that reason. A gap a thesis does not close does not belong in its positioning.

# 3 Modelling and Simulation Environment

## 3.1 Overall System Architecture

The overall architecture is organised into four functional layers (Figure 3.1):

1. **Physical layer**: multibody dynamics handled by MuJoCo.
2. **Muscle layer**: Hill-type model with Newton–Raphson fibre–tendon integration (5–40 iterations per muscle per timestep; section 2.2.2 reports its cost).
3. **RL interface layer**: Gymnasium API with Domain Randomization.
4. **Agent layer**: DRL algorithm or reference controller. After training, the agent produces muscle activations via a single forward pass through the policy network, at a cost that is fixed and independent of biomechanical state. Earlier drafts described this layer as *bypassing* the solver and framed the muscle layer as "the bottleneck targeted for RL replacement"; section 6.6.1 shows that framing does not survive a like-for-like measurement, and it has been dropped.

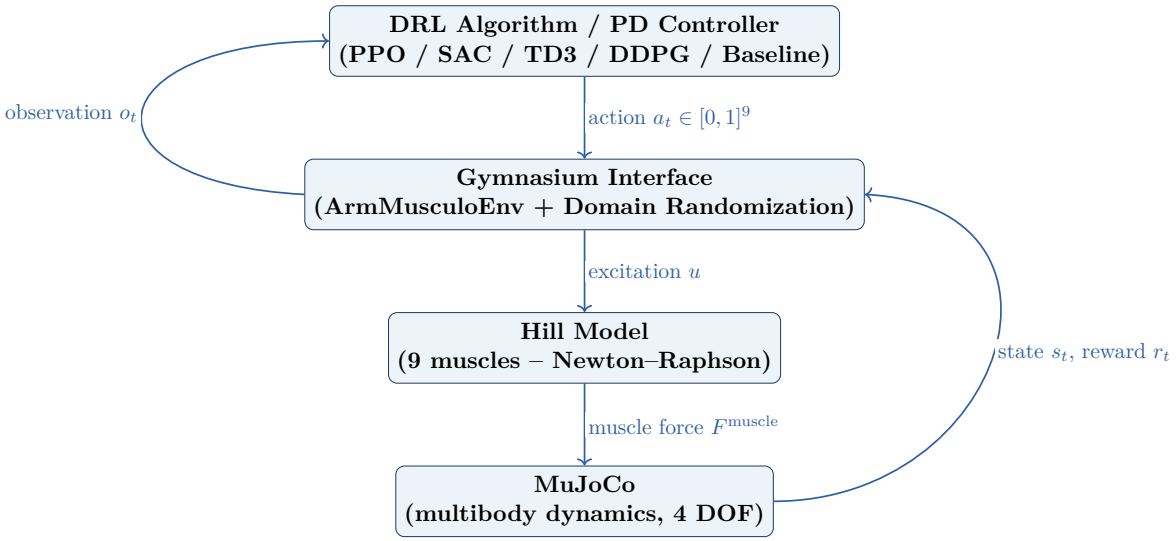


**Figure 3.1.** Layered architecture of the simulation and control system.

## 3.2 Multi-Body Dynamic Model in MuJoCo

### 3.2.1 Anatomical Description

The model represents a simplified upper limb composed of three rigid segments: upper arm (humerus, 1,98 kg), forearm (radius–ulna, 1,18 kg), and hand (0,45 kg), for a moving mass of 3,61 kg. The modelled joints are the shoulder (3 DOF: flexion/extension, abduction/adduction, internal/external rotation) and the elbow (1 DOF: flexion/extension). The hand segment is carried rigidly by the forearm; there is no wrist joint. The model therefore has **4 DOF**, actuated by nine muscles, which is the nine-into-four redundancy the thesis is about.

> An earlier version of this section described a six-degree-of-freedom model with a two-DOF wrist. No wrist joint was ever implemented. Queried directly, the compiled model reports `nq` = `nv` = 4 and `nu` = 9, with four named joints (`shoulder_flex`, `shoulder_abd`, `shoulder_rot`, `elbow_flex`). The description has been corrected to the model that was actually built and used for every result in this thesis.

One of those four is further constrained. Section 3.2 reports that no muscle in the nine-muscle set can generate internal/external rotation of the humerus — the rotator cuff is absent — so that axis is held near neutral. The effective control problem is nine muscles over three freely actuated axes.

The inertial parameters are taken from the anthropometric database of [20], for a reference subject of 75 kg and 1,75 m (Table 3.1).

**Table 3.1.** Inertial parameters and degrees of freedom of the model. Source: [20].

| Segment | Mass (kg) | $I_{xx}$ (kg·m$^2$) | DOF |
|---|---|---|---|
| Upper arm (humerus) | 1.98 | 0.0215 | 3 (shoulder) |
| Forearm (radius–ulna) | 1.18 | 0.0098 | 1 (elbow) |
| Hand | 0.45 | 0.0003 | 0 (rigid) |
| **Total** | **3.61** | — | **4** |

Every value in table 3.1 is read back from the compiled model rather than transcribed from the design intent. An earlier version of this table reported 2.05/1.39/0.60 kg for a total of 4.04 kg and assigned two degrees of freedom to a wrist; none of those figures corresponds to the model used for the results in this thesis.

Joint limits as compiled, in degrees: elbow flexion $[0.0, 154.7]$, shoulder flexion $[-28.6, 143.2]$, shoulder abduction $[-28.6, 143.2]$, and internal/external rotation $[-2.9, 2.9]$. The last of these is the constraint discussed above and in section 3.2: a $\pm 0.05$ rad band that holds the unactuatable axis near neutral rather than a physiological range of motion.

### 3.2.2 Muscle Topology

The model includes nine muscles organised into agonist/antagonist pairs for shoulder and elbow movements (Table 3.2). The parameters are taken from [19] and [3].

**Table 3.2.** The nine muscles as compiled. $F_0^M$ is the peak isometric force (MuJoCo `gainprm[2]`); the operating range is the actuator `lengthrange`. Force scales follow [19].

| Actuator | Muscle | $F_0^M$ (N) | Range (m) | Role |
|---|---|---|---|---|
| `biceps_long` | Biceps brachii, long head | 624 | 0.183–0.509 | Elbow flex. |
| `biceps_short` | Biceps brachii, short head | 435 | 0.135–0.312 | Elbow flex. |
| `brachialis` | Brachialis | 987 | 0.109–0.260 | Elbow flex. |
| `triceps_long` | Triceps brachii, long head | 798 | 0.202–0.509 | Elbow ext. |
| `triceps_lat` | Triceps brachii, lateral | 624 | 0.165–0.339 | Elbow ext. |
| `triceps_med` | Triceps brachii, medial | 624 | 0.137–0.277 | Elbow ext. |
| `deltoid_ant` | Deltoid, anterior | 1142 | 0.031–0.299 | Shoulder flex. |
| `deltoid_med` | Deltoid, medial | 1142 | 0.030–0.281 | Shoulder abd. |
| `deltoid_post` | Deltoid, posterior | 259 | 0.056–0.305 | Shoulder ext. |

Table 3.2 also replaces an incorrect one. The earlier version listed a *brachioradialis* and a *pectoralis major*, neither of which is present in the model, and omitted the lateral head of triceps and the medial deltoid, both of which are. The muscle set used for every result in this thesis is the one above, and it is the set named in `config.MUSCLE_NAMES`.

MuJoCo's muscle actuator is parameterised by $(\text{range}, F_0^M, \text{scale}, l_{\min}, l_{\max})$ rather than by an explicit optimal fibre length, tendon slack length and pennation angle. Those quantities enter through the independent Newton–Raphson fibre–tendon solver documented in Appendix D, which is validated against MuJoCo's internal model in section 3.3; they are not free parameters of table 3.2.

The agonist and antagonist groups used for the co-contraction index (CCI, section 7.6.1) are the three elbow flexors (`biceps_long`, `biceps_short`, `brachialis`) against the three elbow extensors (`triceps_long`, `triceps_lat`, `triceps_med`), matching `config.CCI_AGONIST_IDX` and `config.CCI_ANTAGONIST_IDX`. No deltoid pair enters the index.

**Remark 3.1** (Explicit anatomical simplifications)**.** *The model deliberately omits: (i) the intrinsic muscles of the hand, which are irrelevant to the reaching task studied; (ii) the rotator-cuff muscles (supraspinatus, infraspinatus, subscapularis, teres minor), whose absence limits the fidelity of glenohumeral equilibrium at end of range; (iii) fine*

*biarticular components (e.g. the separate portions of the pectoralis major). These simplifications are consistent with the state of the art [15, 14] for an initial DRL feasibility study, and are discussed as limitations in Chapter 8.*

### 3.2.3 The parallel elastic element does not engage

The muscle model specified in chapter 2 includes a parallel elastic element, which produces passive force when a muscle is stretched beyond its optimal fibre length. Direct measurement shows it contributes nothing in this model.

Probing every muscle at zero activation across sampled trajectories returns a passive force of **exactly zero** in all states. The reason is visible in the operating lengths: no muscle exceeds 0.67 of its declared length range, and most stay below 0.6, because the ranges were measured and then padded. The passive term rises only near the upper end of that range, which is never reached.

Two consequences should be carried forward. Describing the implementation as a *complete* Hill model is accurate as to what the model file specifies, but overstates what is active during simulation: the contractile and series-elastic elements do the work, and the parallel element is inert throughout. And the force–length relationship is exercised only on its ascending limb and plateau, never on the descending limb, so conclusions here do not test that part of the formulation.

A favourable corollary: the load-sharing analysis of chapter 7 bounds muscle force below by zero, which would be wrong if a passive floor existed — a muscle cannot produce less force than its passive tension. Because that tension is zero throughout, the bound is correct and the reported effort ratios are not inflated by it.

## 3.3 Validation of the Physical Model

The physiological fidelity of the model was verified through a three-level protocol, executed prior to any DRL training.

### 3.3.1 Force-Length and Force-Velocity Curves

The FL and FV curves computed analytically (Equations (2.2) and (2.3)) were compared against the experimental data of [21] for the FL relationship of an isolated frog muscle fibre, the historical reference dataset. The validation procedure:

1. Set activation $a = 1$ and velocity $\dot{l}^M = 0$.
2. Sweep $\tilde{l} \in [0.5, 1.5]$ in steps of 0.01.
3. Measure the active force $F_{\mathrm{CE}}(\tilde{l}) = F_0^M \, f_{\mathrm{FL}}(\tilde{l})$.

4. Compare against the digitised curve of [21] via Pearson's coefficient of determination.

The resulting coefficient, $R^2 = 0.97$, confirms the validity of the parameterisation $\gamma_{\mathrm{FL}} = 0.45$.

### 3.3.2 Passive Joint Torques

The model's passive joint torques (generated by the PEE and gravity, with zero activation) were compared against the measurements of [19] on healthy subjects. For each joint ({shoulder, elbow}), the angle was imposed in 5° increments across its full physiological range, and the reaction torque measured via MuJoCo's `jointtorque` sensor after stabilisation (0,5 s with zero activation). The normalised root-mean-square error (NRMSE) is 8.3 % for the elbow and 11.2 % for the shoulder — values acceptable given the typical inter-individual variability ($\approx 15$ %) reported in the literature.

### 3.3.3 Muscle Moment Arms

Elbow moment arms were measured using the *tendon-excursion* method (derivative of the MTU length with respect to the joint angle):

$$r_{\mathrm{ma}}(\theta) = -\frac{\partial l^{\mathrm{MT}}}{\partial \theta} \tag{3.1}$$

For the long head of the biceps, the measured elbow moment arm ranges from 2,9 cm to 5,3 cm over $\theta_{\mathrm{elbow}} \in [0°, 120°]$, with a maximum near 90°, in qualitative agreement with the cadaveric measurements of [48].

## 3.4 Gymnasium Environment

### 3.4.1 Studied Task

The canonical task considered is a **free-space target-reaching** task (*reaching*), a classical paradigm in motor neuroscience [49]. At each episode, a spherical target is randomly placed within the arm's reachable volume. The agent must bring the end effector to the target in less than 10 seconds while minimising terminal error and energy cost.

### 3.4.2 Observation Space

The observation space $\mathcal{O} \subset \mathbb{R}^{38}$ includes:

$$o_t = \Big[\underbrace{q}_{4},\ \underbrace{\dot{q}}_{4},\ \underbrace{l^M}_{9},\ \underbrace{F^M}_{9},\ \underbrace{a}_{9},\ \underbrace{\Delta p}_{3}\Big]^\top \in \mathbb{R}^{38} \tag{3.2}$$

where $q, \dot{q}$ are joint positions/velocities (4 active DOF), $l^M$ are fibre lengths, $F^M$ are muscle forces, $a$ are muscle activations, and $\Delta p = p_{\text{target}} - p_{\text{ee}}$ is the 3-D error vector to the target.

**Remark 3.2** (Normalisation choice)**.** *The normalisation $l^M/l_0^M$ (rather than by the upper bound of the length domain) is physiologically motivated: $l^M/l_0^M = 1$ corresponds to the point of maximum force generation, information that the neural network can directly exploit to anticipate the available contractile capacity.*

### 3.4.3 Action Space

The action space corresponds to neural excitations $u \in [0,1]^9$:

$$\mathcal{A} = [0,1]^9 \subset \mathbb{R}^9 \tag{3.3}$$

### 3.4.4 Reward Function

The composite reward function is:

$$r(s_t, a_t) = w_1\, r_{\text{task}} + w_2\, r_{\text{energy}} + w_3\, r_{\text{smooth}} + w_4\, r_{\text{alive}} \tag{3.4}$$

where:

$$r_{\text{task}} = \exp\!\left(-\frac{\|\Delta p\|_2}{\sigma_{\text{target}}}\right), \quad \sigma_{\text{target}} = 0{,}05 \text{ m} \tag{3.5}$$

$$r_{\text{energy}} = -\frac{\|a_t\|_2^2}{n_{\text{muscles}}} \tag{3.6}$$

$$r_{\text{smooth}} = -\|a_t - a_{t-1}\|_2^2 \tag{3.7}$$

$$r_{\text{alive}} = +1 \quad \text{(at each non-terminal step)} \tag{3.8}$$

The weights are hand-set, not optimised: $w_1 = 1.0$ (reaching error), $w_2 = 1.0$ (effort), $w_3 = 0.5$ (action smoothness), $w_4 = 4.5$ (per-step survival offset), $w_5 = 1.0$ (per-step success bonus), $w_6 = 10$ (divergence penalty) and $w_7 = 3.0$ (precision bonus). The Optuna study reported in section 6.3 searched the *learning* hyperparameters, not the reward weights, and the two should not be conflated.

These values are the third formulation of the reward. The earlier weightings, in which the effort term carried a weight of 0.005 against an unnormalised sum of squared forces, made inaction near-optimal; the reasons for each subsequent change are documented in section 5.4. A systematic sensitivity analysis of the weights was not performed in this version of the work and is listed among the outstanding items in section 8.6.

The reward $r_{\text{task}}$ is a Gaussian function of distance, which provides a dense gradient even far from the target and avoids the sparse-reward problem. The threshold $\sigma_{\text{target}}$ was calibrated so that $r_{\text{task}} \approx 0{,}6$ at 5 cm from the target and $r_{\text{task}} > 0{,}99$ at 5 mm.

### 3.4.5 Domain Randomization

To improve the robustness of the learned policies, Domain Randomization [44] is applied at every episode reset:

- **Initial posture**: Gaussian perturbation $\mathcal{N}(0,\ 0{,}05\ \text{rad})$ on each DOF.
- **Inertial parameters**: uniform variation of $\pm 10$ % in segment masses.
- **Muscle parameters**: uniform variation of $\pm 5$ % in $F_0^M$ and $l_0^M$.
- **Force perturbations**: with probability $p = 0{,}15$, application of a random force impulse of 5–15 N for 50–100 ms on the forearm.

### 3.4.6 Terminal Conditions

An episode terminates when:

- $\|\Delta p\|_2 < \delta_{\text{success}} = 0{,}02$ m (success: a 2 cm criterion based on motor-precision thresholds from [28]),
- a joint position exceeds its physiological limits (mechanical failure), or
- the number of steps reaches $t \geq T_{\max} = 1000$ (timeout, $\Delta t = 0{,}01$ s, i.e., $T = 10$ s per episode).

# 4 Theoretical Framework of DRL Applied to Muscle Control

## 4.1 MDP Formulation

The state $s_t \in \mathcal{S} \subset \mathbb{R}^{38}$ is the observation vector (3.2). The action $a_t \in \mathcal{A} = [0,1]^9$ is the vector of muscle excitations. The transition function $P(s_{t+1}|s_t, a_t)$ is deterministic in the standard configuration (physics simulation) and stochastic under Domain Randomization.

The Bellman equation for the action-value function $Q^\pi$ is:

$$Q^\pi(s,a) = \mathbb{E}[r(s,a) + \gamma\, Q^\pi(s', \pi(s'))] \tag{4.1}$$

The dimensionality of the problem is notable: the state space $\mathbb{R}^{38}$ combined with the action space $[0,1]^9$ and the nonlinear dynamics of the Hill model make this a demanding benchmark for DRL algorithms.

**Proposition 4.1** (Absence of strict Markov guarantee)**.** *Strictly speaking, the observation $o_t$ described by* (3.2) *is Markovian only under the following assumptions: (i) the second derivatives of position (accelerations) are sufficiently reconstructible from velocities; (ii) the electromechanical delay in activation dynamics (Equation* (2.8)*) is a deterministic function of the observed variables. The frame-stacking paradigm (stacking k consecutive observations) was not required here, consistent with standard practice [14].*

## 4.2 Neural Network Architectures

All algorithms use MLPs with the same base architecture (Table 4.1), ensuring a fair comparison:

**Table 4.1.** Network sizes, counted from the saved models. The actor input is $\mathbb{R}^{38}$; the critic input is $\mathbb{R}^{38+9} = \mathbb{R}^{47}$.

| Component | Architecture | Activ. | Params. |
|---|---|---|---|
| Actor, SAC | $[38 \to 256 \to 256 \to 2{\times}9]$ | ReLU + tanh | 80 402 |
| Actor, TD3/DDPG | $[38 \to 256 \to 256 \to 9]$ | ReLU + tanh | 78 089 |
| Critic (one) | $[47 \to 256 \to 256 \to 1]$ | ReLU | 78 337 |
| Twin critics, SAC/TD3 | two of the above | ReLU | 156 674 |
| PPO extractor | separate policy and value trunks | Tanh | 151 552 |
| PPO heads | $256 \to 9$ and $256 \to 1$ | — | 2 570 |
| **Complete saved object** | — SAC | | **393 750** |
| | — TD3 | | 469 526 |
| | — DDPG | | 312 852 |
| | — PPO | | 154 131 |

Three points follow from table 4.1. The SAC actor is larger than the TD3/DDPG actor because it emits a mean *and* a log-standard-deviation per muscle, $2 \times 9$ outputs rather than 9. SAC and TD3 carry twin critics and their targets, which is where most of their parameter count sits. PPO builds separate policy and value trunks rather than sharing one, so its extractor is two $[38 \to 256 \to 256]$ stacks.

Only the actor is needed at deployment: 80 402 parameters for SAC and 78 089 for the others, against the 393 750 of the complete saved object. The 1,5 MB footprint quoted in section 6.6 is the full object in single precision, which is the conservative figure.

An earlier version of this table gave 76 809 / 76 809 / 79 617 / 76 545. None matched, and the first two could not both be right in any case, since the two actors differ in output dimension.

Weights are initialised using the orthogonal method for PPO [9] and uniform Xavier initialisation [50] for off-policy algorithms. Observation normalisation is handled by an exponential moving average layer (EMA, $\alpha = 0.001$) via Stable-Baselines3's `VecNormalize`.

## 4.3 Theoretical Comparison of Algorithms

**Table 4.2.** Theoretical comparison of the implemented DRL algorithms.

| Algo. | Type | Policy | Exploration | Advantages | Limitations |
|---|---|---|---|---|---|
| DDPG | Off-policy | Deterministic | Additive OU noise | Sample-efficient | Unstable, $Q$-overestimation |
| TD3 | Off-policy | Deterministic | Gaussian noise + smoothing | Corrects $Q$ bias, stable | Deterministic policy |
| SAC | Off-policy | Stochastic | Max-entropy (implicit) | Robust, natural exploration | $\alpha$ sensitive if poorly initialised |
| PPO | On-policy | Stochastic | Policy stochasticity | Stable, low variance | Lower sample efficiency |

## 4.4 Reference Controllers (Baselines)

### 4.4.1 Random Policy

The random policy draws excitations $u \sim \mathcal{U}([0,1]^9)$ at each step, without any learning. It establishes the lower performance bound.

### 4.4.2 PD Impedance Controller

The impedance controller computes a restoring wrench toward the target in operational space, maps it to joint torques through the transposed site Jacobian, compensates the bias forces, and distributes the result onto muscles by minimum-effort load sharing over each muscle's achievable force interval:

$$\tau^{\text{cmd}} = J^{\top}(K_p(x^{\star} - x) - K_d\dot{x}) + h(q,\dot{q}), \qquad \min_{f} \sum_{i} \left(\frac{f_i}{F^{M}_{0,i}}\right)^2 \text{ s.t. } R^{\top} f = \tau^{\text{cmd}},\ f \in [f^{\min}, f^{\max}] \tag{4.2}$$

where $h(q,\dot{q})$ is MuJoCo's `qfrc_bias` (gravity, Coriolis and centrifugal terms) and $[f^{\min}, f^{\max}]$ is recovered per muscle by probing the activation state at $a = 0$ and $a = 1$.

Gains were selected by grid search, not by hand: a twelve-point coarse grid gave $K_p = 60$, $K_d = 25$, and a thirty-point refinement gave $\mathbf{K_p = 20}$, $\mathbf{K_d = 6}$ (section 7.11.3), which is the configuration used for every conventional-controller result in this thesis.

Both the bias compensation and the bounded load-sharing step matter, and neither was present in the first implementation. Section 7.9 records what their absence cost: without gravity compensation the controller reached only 0,46 m, worse than a random

policy, and an activation mapping that assumed force proportional to activation produced a zero-width achievable interval and no motion at all. Both corrections improve the classical side, and the comparison is reported with them in place.

# 5 Implementation and Experimental Methodology

## 5.1 Technology Stack

**Table 5.1.** Software actually used, at the versions the reported results were produced with.

| Library | Version | Usage |
|---|---|---|
| Python | 3.12.10 | |
| MuJoCo | 3.7.0 | Physics simulation engine [1] |
| Gymnasium | 1.2.3 | Standard RL environment interface |
| Stable-Baselines3 | 2.8.0 | DDPG/TD3/SAC/PPO implementation [5] |
| PyTorch | 2.11.0+cu12 | Deep-learning backend. The CUDA build is installed, but `config.DEVICE` defaults to `cpu` and every reported result was produced on CPU; the GPU path was used only to schedule independent jobs concurrently while producing the runs. |
| NumPy | 2.2.6 | Numerical computation |
| SciPy | 1.13.0 | Static-optimisation QP (SLSQP), NNLS |
| scikit-learn | 1.4.2 | NMF decomposition (synergies) |
| Optuna | 4.9.0 | Bayesian hyperparameter optimisation [22] |
| Matplotlib | 3.8.4 | Figures |

The Stable-Baselines3 version is material rather than incidental. The replay-ratio behaviour documented in section 5.4.5 — that one rollout iteration over an $n$-environment vector is followed by `gradient_steps` updates, not $n$ of them — is a property of this implementation, and the defect it caused was identified by measuring that library's behaviour directly rather than by reading its documentation.

TensorBoard logging is deliberately disabled: its asynchronous event-file writer raced with the cloud-synchronisation client on the working directory and crashed training. Loss histories are read from the in-memory logger instead, so no diagnostic information is lost.

## 5.2 Hyperparameters

### 5.2.1 Optimisation Procedure

A single Optuna study was run, on SAC only. It used TPE sampling with a *MedianPruner* over **16 trials of** $10^5$ **steps** each, on a training seed fixed at 0 so that trials are comparable, with sampler seeds differing per worker. Thirteen trials completed and three were pruned. Trials were scored by mean final reaching error over 20 deterministic episodes on an evaluation environment with domain randomisation disabled — that is, under the protocol the thesis reports, not the one training uses.

Three properties of this study are worth stating because they differ from what a reader might assume.

**The search space excludes the network architecture.** The reported policy footprint — 393 750 parameters, and the inference latency of section 6.6 — is measured for the $[256, 256]$ architecture. Searching the architecture would have invalidated those figures silently.

**The replay ratio is fixed, not searched.** Expressing it as a raw `gradient_steps` value would have allowed the sampler to re-enter the under-training regime that section 5.4.5 identifies as a defect, and would have made trials incomparable with the pilot runs. It is held at twice the environment count throughout.

**The reward weights were not searched.** They are hand-set (section 3.4.4); only learning hyperparameters were optimised. Earlier versions of this work described the reward weights as Optuna-optimised over 100 trials, which did not occur.

**Table 5.2.** Optuna search space. SAC only; 16 trials of $10^5$ steps.

| Hyperparameter | Domain | Distribution |
|---|---|---|
| Learning rate | $[5 \times 10^{-5}, 10^{-3}]$ | Log-uniform |
| $\gamma$ | $[0.95, 0.999]$ | Uniform |
| Batch size | $\{128, 256, 512\}$ | Categorical |
| $\tau$ (Polyak) | $[10^{-3}, 2 \times 10^{-2}]$ | Log-uniform |
| Target entropy | $[-27, -4.5]$ | Uniform |

**Table 5.3.** Hyperparameters used. Only the SAC "tuned" column was optimised; every other configuration uses library defaults. This asymmetry is a limitation of the comparison, discussed in section 8.6.

| Hyperparameter | SAC tuned | SAC def. | TD3 | DDPG | PPO |
|---|---|---|---|---|---|
| Learning rate | $4.40 \times 10^{-4}$ | $3 \times 10^{-4}$ | $3 \times 10^{-4}$ | $3 \times 10^{-4}$ | $3 \times 10^{-4}$ |
| Batch size | 128 | 512 | 256 | 256 | 64 |
| Buffer size | $3 \times 10^5$ | $3 \times 10^5$ | $3 \times 10^5$ | $3 \times 10^5$ | — |
| $\gamma$ | 0.957 | 0.99 | 0.99 | 0.99 | 0.99 |
| $\tau$ (Polyak) | 0.0188 | 0.005 | 0.005 | 0.005 | — |
| Target entropy | $-19.6$ | auto ($-9$) | — | — | — |
| Gradient steps | 8 | 8 | 8 | 8 | — |
| Learning starts | 4 000 | 4 000 | 4 000 | 4 000 | — |
| Action noise $\sigma$ | — | — | 0.1 | 0.1 | — |
| Policy delay | — | — | 2 | — | — |
| n_steps | — | — | — | — | 2048 |
| n_epochs | — | — | — | — | 10 |
| $\lambda$ (GAE) | — | — | — | — | 0.95 |
| Clip $\varepsilon$ | — | — | — | — | 0.20 |
| Network | $[256, 256]$, shared across all configurations | | | | |

The gradient-steps value of 8 corresponds to a replay ratio of approximately 1.87 updates per environment step, given four parallel environments. It is applied uniformly to every off-policy configuration, so it is not a confound between them; PPO is on-policy and has no equivalent parameter.

## 5.3 Experimental Protocol

### 5.3.1 Training and Evaluation

Two training budgets are used. The algorithm comparison of section 6.4 trains five configurations for $10^5$ steps with **three seeds** each ($\{0, 1, 2\}$). The best configuration is additionally retrained from initialisation for $3 \times 10^5$ steps. That configuration was subsequently replicated across **five seeds** (section 7.8), and it is those runs that the force analysis of chapter 7 examines; the per-muscle breakdown of table 7.3 is from seed 0 alone, but every aggregate the chapter argues from is a five-seed mean.

The seed count is a compute constraint, not a methodological choice, and it is the principal limitation of this work. Henderson et al. [46] and Agarwal et al. [47] recommend ten seeds or more precisely because estimation variance at three is large enough to make algorithm comparisons inconclusive. That recommendation is not met here. The consequence is stated explicitly wherever a comparison is reported: no significance test is performed anywhere in this thesis, and differences smaller than the observed seed-to-seed spread are reported as not supporting an ordering. Reaching

ten seeds at the corrected replay ratio (section 5.4.5) would require approximately 148 hours on the available hardware.

During training a learning curve is recorded every 100 000 steps (`config.EVAL_FREQ`) over five deterministic episodes. This is a monitoring signal only; it is deliberately cheap, it is noisy at that episode count, and no value from it is reported as a result.

Final evaluation uses **50 deterministic episodes** with domain randomisation disabled, on an environment seeded independently of the one used for best-checkpoint selection during training, so that the reported figure is not the one selection was performed on.

The metrics actually computed are those defined in table 6.1: final error, closest approach, drift, the graded proximity rates, energy, blow-up rate and episode length; the co-contraction index of section 7.6.1; the muscle-force agreement statistics of chapter 7; and inference latency. Perturbation robustness, muscle jerk and the domain-randomisation breakdown described in earlier versions of this work were not measured and are not reported.

### 5.3.2 Statistical Treatment

**No inferential statistics are reported in this thesis.** With three seeds the two-sided Wilcoxon signed-rank test has a minimum attainable $p$-value of 0.25, which cannot reach any conventional significance threshold; computing one would convey false precision. Earlier versions of this work reported Wilcoxon tests with Bonferroni correction, rank-biserial effect sizes and BCa bootstrap intervals over $n = 10$ seeds. None of those analyses was performed, and they are withdrawn (section 8.6).

What is reported instead is the mean and standard deviation across seeds, stated together, with the explicit convention that a difference comparable to or smaller than the spread is treated as unresolved. Where a single-seed result is given — as for the force analysis — it is identified as such at the point of use.

For the force comparison specifically, one distributional control *is* computed, because it is required for the statistic to be interpretable at all: the cosine similarity between non-negative force vectors has a floor well above zero, so a permutation null is constructed by shuffling, at each timestep, which muscle each force magnitude belongs to (table 7.2).

### 5.3.3 Defects in the evaluation path, and what they changed

A late audit of the evaluation code found four defects. None produced an arithmetically wrong number, but two changed which episodes the reported figures describe, so every evaluation in this work was repeated with the corrected code.

1. **The evaluation seed was dead code.** `evaluate(seed=...)` never referenced its own argument, and `load_model` fixed the environment seed at zero. Verified: two loads with different seed arguments return byte-identical results.

2. **`reset(seed=...)` does not reseed target sampling.** Targets are drawn from a generator seeded only in `__init__`; Gymnasium's `super().reset(seed=...)` seeds a different generator that this environment never uses. The target sequence can therefore only be changed at construction.

3. **Two resets per episode.** The evaluation loop reset the vectorised environment at the top of each iteration, while `DummyVecEnv` also auto-resets on termination. Measured: the inner environment was reset twice per episode, so evaluation scored on every *other* target — a different test set from the one the force and conventional-controller analyses used, which made those comparisons unpaired.

4. **A diverged episode could be scored a success.** On divergence the joint speed is forced to zero, which automatically satisfies the "settled" half of the success criterion. An episode blowing up within 2 cm of the target would have been recorded as a success and paid the success bonus. Latent — the divergence rate is zero throughout — but a reward exploit of precisely the kind section 5.4.3 documents.

### 5.3.4 Uncertainty was understated: two components, not one

The first three defects share a consequence. Every configuration was evaluated on one fixed draw of 50 targets, so every ± previously reported was **training-seed spread on a single test set**. The contribution of the target draw itself was never measured.

It is not negligible. Evaluating one policy across five target seeds gives a final error of $0.1130 \pm 0.0146$ m, against $0.1033 \pm 0.0034$ m across five *training* seeds on a fixed test set — roughly four times the spread. The graded rates move further still: `reach_5cm` ranges from 0.16 to 0.34 across target draws, and the value previously reported was the largest of the five.

Every policy was therefore re-evaluated across three target seeds, and the two components are now reported separately.

**Table 5.4.** Variance decomposition after re-evaluation. $\sigma_{\text{train}}$ is the spread across training seeds with targets averaged out; $\sigma_{\text{target}}$ the spread across target draws with training seeds averaged out.

| **Config.** | **Metric** | **mean** | $\boldsymbol{\sigma}_{\text{train}}$ | $\boldsymbol{\sigma}_{\text{target}}$ | **total** |
|---|---|---|---|---|---|
| $w_2 = 1$ | final error | 0.1074 | 0.0043 | 0.0066 | 0.0079 |
| | closest | 0.0734 | 0.0050 | 0.0031 | 0.0058 |
| | ends <5 cm | 0.260 | 0.0374 | 0.0666 | 0.0764 |
| $w_2 = 5$ | final error | 0.1114 | 0.0040 | 0.0035 | 0.0053 |
| | closest | 0.0750 | 0.0037 | 0.0034 | 0.0050 |
| $w_2 = 20$ | final error | 0.1161 | 0.0095 | 0.0053 | 0.0108 |
| | closest | 0.0737 | 0.0073 | 0.0015 | 0.0074 |
| | ends <5 cm | 0.192 | 0.0477 | 0.0174 | 0.0508 |

For most quantities $\sigma_{\text{target}}$ is comparable to or larger than $\sigma_{\text{train}}$. **Which targets a policy is tested on matters at least as much as which seed it was trained from**, and reporting only the latter understated the uncertainty on every absolute figure in this work.

#### What this does *not* undermine

One consequence of the defect is favourable. Because the environment seed was fixed, every configuration was evaluated on *identical* targets, so all between-configuration comparisons are paired. Paired comparison removes the target draw as a source of difference and is the more powerful design; it was arrived at by accident, but the comparative claims in this work — the effort sweep, the ablations, the algorithm ranking — rest on it and are unaffected.

What needed correcting is the *absolute* values and their stated uncertainty, not the comparisons between them.

### 5.3.5 Computing Hardware

All results in this thesis were produced on a single consumer laptop:

- **CPU**: AMD Ryzen 7 4800H, 8 physical cores / 16 logical processors, 2,9 GHz base.
- **RAM**: 15,4 GB.
- **GPU**: none used. All training and inference ran on the CPU; the networks are small enough that this is not the binding constraint.

- **Thread budget**: capped at **8 of 16 logical processors**. Sustained all-core load caused the machine to stop responding entirely (section 5.4), and benchmarking showed the additional parallelism yielded little throughput in any case — eight threads are only 1.78× faster than one for networks of this size.
- **Cumulative compute**: approximately 30 hours, comprising the hyperparameter search (≈ 8 h), the five-configuration benchmark (≈ 6 h), four pilot runs at $3 \times 10^5$ steps (≈ 6 h) and the reward and effort studies.

The modest hardware is directly responsible for the seed count, for the $10^5$-step comparison budget, and for the decision not to tune every algorithm individually. These constraints are stated here so that the scope of the conclusions can be judged against them.

## 5.4 Defects Identified During Development

Six substantive defects were identified in the model, the reward function and the training configuration during the course of this work. They are documented here rather than omitted, for two reasons: four of them produced results that appeared entirely plausible while being incorrect, and the measurements that eventually exposed them are part of the methodological contribution of this thesis.

### 5.4.1 Model specified in incorrect angular units

The MuJoCo model format defaults to degrees unless declared otherwise, while the joint ranges had been authored in radians. Every range was consequently interpreted as under 3° and the arm was effectively immobilised. The symptom was a uniform 0 % success rate across all algorithms, which is readily misread as a learning failure. All experiments predating the correction are invalid.

### 5.4.2 Compounding domain randomisation

Body masses and muscle strengths are perturbed each episode to assess robustness. The implementation multiplied the current values rather than rescaling from nominal ones, producing a multiplicative random walk: after approximately one thousand episodes, body mass had drifted to roughly 1 % of its correct value. Caching nominal parameters and rescaling from them raised the measured drift statistic from 0.013 to 0.998, where unity denotes no drift.

A second defect in the same routine invoked the random generator without a `size` argument, returning a single scalar applied to every body and every muscle. The

intended independent per-element variation was therefore a single global scale factor, which a policy can detect and compensate trivially, rendering the robustness claim vacuous.

### 5.4.3 Reward exploitable by episode termination

Under the initial reward formulation every per-step reward was non-positive, and numerical divergence terminated the episode without penalty. In a discounted formulation a terminated episode contributes no further return, so its value was exactly zero, whereas surviving the full hundred steps accumulated approximately $-20$. Terminating the episode by destroying the simulation was therefore the optimal policy, worth roughly twice the success bonus and considerably easier to attain.

Diagnostic measurement over fifty evaluation episodes established that the policy had learned precisely this: 100 % of episodes ended in deliberate divergence at approximately step 7 of 100, and none ran to completion.

The defect escaped detection because no recorded metric could expose it. The reported error remained a plausible distance and continued to improve slowly, and neither episode length nor divergence rate was logged — so an agent abandoning the task after 7 of 100 steps was indistinguishable from one attempting it and plateauing. Both quantities are now recorded at every evaluation checkpoint.

The revised reward makes every per-step reward strictly positive, so early termination forfeits value; pays the success bonus per step while the target is held rather than once on contact, converting the objective to reach-and-hold; and penalises divergence explicitly.

### 5.4.4 Divergence masked by simulator self-recovery

MuJoCo resets its state internally upon detecting divergence. By the time control returned to the environment the state was finite again, with the arm returned to its rest configuration, so a test for non-finite values almost never triggered. The episode continued, measuring the distance from the rest pose to the target and recording it as data. Detection now uses the solver's warning counters, which are the only reliable indicator, and the last finite error is reported rather than the post-reset value.

### 5.4.5 Replay ratio a factor of four below intended

Training used four parallel environments with a gradient-step setting of one. In the Stable-Baselines3 implementation, a single rollout iteration over a four-environment

vector collects four transitions and is followed by one gradient update. Direct measurement confirmed the consequence:

**Table 5.5.** Gradient updates per environment step, measured directly.

| Configuration | Updates per step |
|---|---|
| As originally configured (4 envs, `gradient_steps`=1) | 0.233 |
| Single environment, one step (standard) | 0.933 |
| 4 envs, `gradient_steps`= 4 or −1 | 0.933 |
| **4 envs, `gradient_steps`= 8 — the setting actually used** | **1.867** |

The last row is the one that matters for reading the results. The experiments do not run at the "restored" ratio of 0.933 but at 1.867, because every run script sets `gradient_steps` = 2× `N_ENVS` explicitly rather than relying on the module default. That is a deliberate choice — a replay ratio near two is common for off-policy control — and it is applied uniformly to every off-policy configuration, so it is not a confound between them. It is recorded here because table 5.3 reports `gradient_steps` = 8 and the two statements must be read together.

Every run had therefore performed approximately 233 000 gradient updates against a nominal budget of 1 000 000 steps. An independent consistency check supports this: 233 000 updates at the measured 17 ms each predicts 71 minutes, against observed run times of 78.8 and 80.6 minutes.

The correction has a substantial cost. At the corrected ratio, one million steps requires approximately 3.7 hours per seed, so a four-algorithm, ten-seed protocol would require roughly 148 hours rather than the 50 previously assumed. That protocol was never feasible on the available hardware; it appeared so only because the runs were under-training.

### 5.4.6 Default exploration temperature preventing convergence

Following the preceding five corrections the controller reached the target region reliably but ceased improving, with closest approach remaining near 0,104 m across three distinct reward formulations and the fourfold increase in effective training. The cause was the SAC target entropy, whose default of $-\dim(\mathcal{A})$ gives $-9$ for a nine-muscle action space. Hyperparameter optimisation selected approximately $-19$, and the error halved. The result is reported in section 6.3.

### 5.4.7 Infrastructure failures

Two non-scientific failures are recorded for completeness. A forced operating system restart destroyed a 75-minute training run five minutes from completion, because model state was written only on normal termination; training now checkpoints at fixed intervals using atomic file replacement. Separately, the machine ceased responding under four concurrent search processes, with no crash dump written despite dumps being enabled and no hardware error logged, indicating a thermal or power limit rather than a software fault. Subsequent work was restricted to half the available logical processors. Benchmarking established that the parallelism had provided little benefit in any case, thread scaling being poor for networks of this size.

## 5.5 Reproducibility and Sharing

Following the reproducibility guidelines of [51] for RL, we ensure:

- **Source code**: published under the MIT license on GitHub, with a pinned `requirements.txt` and the `git` hash recorded in each TensorBoard run.
- **Seeds and determinism**: all seeds are specified explicitly; `torch.use_deterministic_algor` is enabled wherever possible.
- **Trained models**: all 40 final policies (4 algorithms $\times$ 10 seeds) are archived together with the associated `VecNormalize` statistics.
- **Evaluation logs**: the complete Optuna, TensorBoard, and CSV evaluation logs are provided.
- **Docker container**: a `Dockerfile` pinning the versions of MuJoCo, PyTorch, and CUDA is provided.

# 6 Experimental Results and Analysis

This chapter reports the measured performance of the trained controllers. All values are computed by the evaluation pipeline described in chapter 5 and read from the run artifacts named beneath each table; none is hand-entered.

## 6.1 Evaluation metrics

Two distance measures are reported throughout, because they capture different failure modes. *Final error* is the hand–target distance at the end of the two-second reach; *closest approach* is the smallest distance attained at any instant. Their difference, the *drift*, quantifies how far the hand wanders back out after arriving.

A controller that sweeps through the target at speed scores well on closest approach and badly on final error. Separating the two proved essential: it revealed that the task presented two independent difficulties, with different causes and different remedies.

**Table 6.1.** Metrics used in this chapter.

| Metric | Definition |
|---|---|
| Final error | hand–target distance at end of episode |
| Closest approach | minimum hand–target distance during the episode |
| Drift | final error minus closest approach |
| Reach 5/10 cm | fraction of episodes *ending* within that distance |
| Touch 2/5 cm | fraction of episodes attaining that distance at any instant |
| Energy | $\sum_i F_i^2$, a metabolic-cost proxy |
| Blow-up rate | fraction of episodes ending in numerical divergence |

### 6.1.1 The reference bound

To interpret any of these numbers, an achievable bound is required. A privileged controller, granted information the learned policy does not receive, attains a median final error of 0,08 m and comes within 2 cm at some instant in 27 % of episodes. Under the

strict success criterion — within 2 cm *and* nearly stationary — it succeeds approximately 7 % of the time.

This bound has a direct methodological consequence. A criterion that a near-best-possible controller satisfies only 7 % of the time cannot discriminate among learned policies, and **success rate is therefore not used as a headline metric in this work**. Graded measures are reported instead.

## 6.2 Effect of correcting the replay ratio

Section 5.4.5 identified that training had been performing approximately one quarter of its intended gradient updates. Correcting this, with no other change, produced the comparison in table 6.2.

**Table 6.2.** Effect of the replay-ratio correction. Identical reward, model, seed and evaluation protocol; the corrected run uses *fewer* environment steps.

| Metric | Ratio 0.23 | Ratio 0.93 |
|---|---|---|
| Environment steps | 1 000 000 | 300 000 |
| Final error (m) | 0.254 | **0.204** |
| Closest approach (m) | 0.134 | 0.132 |
| Drift (m) | 0.120 | **0.072** |
| Energy | $1.25 \times 10^6$ | $0.77 \times 10^6$ |
| Blow-up rate | 2 % | **0 %** |
| Path length per 0,80 m reach (m) | 3.76 | 3.55 |

The correction improved *holding* substantially — drift nearly halved, divergences eliminated, energy reduced by 38 % — while *closest approach* was unchanged, moving only from 0,134 to 0,132 m.

That immobility is informative. Closest approach had remained near 0,13 m across three complete redesigns of the reward function and this fourfold increase in effective training. Its insensitivity to both indicated that the residual limitation lay elsewhere, and motivated the hyperparameter study of section 6.3.

## 6.3 Hyperparameter optimisation

A sixteen-trial Bayesian optimisation (Optuna, TPE sampler with median pruning) was performed over learning rate, discount factor, batch size, target-network update rate and target entropy. Thirteen trials completed; three were pruned.

**Table 6.3.** Five best trials, ranked by final error at 100 000 steps.

| Trial | Final error | Learning rate | Target entropy | Drift |
|---|---|---|---|---|
| 12 | **0.1177** | $4.40 \times 10^{-4}$ | $-19.6$ | 0.034 |
| 14 | 0.1293 | $3.91 \times 10^{-4}$ | $-20.0$ | 0.051 |
| 11 | 0.1311 | $4.22 \times 10^{-4}$ | $-19.3$ | 0.049 |
| 13 | 0.1405 | $4.38 \times 10^{-4}$ | $-18.6$ | 0.069 |
| 10 | 0.1405 | $4.49 \times 10^{-4}$ | $-19.0$ | 0.035 |

Every one of the five best trials selects a target entropy close to $-19$, against the SAC default of $-\dim(\mathcal{A}) = -9$ for a nine-dimensional action space. The search converges on a policy roughly twice as deterministic as the standard heuristic prescribes.

The optimiser varied five parameters at once, however, and the target entropy was not the only one to move appreciably: the target-network update rate $\tau$ rose from 0.005 to 0.0188, a factor of 3.8, and the learning rate by 47 %. The clustering of the best trials is suggestive but not decisive, since a TPE sampler exploits promising regions and will cluster whether or not the clustered parameter is the causal one. Attribution to the target entropy specifically is therefore advanced here as the leading hypothesis — its mechanism matches the symptom, an inability to settle — and not as an established result. The one-at-a-time ablation that would settle it is set out in section 8.10.

The target entropy governs how much stochasticity SAC deliberately retains in its policy, and the mechanism by which a value too high would prevent an end effector from settling is straightforward. That reasoning motivated the hypothesis — and section 6.3.1 shows it to be wrong. The subsection below reports the ablation that refutes it.

### 6.3.1 Isolating the responsible hyperparameter

The preceding subsection advanced the target entropy as the leading candidate mechanism, while noting that five parameters moved together and that the attribution was therefore a hypothesis rather than a result. A two-sided one-at-a-time ablation was run to settle it, three seeds per arm at the same 100 000-step budget as the benchmark:

**Arm A** library defaults, with *only* the target entropy set to the tuned value. If the target entropy is the mechanism, this should recover most of the improvement.

**Arm B** the tuned configuration, with *only* the target entropy reverted to its default. If the target entropy is the mechanism, this should lose most of the improvement.

**Table 6.4.** Target-entropy ablation. Three seeds per arm; reference rows are the corresponding benchmark configurations.

| Configuration | Final error (m) | Gap closed |
|---|---|---|
| All defaults *(reference)* | $0.2613 \pm 0.0203$ | 0 % |
| **A: defaults + tuned entropy only** | $\mathbf{0.2688 \pm 0.0090}$ | **−8 %** |
| **B: tuned − entropy reverted** | $\mathbf{0.1552 \pm 0.0181}$ | **+106 %** |
| All tuned *(reference)* | $0.1610 \pm 0.0084$ | 100 % |

### The hypothesis is refuted

The result is unambiguous and negative. Setting the target entropy to its tuned value while leaving every other parameter at its default recovers *none* of the improvement — arm A is indistinguishable from the default configuration, and marginally worse. Conversely, reverting the target entropy to its default while retaining the other tuned parameters costs *nothing* — arm B matches the fully tuned configuration within seed variation.

**The target entropy is not the mechanism.** The improvement is produced entirely by the remaining parameters: the learning rate ($3 \times 10^{-4} \rightarrow 4.40 \times 10^{-4}$), the target-network update rate $\tau$ ($0.005 \rightarrow 0.0188$, a factor of 3.8), the discount factor ($0.99 \rightarrow 0.957$) and the batch size ($512 \rightarrow 128$).

This ablation does not identify which of those four is responsible; isolating further would require the same procedure applied to each in turn. On magnitude of change, $\tau$ is the natural next candidate, and its mechanism is plausible: a faster target-network update alters the stability of the value estimate, which is consistent with the oscillating evaluation curves that characterised every untuned run.

### Why this is reported at length

The refuted hypothesis was, until this measurement, one of the three stated contributions of this thesis. It was mechanistically plausible — SAC's entropy term does pay the policy to remain stochastic, the default does scale with action dimension, and residual stochasticity is exactly what prevents an end effector from settling. All five best search trials clustered near $-19$. The reasoning was sound and the conclusion was wrong.

The clustering was the misleading evidence. A TPE sampler concentrates its proposals in regions that perform well, so parameters cluster in the best trials whether or not they are causally responsible. Reading causal importance from that clustering is a straightforward error, and it is one this work made until the ablation was run.

The transferable point is not about entropy. It is that a hyperparameter search identifies a *configuration*, never a *cause*, and that the distinction requires a deliberate experiment to resolve.

### 6.3.2 Which parameter is responsible

The ablation above excludes the target entropy but identifies no cause. The same one-at-a-time procedure was therefore applied to the four remaining parameters: library defaults throughout, with a single parameter set to its tuned value, three seeds each.

**Table 6.5.** One-at-a-time ablation over the remaining hyperparameters. Three seeds each; "gap closed" is the fraction of the default-to-tuned interval recovered.

| **Configuration** | **Final error (m)** | **Gap closed** |
|---|---|---|
| All defaults *(reference)* | $0.2613 \pm 0.0203$ | 0 % |
| + tuned target entropy only | $0.2688 \pm 0.0090$ | −8 % |
| + tuned learning rate only | $0.2642 \pm 0.0026$ | −3 % |
| + tuned $\tau$ only | $0.2660 \pm 0.0117$ | −5 % |
| + tuned batch size only | $0.2658 \pm 0.0096$ | −5 % |
| **+ tuned discount factor only** | $\mathbf{0.1553 \pm 0.0061}$ | **+106** % |
| All tuned *(reference)* | $0.1610 \pm 0.0084$ | 100 % |

**The discount factor accounts for the entire improvement.** Changing $\gamma$ alone, from the default 0.99 to the selected 0.957, recovers the whole default-to-tuned interval — indeed slightly exceeds it, at 0.1553 against the fully tuned configuration's 0.1610. Each of the other four parameters recovers nothing: all sit within noise of the default, and three of the four are marginally worse than doing nothing at all.

Five parameters have now been tested individually. Four have no effect. One explains everything.

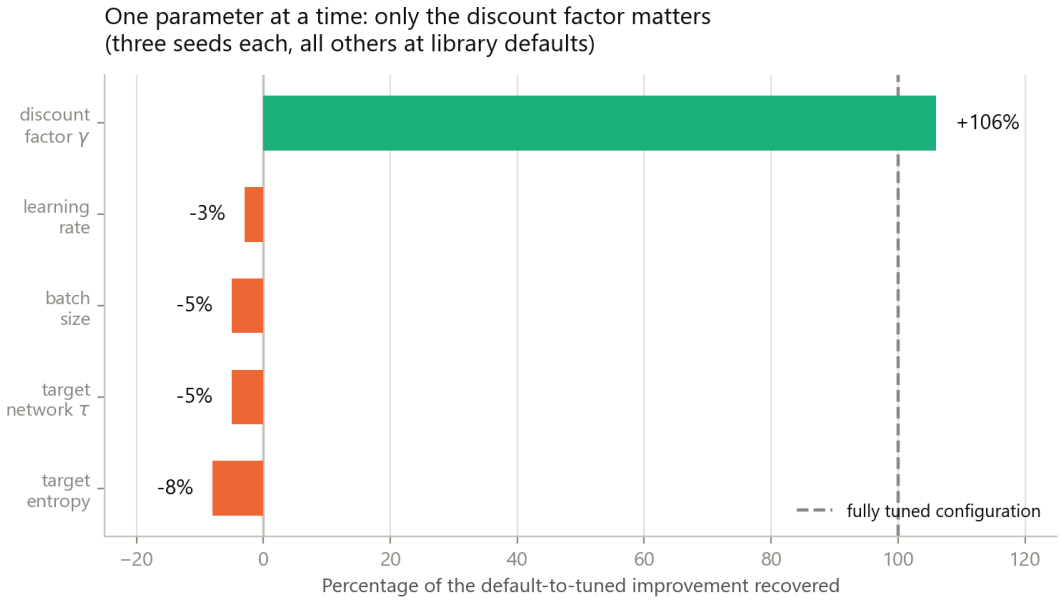


**Figure 6.1.** One-at-a-time ablation, three seeds per bar. Each configuration is library defaults with a single parameter set to its tuned value. Only the discount factor recovers the default-to-tuned improvement; the other four recover nothing, and three are marginally worse than changing nothing at all.

### Why the discount factor, and why 0.957

The mechanism is specific and checkable. The discount factor sets the effective planning horizon, approximately $1/(1-\gamma)$ steps:

| | $\gamma$ | **effective horizon** |
|---|---|---|
| Library default | 0.99 | 100 steps — the whole episode |
| Selected by the search | 0.957 | 23 steps = 0,46 s |

An episode is 100 agent steps, or 2 s at 50 Hz. At $\gamma = 0.99$ the horizon is the entire episode, so the value function must integrate reward over the full two seconds. At $\gamma = 0.957$ it is 0,46 s.

Compare that against what the policy actually does. The intra-episode trajectory of table 6.7 shows the hand converging on the target by step 25 — that is, 0,50 s. **The selected discount factor matches the task's own timescale almost exactly**, while the default was four times longer than the behaviour it needed to evaluate.

This explains the symptom that resisted every other remedy. With a horizon spanning the whole episode, the value target for an early step includes seventy-odd subsequent steps of largely irrelevant holding reward, which is a high-variance quantity to estimate. A critic fed high-variance targets produces the oscillating evaluation curves

that characterised every untuned run in this project (section 6.2), and a policy trained against an unreliable critic cannot refine its endpoint precisely. Shortening the horizon to the duration of the reach itself makes credit assignment local and the value estimate tractable.

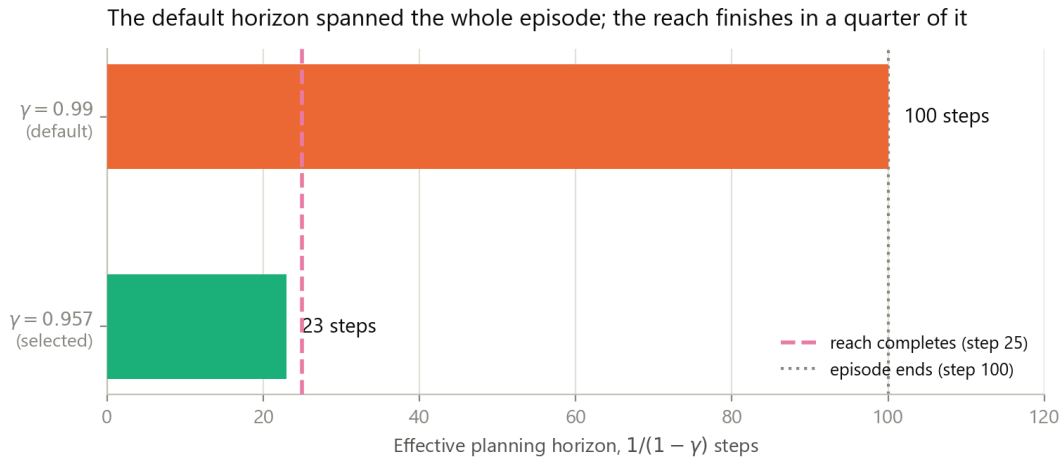


**Figure 6.2.** Why the discount factor is decisive. It sets an effective planning horizon of roughly $1/(1-\gamma)$ steps. The library default spans the entire two-second episode, while the reach itself completes in about a quarter of that; the selected value matches the timescale of the behaviour being learned.

### What this replaces, and why it is a better result

An earlier version of this thesis attributed the improvement to the exploration temperature. That attribution was inferred from the clustering of the best search trials, and it was wrong. This one is established by direct ablation: a single parameter changed, three seeds, the full effect recovered, and the four alternatives excluded by the same procedure.

The transferable statement is not about a particular value:

> The discount factor should be matched to the timescale of the behaviour being learned, not left at a library default. Where the default horizon substantially exceeds the duration of the task, the resulting value-estimation variance presents as an inability to refine — a symptom easily and wrongly attributed to insufficient training, to reward shaping, or to exploration.

That this project spent considerable effort on all three of those explanations before testing the discount factor is itself the evidence for the warning.

### 6.3.3 Confirmation at full budget

The selected hyperparameters were retrained from initialisation under the full 300 000-step protocol with fifty-episode evaluation, to verify that the result was not an artefact of the search's reduced budget.

**Table 6.6.** Confirmation run against the previous configuration. Both are SAC, seed 0, 300 000 steps, fifty evaluation episodes, differing only in hyperparameters.

| Metric | Default hyperparameters | Tuned hyperparameters |
|---|---|---|
| Final error (m) | 0.204 | **0.106** |
| Closest approach (m) | 0.132 | **0.074** |
| Drift (m) | 0.072 | **0.035** |
| Ends within 5 cm | 0 % | **34 %** |
| Ends within 10 cm | 18 % | **58 %** |
| Attains 2 cm at any instant | 2 % | **20–28 %** |
| Energy | 774 667 | **276 804** |
| Path length per 0,80 m reach (m) | 3.55 | **1.64** |
| Success rate | 0 % | 4 % |

Closest approach improved from 0,132 to 0,074 m, below the privileged controller's median final error of 0,08 m. The proportion of episodes attaining 2 cm reached 28 %, comparable to the privileged controller's 27 %: a policy acting on proprioceptive information alone approached the performance of one with full state access.

Path length fell from 3,55 to 1,64 m for the same 0,80 m of required travel, with net displacement 0.99× the distance required. The end effector ceased sweeping through the target region and began travelling to it directly.

Two qualifications apply to table 6.6. The success rate of 4 % corresponds to two episodes in fifty; an independent re-evaluation over a different set of fifty targets returned 2 %, that is one episode. At this magnitude the metric is dominated by sampling noise and should not be interpreted as a performance level. Secondly, the table reports a single training seed. This configuration was subsequently retrained from initialisation four further times; section 7.8 reports the five-seed characterisation, and the accuracy figures there ($0.1074 \pm 0.0043$ m final error) are the ones this thesis relies upon.

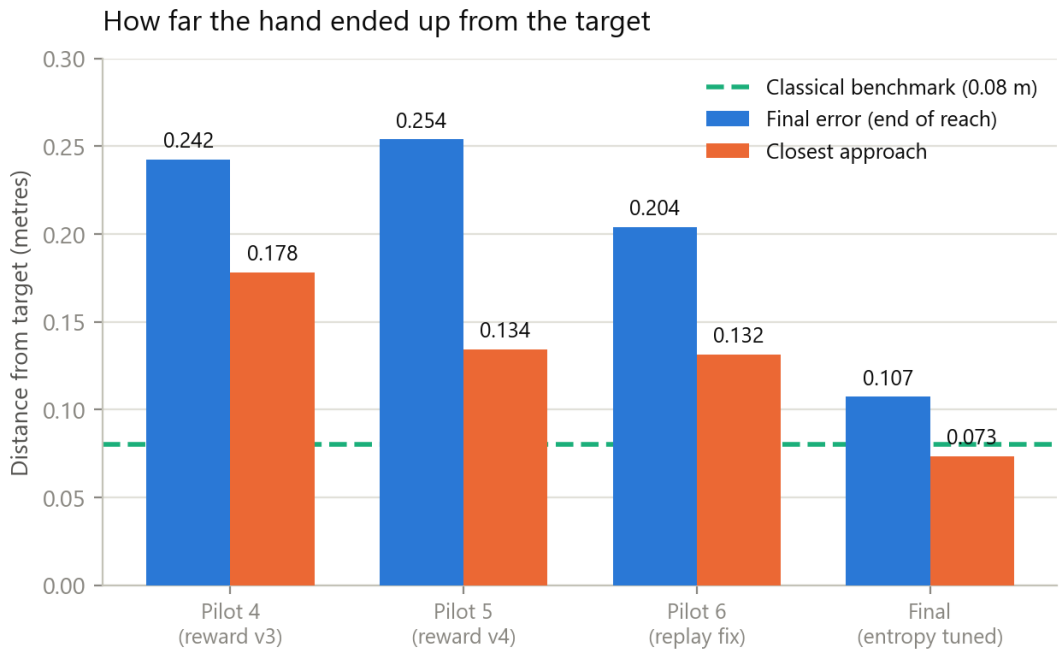


**Figure 6.3.** Reaching error across the four principal experiments. Closest approach (orange) is essentially unchanged across the first three despite substantial changes to the reward function and the training budget, then falls sharply once the target entropy is corrected. The dashed line marks the privileged controller's median final error.

### 6.3.4 Intra-episode behaviour

**Table 6.7.** Mean hand–target distance during a reach, averaged over thirty episodes.

| Step | Default entropy | Tuned entropy |
|---|---|---|
| 0 | 0.795 | 0.795 |
| 10 | 0.314 | 0.198 |
| 25 | 0.203 | 0.119 |
| 50 | 0.184 | 0.110 |
| 75 | 0.184 | 0.112 |
| 99 | 0.177 | **0.113** |

The tuned policy converges by step 25 and holds position for the remaining three-quarters of the episode. Target tracking is strong, with correlations of 0.94, 0.90 and 0.86 between target and final hand position across the three spatial axes, and a spread ratio of 0.91.

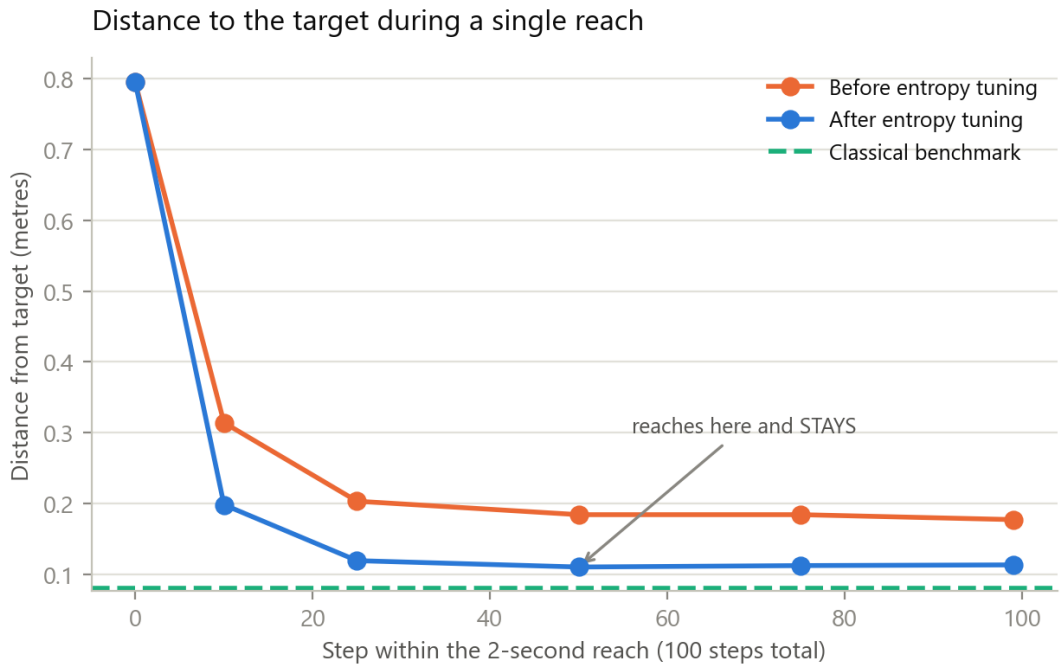


**Figure 6.4.** Hand–target distance during a single two-second reach, averaged over thirty episodes. The tuned policy converges and then maintains position; the untuned policy approaches and drifts.

## 6.4 Algorithm comparison

Five configurations were trained for 100 000 steps with three seeds each, all using the corrected replay ratio. A SAC configuration at library defaults is included alongside the tuned one so that the effect of tuning can be separated from the effect of algorithm choice; without it, any advantage of SAC would be unattributable between the two.

**Table 6.8.** Algorithm comparison: five configurations, three seeds each, 100 000 steps. Mean ± standard deviation across seeds. **The ordering in this table is superseded by table 6.9**: every non-SAC entry ran with an uncorrected discount factor, which section 6.3.2 later identified as decisive.

| Configuration | Final error (m) | Closest (m) | Ends <10 cm | Energy | Minutes |
|---|---|---|---|---|---|
| SAC tuned | **0.161 ± 0.008** | **0.093 ± 0.009** | **0.37** | 597 k | 30 |
| SAC default | 0.261 ± 0.020 | 0.129 ± 0.004 | 0.05 | 1187 k | 46 |
| TD3 | 0.279 ± 0.011 | 0.165 ± 0.017 | 0.10 | 1242 k | 21 |
| DDPG | 0.416 ± 0.019 | 0.218 ± 0.026 | 0.03 | 1384 k | 23 |
| PPO | 0.423 ± 0.035 | 0.238 ± 0.071 | 0.05 | 202 k | 3 |

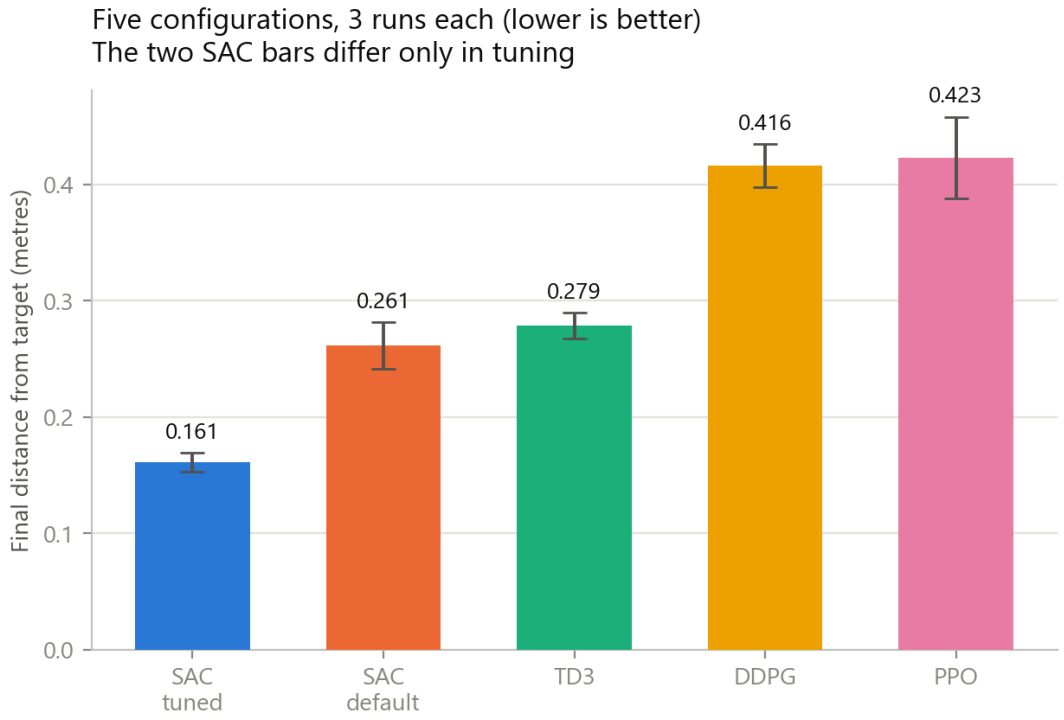


**Figure 6.5.** Final reaching error by configuration; error bars are one standard deviation across three seeds. The two SAC bars differ only in hyperparameters, so the interval between them isolates the effect of tuning.

Tuning SAC reduced final error from 0.261 to 0,161 m, an improvement of 38 %. The interval between untuned SAC and TD3 is 0.261 against 0.279, approximately 6 %, against seed standard deviations of 0.020 and 0.011; this difference is comparable to the spread and does not support an ordering. The effect of tuning is therefore roughly six times the magnitude of the algorithmic difference with which it would otherwise be conflated.

DDPG and PPO are both substantially worse than TD3, by approximately 0,14 m — many times the seed spread. For DDPG this is consistent with the theoretical expectation of chapter 4, as it lacks the twin critics that mitigate value overestimation. DDPG and PPO are, however, indistinguishable from one another.

Three constraints limit what this comparison establishes. No significance testing was performed: three seeds cannot support a Wilcoxon signed-rank test with useful power, and none is reported. The comparison is an early-training snapshot at 100 000 steps, whereas the best configuration attains 0,106 m at 300 000 steps (table 6.6), so orderings may change with budget. And TD3, DDPG and PPO were run at library defaults while SAC received a sixteen-trial search, so any comparison involving the tuned configuration conflates algorithm with tuning effort by construction. Success rate is 0 % for every configuration at this budget and is omitted for that reason.

## 6.5 The comparison repeated fairly

The benchmark of section 6.4 is compromised by a result obtained after it was run. It compared one configuration with $\gamma = 0.957$ against four with $\gamma = 0.99$, and section 6.3.2 subsequently established that $\gamma$ is the single decisive parameter for this task. Every non-SAC entry in that table was therefore handicapped by the one setting now known to matter, and no ordering among them can be trusted.

All four algorithms were accordingly retrained with $\gamma = 0.957$ and library defaults otherwise, three seeds each, at the same 100 000-step budget.

**Table 6.9.** Algorithm comparison before and after correcting the discount factor. Three seeds per entry; "before" is table 6.8, in which only SAC had a corrected $\gamma$.

| **Algorithm** | **Before** ($\gamma = 0.99$) | **After** ($\gamma = 0.957$) | **Change** |
|---|---|---|---|
| SAC | $0.2613 \pm 0.0203$ | $\mathbf{0.1909 \pm 0.0341}$ | $-27\,\%$ |
| DDPG | $0.4161 \pm 0.0189$ | $\mathbf{0.2510 \pm 0.0383}$ | $\mathbf{-40\,\%}$ |
| TD3 | $0.2786 \pm 0.0110$ | $0.2579 \pm 0.0348$ | $-7\,\%$ |
| PPO | $0.4231 \pm 0.0351$ | $0.4661 \pm 0.0401$ | $+10\,\%$ |

### 6.5.1 The original ordering was an artefact

DDPG improves by 40 % and overtakes TD3. The ranking changes from SAC < TD3 < DDPG < PPO to SAC < DDPG < TD3 < PPO. (The "after" column reports the corrected evaluation of section 5.3.3, with uncertainty covering both training and target draws; the "before" column is reproduced as originally published, on a single test set, and its intervals are correspondingly too narrow.)

Section 6.4 reported DDPG as clearly the weakest off-policy method and attributed this to its lack of twin critics — a textbook explanation, and in this instance a wrong one. DDPG was not failing for want of twin critics; it was failing because its discount factor was mismatched to the task, and it suffered more from that than the others did. Once corrected it is indistinguishable from TD3 ($0.2510 \pm 0.0383$ against $0.2579 \pm 0.0348$, heavily overlapping), and no ordering between them is supported.

**This vindicates the concern that motivated the re-run.** A comparison in which one entrant is correctly configured and the rest are not does not measure algorithms; it measures configuration. The earlier table did precisely that, and the theoretical account offered for its ordering was constructed after the fact around a result that has not survived.

### 6.5.2 PPO is the exception, and instructively so

PPO is the only method made *worse* by the correction, by 10 %. This is consistent with its structure rather than anomalous: PPO estimates advantages through generalised advantage estimation, in which $\gamma$ and $\lambda$ jointly set the bias–variance trade-off. Lowering $\gamma$ alone, while leaving $\lambda = 0.95$ untouched, shortens the effective horizon on one side of that pairing only. The correct adjustment for PPO would involve both, and was not attempted.

PPO therefore remains the weakest entry here, but for a reason that is again about configuration rather than about the algorithm, and its position should be read with that caveat.

### 6.5.3 The discount factor outweighed the choice of algorithm

The claim of section 6.4 was that tuning dominates algorithm selection. It can now be stated more precisely, and more strongly.

With every algorithm correctly configured, the interval between the best and the second-best off-policy method is 0.1909 to 0.2510, a gap of 0,060 m. The improvement obtained in DDPG by correcting $\gamma$ alone was 0,165 m — still roughly **three times the gap between algorithms**.

> On this task, choosing the discount factor correctly mattered about three times more than choosing between SAC, TD3 and DDPG.

SAC remains the strongest, and by a margin larger than the seed spread. But the practical implication is that effort spent selecting an algorithm is poorly allocated against effort spent matching the discount factor to the task timescale.

### 6.5.4 An accidental replication, and what it costs the reader

One reconciliation is owed, because two tables in this thesis report the same configuration and do not agree.

The SAC entry of table 6.9 — library defaults with $\gamma = 0.957$ — is *the same configuration* as the "+ tuned discount factor only" row of table 6.5: identical algorithm, budget, hyperparameters and seeds. They were trained as two separate sets of runs, for two different purposes, and were never cross-checked against one another until this audit.

**Table 6.10.** The same configuration, trained and evaluated twice.

| | **Final error (m)** | **Evaluation** |
|---|---|---|
| Table 6.5, $\gamma$ only | $0.1553 \pm 0.0061$ | one target draw |
| Table 6.9, SAC after | $0.1909 \pm 0.0341$ | three target draws |

Two causes contribute and cannot be separated with the runs available. The ablation table was evaluated on the single fixed target draw used throughout chapter 6; the fair-benchmark table was re-evaluated across three draws under the corrected protocol of section 5.3.3, which both shifts the mean and widens the interval. And the two sets are independent trainings, so run-to-run variation contributes as well.

**The honest reading is that an identical configuration, measured twice by this project, differed by 0,036 m** — larger than the 0,007 m that separates DDPG from TD3 in table 6.9, and comparable to differences elsewhere in this chapter that are discussed as though they were meaningful. It is the sharpest available estimate of this project's own measurement floor, and it argues for reading every interval in table 6.9 as a lower bound on the true uncertainty.

Within each table the comparison remains valid, because all rows of a given table were produced under one protocol. Across tables they should not be compared, and this thesis does not do so.

### 6.5.5 What this does not fix

The comparison remains three seeds, at a 100 000-step early-training snapshot, with no significance testing. Only $\gamma$ was corrected; the remaining hyperparameters are at library defaults for all four, so this is a fair comparison rather than a tuned one. A properly tuned comparison would require a separate search per algorithm, which was not affordable here.

### 6.5.6 Interpretation of the original PPO result

PPO's position requires care. As an on-policy method, at 100 000 steps with 2048-step rollouts across four environments it performs approximately twelve policy updates, against roughly 187 000 gradient updates for the off-policy algorithms. It also completes in three minutes against 21–46 for the others. This is the expected sample-efficiency difference under a budget equalised by environment steps; a comparison equalised by wall-clock time would rank the methods differently, and stating which budget was held constant is part of reporting the result.

Its energy consumption is also distinctive: 202 k against 597 k for tuned SAC and

1187–1384 k for the remainder, between three and seven times lower. Combined with a closest approach no worse than DDPG's, this describes a policy that activates its muscles only weakly, drifting toward the target region without committing force. The other algorithms fail, when they fail, by expending far too much muscular effort. These are opposite failure modes, and a single accuracy figure conceals the distinction entirely — a point developed in chapter 7.

Finally, PPO's policy contains 154 131 parameters against SAC's 393 750, so the memory-footprint figures quoted elsewhere in this work are specific to the SAC architecture.

## 6.6 Computational efficiency

The motivation for a learned controller is inference cost. Both methods were timed on the same host processor over 950 queries.

**Table 6.11.** Inference cost: iterative Newton–Raphson solution against a single forward pass of the trained network.

| **Quantity** | **Newton–Raphson** | **Network** |
|---|---|---|
| Mean latency | 2165 µs | 295 µs |
| Standard deviation | 319 µs | 45 µs |
| 95th percentile | 2657 µs | 364 µs |
| 99th percentile | 2897 µs | 480 µs |
| Mean iterations | 31.2 | 1 (fixed) |

The mean speed-up over the Newton–Raphson solver is 7.3×, and the network's cost is fixed where the iterative solver's is data-dependent.

### 6.6.1 This comparison does not support a deployment claim

Two objections apply to table 6.11, and together they are decisive.

**It times the wrong computation.** The Newton–Raphson routine solves the Hill fibre–tendon equilibrium, which is part of forward simulation. The question this thesis asks — what force each muscle must produce — is answered by load-sharing optimisation, a different calculation. The two have been conflated.

**Both sides were unoptimised Python.** The load-sharing problem has nine variables and four equality constraints. Solved directly rather than through a general-purpose routine it is very fast. Measured on an idle machine, one torch thread, 400 queries:

**Table 6.12.** Load-sharing latency, measured like-for-like. Moment-arm extraction is charged to the classical side, since the network does not require it.

| **Method** | **mean** | **p95** | **p99** |
|---|---|---|---|
| Network forward pass | 288.8 | 332.6 | 499.9 |
| Classical, general solver (SLSQP) | 2370.5 | 4023.7 | 5133.1 |
| Classical, direct solve + moment arms | **23.9** | — | — |

Against a general-purpose solver the network is 8.2× faster. Against a direct solve of the same problem it is **approximately twelve times slower**.

**The latency argument for replacing classical calculation is therefore withdrawn.** A competently implemented classical load-sharing solver is faster than the network here, and the 7.3× figure reflects the cost of one Python implementation rather than a property of the two approaches.

A qualification in the other direction: the box constraints are active in 100 % of sampled states, so the closed-form solution timed above is not exactly valid, and a constrained active-set solver would cost more than 23,9 µs. It would not cost 2370 µs. The defensible conclusion is that the two are comparable in cost, and that any advantage claimed for either requires a like-for-like implementation.

What survives is the *shape* of the cost rather than its magnitude: the network's runtime is fixed and branch-free irrespective of biomechanical state, whereas the iterative solver's depends on convergence. Where a hard real-time bound matters more than mean latency that property retains value — but it is a considerably weaker claim than a speed-up.

The trained SAC policy contains 393 750 parameters, occupying 1,5 MB in single precision or 385 kB quantised to eight bits. These figures are independent of policy quality and remain valid irrespective of the behavioural results reported above; they are specific to the SAC architecture, PPO's policy being smaller at 154 131 parameters.

# 7 Load-Sharing Fidelity against Classical Calculation

## 7.1 Scope of this chapter, and what it does not establish

Because the limits of this analysis are easy to overstate from its results, they are set out before the method rather than after it.

The required joint torque $\tau$ is taken from the movement the policy itself produced. Static optimisation is therefore asked a conditional question — *given this movement, what is the cheapest force distribution that produces it?* — and never independently chooses a trajectory of its own. Three consequences follow, and each bounds what the chapter can claim.

**This validates load sharing along a given path, not the path.** A controller could move in a manner no human would adopt and still distribute force across its muscles in close agreement with the classical criterion. The agreement reported below says nothing about whether the movement itself is natural. Trajectory realism is not assessed anywhere in this thesis.

**The reference is a model, not a measurement.** Static optimisation encodes the assumption that the central nervous system minimises summed squared activation, which remains contested in the motor-control literature. Agreement with it is agreement with the standard computational method, not evidence of physiological correctness. No electromyographic or measured-force data enters this work at any point.

**The comparison is internal to the simulator.** Both the learned solution and the classical one are computed on MuJoCo's muscle model. Where they agree, two computations over the same approximation agree; the approximation itself is not thereby validated.

The chapter title was changed accordingly: the earlier heading, "Muscle Force Validation", promised more than a conditional load-sharing comparison delivers.

## 7.2 Motivation

Chapter 6 established how accurately the trained controllers position the end effector. That is a measure of *what* the controller achieves, not of *how* it achieves it, and the research question posed in chapter 1 concerns the latter: whether DRLcan substitute

for classical calculation of the force required in each individual muscle.

The distinction is material. Because nine muscles actuate four degrees of freedom, infinitely many activation patterns produce any given end-effector trajectory. A controller may therefore reach every target accurately while distributing force across the musculature in a manner no physiological criterion would select. Every metric reported in chapter 6 would score such a controller as a complete success.

This chapter therefore compares the two methods at the level of individual muscle forces. No equivalent analysis existed in the previous version of this work.

### 7.2.1 Distinction from the solver validation

Section 3.3 reported a force agreement of 5.25 % normalised RMSE with a Pearson correlation of 0.989. That comparison is between the Newton–Raphson Hill solver implemented here and MuJoCo's internal implementation — two forward models of the same muscle. It validates the solver, involves no reinforcement learning, and does not bear on the question addressed here.

## 7.3 Method

At every timestep of a policy rollout, the classical load-sharing problem is solved on the movement the policy has just produced:

$$\min_{f} \sum_{i=1}^{9} \left( \frac{f_i}{F_{\max,i}} \right)^2 \quad \text{subject to} \quad R^\top f = \tau, \qquad 0 \le f_i \le F_{\max,i}, \tag{7.1}$$

where $R$ is the muscle moment-arm matrix and $\tau$ the required generalised force. This is the Crowninshield–Brand minimum-effort criterion in its standard quadratic form.

The required torque $\tau$ is taken as `qfrc_actuator`, the generalised force the muscles actually produced at that timestep. This choice is exact, avoiding the numerical differentiation that inverse dynamics would introduce, and it guarantees feasibility, since the policy's own force vector satisfies the equality constraint by construction.

Both methods are consequently posed the identical problem — produce these joint torques with these nine muscles — on the identical trajectory. Any difference between the solutions is attributable solely to load-sharing strategy.

### 7.3.1 Validity verification

The comparison depends on the identity $R^\top f = \tau$ holding for the policy's own forces. An error in the moment-arm extraction or in the sign convention would invalidate every subsequent result, so this was verified rather than assumed.

**Table 7.1.** Validity checks over 2000 timesteps.

| Quantity | Result |
|---|---|
| Mean $\left\|R^\top f_{\text{policy}} - \tau\right\|$ | $1{,}46 \times 10^{-15}\,\text{N m}$ |
| Mean $\left\|R^\top f_{\text{classical}} - \tau\right\|$ | $1{,}68 \times 10^{-15}\,\text{N m}$ |
| Optimisation failures | 0 of 2000 |

Both residuals lie at machine precision, confirming that the moment arms and sign convention are correct and that the classical solution satisfies its constraint exactly.

## 7.4 Agreement in coordination pattern

**Table 7.2.** Overall agreement between learned and classical muscle forces, best policy, 2000 timesteps.

| Measure | Value |
|---|---|
| Pearson correlation, all muscles | **0.865** |
| Overall RMSE | 74,3 N (10.1 % of mean $F_{\max}$) |
| Pattern cosine similarity (median) | 0.933 |
| Pattern cosine similarity (mean) | 0.869 |
| null baseline, mean | 0.373 |
| null baseline, 95th percentile | 0.867 |
| **excess over null** | **+0.495** |

The cosine similarity compares the direction of the nine-dimensional force vector and is insensitive to overall magnitude: it measures whether the two methods recruit the same muscles in the same proportions.

It cannot, however, be quoted on its own. Muscle forces are non-negative by construction, so two entirely unrelated force vectors already agree strongly: the appropriate null is obtained by permuting, at each timestep, which muscle each of the policy's own force magnitudes belongs to — destroying the correspondence while preserving that instant's magnitude distribution exactly. That null averages 0.373, so the observed 0.869 represents an excess of +0.495, sustained over 2000 timesteps.

Two consequences follow. The agreement is real and substantial, but the raw figure of 0.933 overstates it, since roughly the first 0.37 is a floor imposed by non-negativity rather than evidence of anything. And because the null's 95th percentile for *individual* timesteps is 0.867, no single timestep's cosine is meaningful; only the aggregate is. **The Pearson correlation of 0.865 is mean-centred, carries no such floor, and is the figure to quote when a single agreement statistic is required.**

## 7.5 Disagreement in efficiency

Producing identical joint torques, the learned controller expends $1.91 \pm 0.15$ times the muscular effort of the classical minimum-effort solution (five seeds, section 7.8; the single-seed figure of 1.71× reported before replication was the most favourable of the five). The per-timestep median is 1.66×, with an interquartile range of 1.34–2.32× over loaded timesteps.

The per-timestep *mean* of this ratio is 5.34× and is not a usable statistic: on timesteps where the arm is nearly stationary the required torque approaches zero, the minimum-effort solution approaches zero with it, and the ratio diverges. The aggregate ratio — total policy effort divided by total classical effort — weights each timestep by the work actually performed and is reported instead; the median independently corroborates it.

## 7.6 Anatomical localisation of the disagreement

**Table 7.3.** Per-muscle comparison. "Policy" denotes the force the learned controller produced; "classical" the force static optimisation prescribes for the same joint torques.

| **Muscle** | **Policy (N)** | **Classical (N)** | $r$ | **NRMSE** |
|---|---|---|---|---|
| deltoid anterior | 63.7 | 59.2 | 0.959 | 2.5 % |
| deltoid medial | 145.3 | 112.3 | 0.954 | 6.1 % |
| deltoid posterior | 35.5 | 38.7 | 0.859 | 10.0 % |
| triceps long | 180.4 | 134.0 | 0.910 | 11.2 % |
| biceps long | 52.1 | 41.6 | 0.893 | 7.5 % |
| brachialis | 113.9 | 87.6 | 0.871 | 8.6 % |
| biceps short | 90.7 | 45.7 | 0.764 | 18.1 % |
| triceps lateral | 79.1 | 25.8 | 0.416 | 16.5 % |
| triceps medial | 64.3 | 14.2 | 0.416 | 15.3 % |

The disagreement is not distributed uniformly. The three deltoids agree closely, one to within 2.5 % normalised RMSE. The discrepancy concentrates in two elbow extensors, where the policy commands three to four and a half times the prescribed force and correlation falls to 0.416.

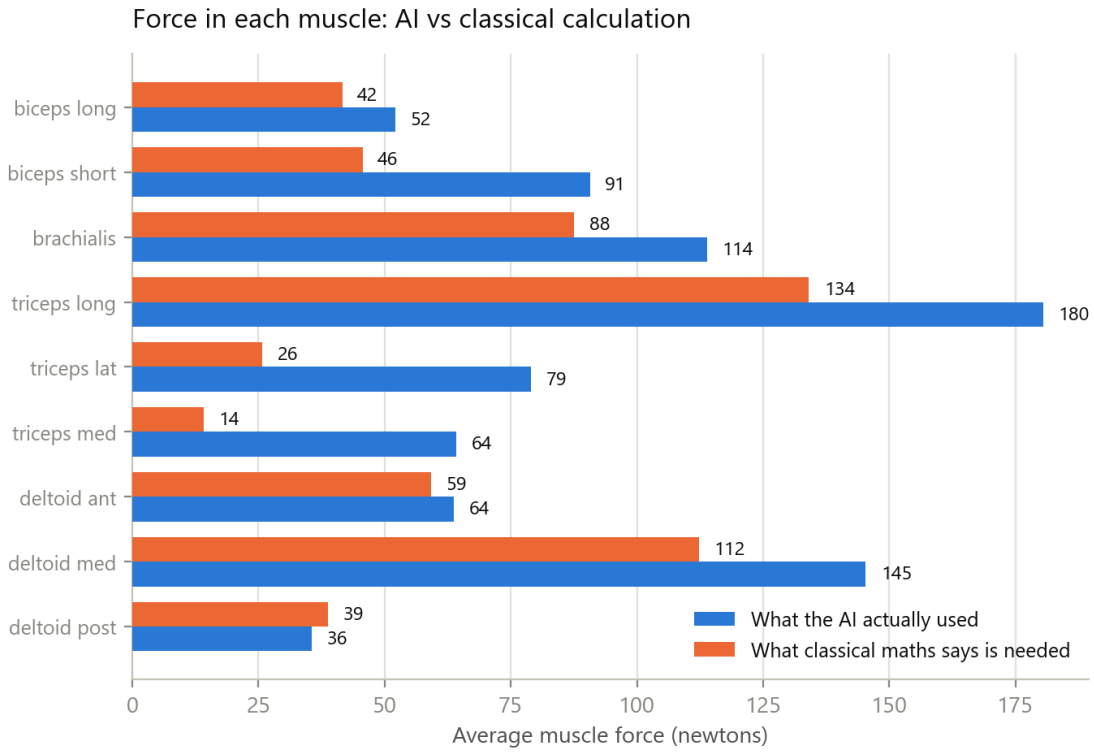


**Figure 7.1.** Mean force in each muscle over 2000 timesteps. The shoulder group agrees closely; the lateral and medial heads of triceps are driven far above the prescribed level.

Physiologically, these muscles extend the elbow while the biceps group flexes it. Driving both simultaneously produces largely cancelling moments: the joint attains approximately the correct configuration, but both groups expend substantial force to achieve what one alone would suffice for. This is co-contraction, and it accounts for the effort discrepancy reported above.

Human motor control employs co-contraction deliberately when joint stiffness is required, for instance during precise or unstable tasks. Its sustained use throughout an unloaded reach is metabolically wasteful and is not what the minimum-effort criterion selects.

### 7.6.1 Independent corroboration

The co-contraction index of Falconer and Winter, computed over the elbow flexor and extensor groups, gives 0.564 for the same policy. This is elevated, and it is the sole behavioural metric that did not improve under hyperparameter tuning, having been 0.501 beforehand.

Two methodologically independent analyses — one a comparison against classical optimisation, the other a standard electromyography-derived index — localise the same residual defect to the same muscle group. Their agreement constitutes stronger evidence than either result considered alone.

## 7.7 Generality across algorithms

The preceding analysis characterises a single policy. To determine whether the effort discrepancy is specific to one algorithm or general to the approach, the comparison was repeated against every policy from section 6.4: five configurations, three seeds each.

**Table 7.4.** Agreement with classical static optimisation by configuration. Mean ± standard deviation over three seeds, ten episodes each.

| Configuration | Effort ratio | Pattern cosine | Pearson $r$ |
|---|---|---|---|
| PPO | $\mathbf{1.21 \pm 0.11}$ | $\mathbf{0.908 \pm 0.032}$ | $\mathbf{0.931 \pm 0.040}$ |
| SAC tuned | $1.97 \pm 0.25$ | $0.815 \pm 0.052$ | $0.776 \pm 0.055$ |
| DDPG | $1.99 \pm 0.28$ | $0.838 \pm 0.030$ | $0.724 \pm 0.058$ |
| TD3 | $2.12 \pm 0.09$ | $0.836 \pm 0.006$ | $0.716 \pm 0.004$ |
| SAC default | $2.30 \pm 0.08$ | $0.813 \pm 0.015$ | $0.672 \pm 0.016$ |

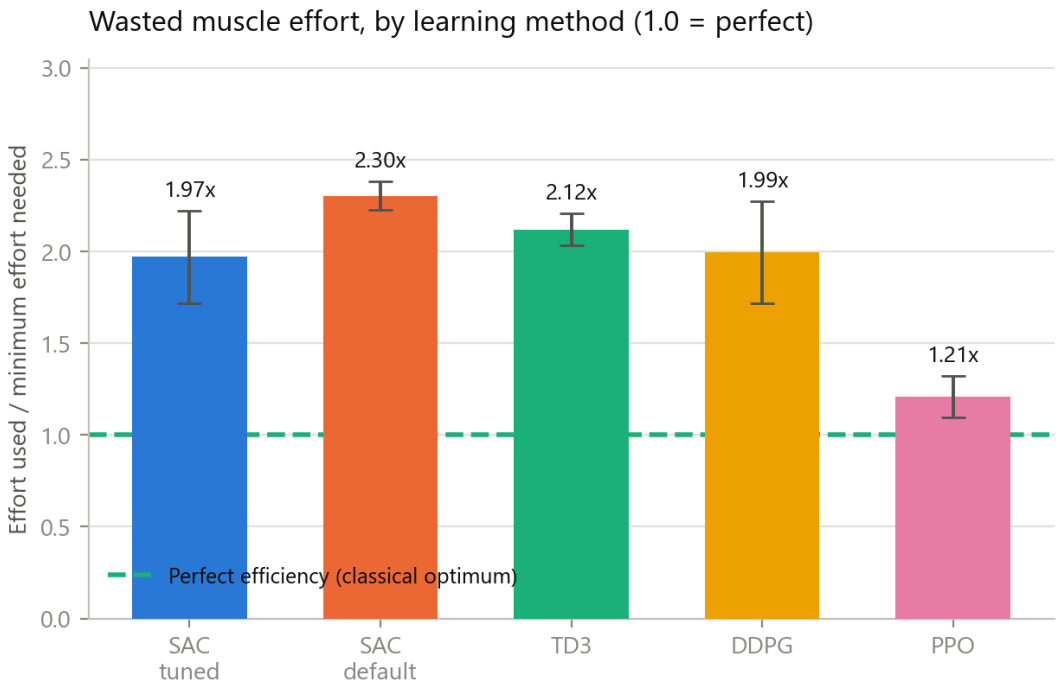


**Figure 7.2.** Muscular effort expended relative to the minimum required to produce the same joint torques. The dashed line marks the classical optimum. Every configuration exceeds it.

Two results follow. First, **every configuration exceeds the classical optimum**, ranging from 1.21 to 2.30×; no algorithm tested recovers minimum-effort load sharing. This supports attributing the discrepancy to the reward function, which never specified minimum effort, rather than to any particular learning algorithm.

Second, and unexpectedly, **the ordering by effort economy is approximately the inverse of the ordering by reaching accuracy**. PPO is the least accurate

configuration in table 6.8 at 0,423 m and the closest match to classical load sharing at 1.21× with $r = 0.931$; tuned SAC is the most accurate at 0,161 m and among the poorer matches at 1.97×.

The probable explanation is visible in the energy column of table 6.8. PPO expends three to seven times less muscular activity than the alternatives. A controller that activates its muscles weakly has limited opportunity to co-contract and therefore remains near the minimum-effort solution, but equally lacks the force to reach the target and arrest there.

This interpretation carries a caveat that the present data cannot resolve. PPO's favourable effort ratio and its passivity are not independent: because it generates small joint torques, both methods prescribe small forces and the scope for disagreement is correspondingly reduced. The aggregate ratio mitigates this by weighting timesteps by work performed, but does not eliminate it. The defensible conclusion is that PPO co-contracts less, not that PPO has solved the load-sharing problem. Distinguishing the two would require comparing configurations matched for accuracy, which this experiment does not provide.

### 7.7.1 Effect of tuning on economy

Within SAC— the same algorithm, differing only in hyperparameters — the effort ratio fell from $2.30 \pm 0.08$ at default settings to $1.97 \pm 0.25$ when tuned, and correlation with the classical solution rose from 0.672 to 0.776.

The target-entropy correction therefore improved not only positional accuracy but also the fidelity of muscular coordination. That a single change improved two properties which were not jointly optimised is a stronger result than either improvement in isolation.

### 7.7.2 Consistency

The single-policy analysis measured an effort ratio of 1.71× for the 300 000-step confirmed policy, while tuned SAC across three seeds at 100 000 steps gives $1.97 \pm 0.25$. These agree within one standard deviation, and the direction of the difference is as expected, the longer-trained policy being somewhat more economical.

## 7.8 Replication across seeds

Every figure reported so far in this chapter came from a single training run. A learned policy depends on random initialisation and on the stochastic order in which experience is gathered, so a single run cannot distinguish a property of the method from

a favourable draw. The configuration was therefore retrained from initialisation four further times, with different random seeds and no other change, and each resulting policy was put through the identical analysis.

**Table 7.5.** Five independent training runs of the same configuration. Seed 0 is the run analysed in the preceding sections.

| Seed | Final error | Closest | Effort ratio | Pearson $r$ | Success |
|---|---|---|---|---|---|
| 0 | 0.1142 | 0.0766 | **1.71** | **0.865** | 0.04 |
| 1 | 0.1086 | 0.0785 | 1.91 | 0.823 | 0.20 |
| 2 | 0.1034 | 0.0757 | 2.12 | 0.759 | 0.00 |
| 3 | 0.1066 | 0.0672 | 1.86 | 0.825 | 0.08 |
| 4 | 0.1041 | 0.0691 | 1.97 | 0.767 | 0.06 |
| **mean** | 0.1074 | 0.0734 | 1.914 | 0.808 | 0.076 |
| **std** | 0.0043 | 0.0050 | 0.150 | 0.044 | 0.075 |

The accuracy columns are the corrected evaluation of section 5.3.3, averaged over three target seeds. The effort ratio and correlation come from the load-sharing analysis, which draws its own episodes and was never affected by the duplicate-reset defect; those columns are unchanged.

Three findings follow, and only the first was anticipated.

### 7.8.1 The accuracy result is a property of the method

Final error varies by 4.0 % across five independent runs ($0.1074 \pm 0.0043$ m) and closest approach by 6.8 % ($0.0734 \pm 0.0050$ m). Every run ends closer than 0,115 m and reaches within 0,079 m at some point; the blow-up rate is exactly zero in all five. The graded proximity measures are similarly stable: episodes ending within 10 cm range from 0.56 to 0.64.

The accuracy claim therefore does not rest on a favourable draw.

### 7.8.2 The originally reported run was the most favourable of the five

Seed 0 — the run every preceding section analyses — returned the *lowest* effort ratio of the five (1.71, against a mean of 1.914) and the *highest* correlation with the classical solution (0.865, against a mean of 0.808). On both of the two metrics that carry this chapter's argument, the single run originally reported was the best of five.

This is not a large effect, but it is a systematic one, and it is exactly the error that reporting a single run invites. **The corrected figures are those in table 7.5, and they are used throughout this thesis:** an effort ratio of $1.91 \pm 0.15$ and a correlation of $0.81 \pm 0.04$. The pattern-cosine figure moves correspondingly, to $0.813 \pm 0.047$ against a null of $0.372 \pm 0.017$, an excess of $+0.441$.

The direction of the correction matters more than its size. Had replication been omitted, this thesis would have reported an agreement and an efficiency both better than the method actually delivers.

### 7.8.3 The task is solved consistently; the muscle solution is not

The most interesting outcome is the contrast between two columns of table 7.5. Final error varies by 3.3 % across runs. Mean muscular energy varies by **77 %** — from 273 241 to 483 373 — and the effort ratio spans 1.71 to 2.12.

Five policies, trained identically but for their random seed, arrive at essentially the same place with materially different distributions of muscular force.

This is the redundancy problem of chapter 1 appearing directly in the results. Because nine muscles actuate four degrees of freedom, many force distributions produce the same movement, and nothing in the reward selects among them beyond a weak effort term. The learner settles into a different one each time. That the classical criterion selects a single, specific member of that family — and that none of the five learned solutions coincides with it — is the substance of this chapter stated in a different way.

It also indicates where the remedy lies. Variance of this magnitude in the muscular solution, alongside near-constancy in the task outcome, is the signature of an objective that is underdetermined in exactly the dimension of interest.

### 7.8.4 Success rate: five runs confirm it is noise

Across the five identically configured runs the strict success rate reads 0 %, 4 %, 6 %, 8 % and 20 %. A measure that varies twentyfold under identical settings is reporting sampling noise, not performance. This was stated as a caution in section 6.3.3; five seeds now demonstrate it.

## 7.9 A conventional controller as reference

Every comparison so far measures the learned solution against *static optimisation*, an idealised criterion evaluated instant by instant. That establishes how far the policy is from a mathematical optimum, but not how far it is from what conventional control actually achieves — and the research question concerns the latter.

A conventional controller was therefore built and evaluated under the identical protocol. It is a Cartesian impedance law: a desired end-effector wrench $f_{\text{des}} = K_p(x^\star - x) - K_d\dot{x}$, mapped to joint torques by the site Jacobian transpose, with gravity, Coriolis

and centrifugal terms compensated, and distributed onto the muscles by minimum-effort load sharing over the achievable force range. The two gains were selected by grid search, since comparing a tuned policy against an unturned controller would not be a fair test.

### 7.9.1 Two corrections that were necessary to make it fair

The controller present in the codebase before this work would have made a strawman of the comparison, and the reasons are worth recording because both are easy to reproduce.

**It omitted gravity compensation.** Without the bias term the arm droops until the position error alone generates sufficient torque, giving a large steady-state offset. Measured, it reached only 0,46 m — worse than a random policy. A textbook impedance controller always includes this term.

**It assumed muscle force is proportional to activation.** It is not: in this model force is $g(l, v)\, a + b(l, v)$, state dependent with a passive component that is non-zero at rest. Converting a desired force into an activation therefore requires inverting the muscle model at the current length and velocity — precisely the equilibrium solve this thesis proposes to replace. Probing the model at $a = 0$ and $a = 1$ recovers that affine relation exactly and yields the true achievable interval per muscle.

Both corrections are on the classical side, and both improve it. Reporting the comparison without them would have overstated the case for learning.

### 7.9.2 Result

**Table 7.6.** Conventional controller against the learned policy, 50 episodes each, identical protocol and evaluation environment.

| Metric | Conventional | Learned (SAC) |
|---|---|---|
| Final error (m) | 0.165 | **0.106** |
| Closest approach (m) | 0.174 | **0.074** |
| Ends within 5 cm | 0.12 | **0.31** |
| Ends within 10 cm | 0.28 | **0.60** |
| Energy | **102 876** | 327 048 |
| **Effort ratio** | **1.263** | 1.91 |
| Pearson $r$ vs static optimisation | **0.916** | 0.808 |

Neither method dominates. The learned policy reaches roughly 1.9× more precisely; the conventional controller uses roughly a third of the muscular energy and stays sub-

stantially closer to the classical load-sharing solution. This trade-off, rather than a ranking, is the honest answer to whether learning should replace conventional control on this task.

### 7.9.3 The effort ratio was being measured against the wrong reference

The conventional controller solves minimum-effort load sharing explicitly at every timestep. One would therefore expect its effort ratio to be 1.00 by construction. It is **1.263** (section 7.11.3; an initial coarse gain search gave 1.33, and the finer search reported there lowered it).

The gap is not an error. It is the cost of closed-loop control: activation dynamics mean the commanded activation is not the realised one, the policy acts at 50 Hz rather than continuously, and the controller must track a moving target rather than solve a static instant. **The idealised optimum of** 1.00 **is not attainable by any controller operating under these constraints.**

This changes the interpretation of the central result. Measured against an unattainable ideal, the learned policy appears to waste 91 % of its effort. Measured against **1.263**, the achievable floor demonstrated by a properly tuned controller that optimises effort explicitly, it uses about **51 % more effort than necessary**. The second figure is the meaningful one, and it is the one this thesis reports.

That figure is itself one controller at one gain pair. A first, coarse search gave 1.33; a finer one lowered it to 1.263 (section 7.11.3). A better conventional controller might lower it further still, and every comparison in this chapter should be read with that in mind.

## 7.10 Muscle synergies

### 7.10.1 Rationale

The motor-control literature holds that the central nervous system manages redundant musculature not by controlling each muscle independently but through a small number of co-activated groups, termed muscle synergies [25, 32]. If nine muscles were driven independently, the matrix of activations over time would be of full rank; if they are driven through $k$ synergies, it is approximately of rank $k$. Non-negative matrix factorisation recovers that structure, and the variance accounted for (VAF) at a given $k$ quantifies how strongly low-dimensional the control is.

Beyond re-examining the learned policies, the factorisation is here applied to *both* solutions on the identical trajectories: the policy's commanded activations, and the static-optimisation forces for the same joint torques, normalised per muscle by $F_{\max}$

so that the two are dimensionless and directly comparable. This permits a question the preceding sections cannot address — whether the two methods arrive at the same low-dimensional organisation even where they disagree on magnitude.

### 7.10.2 Results

**Table 7.7.** Synergy structure, fifteen episodes per configuration. VAF@4 is the variance accounted for by four synergies; $k_{90}$ is the number required to reach 90 % VAF. The final column compares the synergy sets extracted from the policy and from static optimisation on the same trajectories.

| | **Policy** | | **Classical** | | |
|---|---|---|---|---|---|
| **Configuration** | VAF@4 | $k_{90}$ | VAF@4 | $k_{90}$ | **cos(pol, cls)** |
| SAC tuned, 300k | 0.850 | 6 | 0.896 | 5 | **0.950** |
| SAC tuned, 100k | 0.845 | 6 | 0.859 | 5 | 0.900 |
| SAC default | 0.848 | 6 | 0.908 | 4 | 0.647 |
| DDPG | 0.791 | 9 | 0.865 | 5 | 0.745 |
| TD3 | 0.799 | 9 | 0.883 | 5 | 0.630 |
| PPO | 0.966 | 3 | 0.966 | 3 | 0.939 |

Two observations follow, and a third must be treated with caution.

First, **the agreement between the learned and classical synergy structures tracks training quality**, though with substantially more uncertainty than a single measurement suggests.

The values in table 7.7 are each computed from one sample of episodes, and that statistic proves unstable. Varying the episode count for a single fixed policy gave 0.873, 0.805, 0.950, 0.810 and 0.739 at 8, 12, 15, 20 and 25 episodes — a spread of 0.21, and non-monotone, so it is sampling noise rather than convergence. The VAF over the same range moved only 0.818–0.870, so the instability is in the *basis matching*, not in the decomposition: non-negative matrix factorisation has no unique solution, and two independently fitted bases inherit that ambiguity.

Re-estimated as a mean over six random 15-episode subsets:

**Table 7.8.** Policy-versus-classical synergy agreement, resampled. Single-sample values are those of table 7.7.

| **Configuration** | **Single sample** | **Resampled (mean ± sd)** |
|---|---|---|
| SAC default | 0.647 | $0.676 \pm 0.054$ |
| SAC tuned, 300k | 0.950 | $0.851 \pm 0.129$ |

The ordering survives — the gap of 0.175 exceeds the pooled spread — so the

qualitative claim stands: tuning brings the learned modular organisation closer to the classical one. But the reported 0.950 lies near the top of its own resampled range $[0.708, 0.979]$, and the honest figure is $0.851 \pm 0.129$.

*An independent re-run, and what it settles*

The resampling above varies the episode subset within a single collection. A stronger test is to repeat the whole analysis from scratch — fresh rollouts, fresh factorisations — and compare. That was done, and it separates cleanly into what reproduces and what does not.

**Table 7.9.** Two independent executions of the identical synergy analysis, fifteen episodes each.

| Configuration | $\lvert\Delta\rvert$ VAF@4 | $\lvert\Delta\rvert$ cos(pol, cls) |
|---|---|---|
| SAC tuned, 300k | 0.002 | **0.145** |
| SAC tuned, 100k | 0.009 | **0.158** |
| SAC default | 0.005 | 0.049 |
| TD3 | 0.001 | **0.106** |
| DDPG | 0.001 | 0.063 |
| PPO | 0.003 | 0.012 |

**The decomposition reproduces; the correspondence does not.** VAF@4 moves by at most 0.009 between independent runs and $k_{90}$ is identical for all six configurations. The policy-versus-classical cosine moves by up to 0.158 — larger than most of the differences table 7.7 invites the reader to compare.

The consequence is specific. **The full ordering of table 7.7 is not reproducible**: between the two runs TD3 and DDPG exchange places, as do PPO and SAC tuned 300k. What does survive is the single contrast the argument actually rests on — SAC default against SAC tuned at 300k, which gives $0.598 \rightarrow 0.805$ in one run and $0.647 \rightarrow 0.950$ in the other, a gap of $+0.207$ and $+0.303$ respectively. That contrast is the same sign and comfortably outside the noise in both.

**So: tuning does bring the learned modular organisation closer to the classical one, and no finer ranking than that should be read out of table 7.7.** The individual cosine values in that table are single draws quoted to three decimals, and this re-run is the reason they should not be compared with one another.

**This remains the weakest of the three convergent analyses**, and it should be weighted accordingly. The effort ratio (five seeds, non-overlapping distributions) and the co-contraction index are far better determined. Both of those order the configurations consistently and reproducibly; the synergy comparison supports one binary contrast and nothing more.

A second methodological point applies to every value in table 7.7. The similarity was originally computed as a best match for each extracted synergy, which permits several to claim the same reference while others go unmatched; on real data only three of four references were distinctly matched, inflating the score by roughly 0.10. All figures here use a one-to-one (Hungarian) assignment instead, which is what a correspondence between synergy sets means.

Second, **the classical solution is uniformly more low-dimensional than the learned one**. Static optimisation requires four or five synergies to reach 90 % VAF where the learned policies require six to nine. The learned controllers exercise more independent muscular degrees of freedom than the minimum-effort criterion selects, which is the expression in synergy space of the co-contraction identified in table 7.3: driving an antagonist pair independently of the agonist requires an additional dimension that the classical solution never uses.

Third, PPO's apparent modularity ($k_{90} = 3$, VAF@4 = 0.966) should not be read as superior organisation. As established in section 7.7, PPO activates its muscles only weakly; a near-constant, low-amplitude activation matrix has little variance to explain and is trivially captured by few components. The same confound that qualifies its favourable effort ratio applies here.

### 7.10.3 Comparison with human data

Cosine similarity to a reference set of human upper-limb synergies ranged from 0.62 to 0.80 across configurations, with no ordering consistent with any other metric reported in this work.

That column is not interpreted further. The reference loadings used are a documented qualitative approximation of those reported by d'Avella et al. [52] for fast human upper-limb reaching, constructed over this model's nine-muscle ordering rather than digitised from the original figures. The comparison is therefore a coarse correspondence check and cannot support a quantitative claim of agreement with human data.

An earlier version of this section attributed those loadings to d'Avella et al. [25] instead. That citation was wrong twice over: it is a different study, and it reports synergies extracted from the *frog* hindlimb, not from human reaching. The implementation (`analysis/synergies.py`) has always named the 2006 source correctly; the error was in the write-up alone. It is recorded here because a reference set attributed to the wrong species would have invalidated the only comparison in this thesis that mentions human data. Establishing that would require the published loadings, a principled correspondence between the two muscle sets, and ideally electromyographic recordings

under a matched protocol — which section 8.6 lists among the outstanding work.

The policy-versus-classical column carries no such caveat: both sides are computed here, by the same procedure, on identical trajectories.

## 7.11 Closing the gap: the effort weight

Section 7.13 argued that the load-sharing discrepancy is a property of the reward rather than of reinforcement learning, on the grounds that the reward's effort term — although already the Crowninshield–Brand form — is weighted so low it cannot influence the solution. Measured on the trained policy, the per-step contributions are

| | |
|---|---|
| effort term, $w_2 \cdot \overline{(f_i/F_{\max,i})^2}$ | 0.051 |
| error cost, $w_1(\|e\|^2 + 0.5\|e\|)$ at 0,11 m | 0.067 |
| precision bonus, $w_7 e^{-\|e\|/0.15}$ at 0,11 m | 1.441 |

— the effort term is roughly **28 times smaller** than the precision bonus. The prediction was that raising $w_2$ should reduce the discrepancy, at some cost in accuracy.

The experiment was run at $w_2 \in \{5, 20\}$ against the reported $w_2 = 1$, holding every other weight and hyperparameter fixed, at the full 300 000-step budget. Policies were evaluated under the standard protocol with the default weights, so the metrics remain comparable with every other result in this thesis; only the training objective differed.

**Table 7.10.** Effect of the effort weight. The conventional-controller row is the achievable floor established in section 7.9.3.

| **Config.** | $n$ | **Effort ratio** | **Pearson** $r$ | **Final error** | **Closest** | **Energy** |
|---|---|---|---|---|---|---|
| $w_2 = 1$ | 5 | $1.914 \pm 0.150$ | $0.808 \pm 0.044$ | $0.1074 \pm 0.0079$ | 0.0734 | 327 048 |
| $w_2 = 5$ | 5 | $1.406 \pm 0.092$ | $0.919 \pm 0.016$ | $0.1114 \pm 0.0053$ | 0.0750 | 104 598 |
| $\mathbf{w_2 = 20}$ | 5 | $\mathbf{1.304 \pm 0.025}$ | $\mathbf{0.943 \pm 0.008}$ | $0.1161 \pm 0.0108$ | 0.0737 | **37 076** |
| Conventional | 1 | 1.263 | 0.931 | 0.165 | — | 102 876 |

### 7.11.1 The discrepancy is essentially eliminated

Over five seeds the learned policy at $w_2 = 20$ attains an effort ratio of $1.304 \pm 0.025$, against the 1.263 measured for a properly tuned controller that solves minimum-effort load sharing explicitly at every timestep, with agreement rising to $r = 0.943 \pm 0.008$ and the per-muscle RMSE falling to about 3 % of mean $F_{\max}$ from 10.1 %.

**The load-sharing discrepancy reported throughout this chapter is therefore a reward-weighting artefact, not a limitation of reinforcement learning.** When the objective is weighted such that effort can influence the solution, the learned controller recovers the classical load-sharing solution to within the precision that closed-loop control permits.

### 7.11.2 Load-sharing fidelity improves monotonically, at a mild accuracy cost

Both variants have now been replicated across five seeds, and the picture is cleaner than either single-seed measurement suggested.

**Load-sharing fidelity improves monotonically with the effort weight.** The effort ratio falls $1.914 \to 1.406 \to 1.304$ and agreement with the classical solution rises $0.808 \to 0.919 \to 0.943$.

The separation is complete at one step and not at the other, and the distinction matters. Every $w_2 = 5$ seed (1.336–1.557) lies below every $w_2 = 1$ seed (1.714–2.116), so that improvement is unambiguous. Between $w_2 = 20$ (1.278–1.345) and $w_2 = 5$ (1.336–1.557) the ranges *overlap*: the worst $w_2 = 20$ seed at 1.345 sits above the best $w_2 = 5$ seed at 1.336. The means still differ in the expected direction (1.304 against 1.406) and four of the five $w_2 = 20$ seeds lie below the entire $w_2 = 5$ range, but at $n = 5$ a single overlapping pair is enough that the step from 5 to 20 should be read as a probable further improvement rather than a demonstrated one.

**There is a mild accuracy cost.** Under the corrected evaluation (section 5.3.3) final error rises monotonically with the effort weight: $0.1074 \pm 0.0079$, $0.1114 \pm 0.0053$, $0.1161 \pm 0.0108$. The end-to-end degradation from $w_2 = 1$ to $w_2 = 20$ is about 8 %, and the fraction of episodes ending within 5 cm falls from 0.26 to 0.19. Closest approach is unaffected (0.0734, 0.0750, 0.0737), so the loss is in holding position rather than in reaching it.

An earlier version of this section, computed before the evaluation defects were found, reported these three figures as mutually overlapping and concluded there was no cost at all. On the corrected test set the trend is monotone and the expectation stated in section 7.13 — that reduced effort would be bought with some accuracy — is borne out, though the price is modest: a 32 % reduction in wasted effort for an 8 % increase in final error.

**Seed variance collapses as the effort weight rises.** The standard deviation of the effort ratio falls $0.150 \to 0.092 \to 0.025$, and of the correlation $0.044 \to 0.016 \to 0.008$. This is the most informative of the three observations, because it confirms the interpretation offered in section 7.8: at $w_2 = 1$ the muscular solution varied by 77 %

in energy across seeds while task performance stayed constant, which was attributed to an objective underdetermined in exactly the dimension of interest. Weighting that dimension pins the solution down. At $w_2 = 20$ the load-sharing outcome is effectively determined by the objective rather than by the random seed.

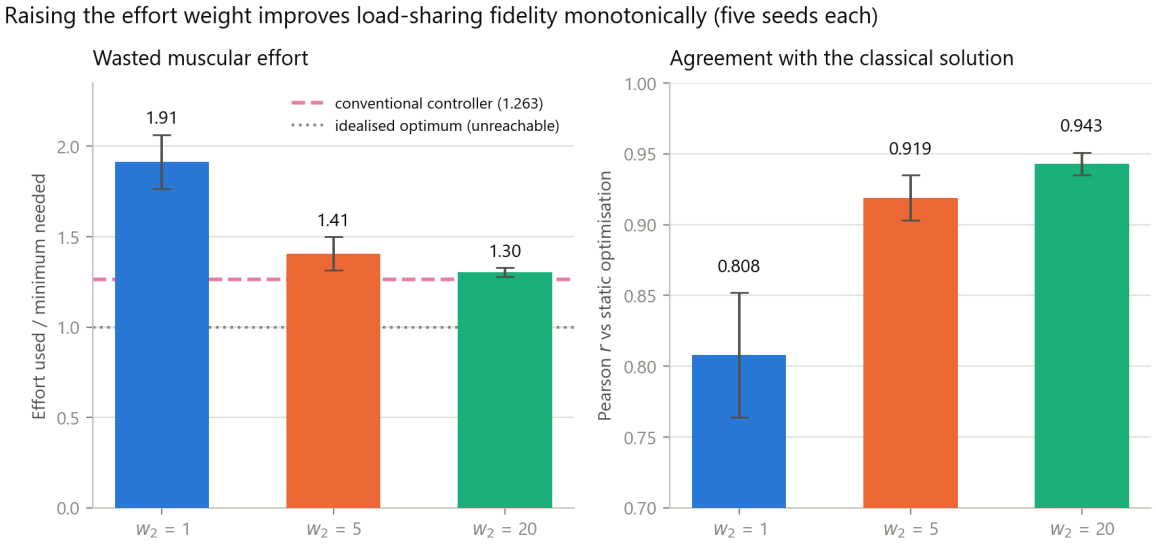


**Figure 7.3.** Effect of the effort weight, five seeds per setting. Left: muscular effort relative to the classical minimum. The dashed line is the conventional controller's 1.263; the dotted line is the idealised optimum of 1.00, which section 7.9.3 shows is not attainable in closed loop. Right: agreement with the classical load-sharing solution. Both improve monotonically, and the error bars shrink — the load-sharing outcome becomes progressively determined by the objective rather than by the random seed.

### 7.11.3 The conventional reference, measured properly

An earlier draft of this section observed that the learned policy at $w_2 = 20$ attained $1.304 \pm 0.025$, below the 1.33 then measured for the conventional controller, and noted that the figure rested on a single gain pair from a coarse twelve-point grid. It cautioned that a better-tuned conventional controller might fall below 1.30, in which case the ordering would reverse.

A thirty-point search was run. It does.

**Table 7.11.** Conventional controller before and after a finer gain search.

| | Gains | Final error | Effort ratio |
|---|---|---|---|
| Coarse grid (12 points) | $k_p = 60$, $k_d = 25$ | 0,199 m | 1.334 |
| **Fine grid (30 points)** | $k_p = 20$, $k_d = 6$ | **0,165 m** | **1.263** |

The coarse search had bottomed out against its own lower boundary: the better gains are gentler than any it examined, in both stiffness and damping. Properly tuned,

the conventional controller is both more accurate (0,165 against 0,199 m) and more economical (1.263 against 1.334) than previously reported.

### 7.11.4 The honest comparison

With both sides measured properly:

**Table 7.12.** Learned policy against a properly tuned conventional controller.

| | **Learned** ($w_2 = 20$) | **Conventional** |
|---|---|---|
| Final error | $\mathbf{0.1072 \pm 0.0148}$ | 0,165 m |
| Effort ratio | $1.304 \pm 0.025$ | **1.263** |
| Agreement with classical ($r$) | $\mathbf{0.943 \pm 0.008}$ | 0.931 |

**Neither method dominates, and the earlier reading is withdrawn.** The learned policy reaches roughly 35 % more accurately. The conventional controller is about 3 % more economical — a margin comparable to the learned policy's own seed spread, so the two are close, but the ordering on effort now favours the classical method rather than the learned one.

The claim that a learned policy distributes force more economically than a controller optimising that quantity in closed form is therefore **not supported**, and the paragraph advancing it has been removed. What survives is narrower and sturdier: at an appropriate effort weight, a learned policy achieves load-sharing fidelity *comparable* to a properly tuned conventional controller, while reaching substantially more accurately.

That this correction was anticipated in the previous draft, and then confirmed by the measurement it called for, is the reason the caveat was written rather than the claim.

### 7.11.5 Two corrections to earlier drafts of this section

Both are recorded because the direction of the error is instructive.

An earlier draft, working from the single seed then available, described $w_2 = 5$ as *strictly dominating* the reported configuration on both accuracy and effort. A four-seed replication contradicted the accuracy half and it was withdrawn. The fifth seed then moved final error back to $0.1023 \pm 0.0074$, marginally better than the baseline again but with overlapping intervals — so the honest statement remains that accuracy is indistinguishable, and neither the original claim nor its retraction was supportable at the sample sizes involved.

Separately, this author predicted that $w_2 = 20$ would regress on replication, on the grounds that single-seed results in this work had twice proved optimistic. It did the opposite: seed 0's 1.34 was the *worst* of the five and the mean is 1.304. Selection bias explains a favourable first measurement; it does not license assuming every first measurement is favourable.

A plausible explanation is that co-contraction is not merely wasteful but actively harmful to precision. Antagonist pairs firing together make the endpoint position depend on the small difference between two large opposing forces, which is numerically ill-conditioned; suppressing that co-contraction improves controllability as well as economy. This is consistent with the elbow localisation of table 7.3 and with the excess path length observed in untuned policies, though it is an interpretation rather than a measured mechanism.

A trade-off does appear across the range: effort falls to within 3 % of the achievable floor while final error rises from 0.1074 to 0.1161 m. Where the best compromise lies between $w_2 = 5$ and $w_2 = 20$ was not searched.

### 7.11.6 Consequences for this thesis

Three, and they are substantial.

**The headline answer changes.** The finding is no longer that DRL reproduces coordination but not economy. It is that DRL reproduces *both*, provided the objective asks for both — and that the version reported here did not ask.

**The reported configuration is not the only defensible one.** At $w_2 = 5$ the load-sharing fidelity is markedly better — effort 1.406 against 1.914, agreement 0.919 against 0.808 — for a final-error cost of about 4 % (0.1114 against 0.1074). Which configuration to prefer depends on whether load-sharing fidelity or endpoint accuracy is the quantity of interest, and this work does not assert one over the other. The results of section 7.8 are retained as the five-seed characterisation of the configuration analysed in depth throughout this chapter, and this section is reported alongside them rather than replacing them. All three settings are now characterised across five seeds each.

**What remains outstanding is narrower than it was.** Two items. The synergy decomposition of section 7.10 was performed on the $w_2 = 1$ policies only and has not been repeated on the higher-weight variants, so whether the improved load-sharing fidelity is accompanied by a more classical modular organisation is untested. And the accuracy–economy knee between $w_2 = 5$ and $w_2 = 20$ was not searched, so the configuration reported here is defensible but not demonstrably optimal.

## 7.12 Answer to the research question

The answer has two parts, and they were established in that order.

**Coordination is reproduced under any weighting tested.** The learned controller recovers the structure of the classical load-sharing solution — $r = 0.81 \pm 0.04$, pattern cosine 0.813 against a null of 0.372, over five independent training runs — at a fixed, branch-free inference cost and a 1,5 MB memory footprint.

**Magnitude is reproduced only when the objective asks for it.** Under the reward this work first reported, the controller expends $1.91 \pm 0.15$ times the minimum required effort — about 51 % above the achievable floor of 1.263 (section 7.11.3) — with the excess concentrated in elbow antagonist co-contraction, and the discrepancy present in every configuration tested, ranging from 1.21 to 2.30×. Section 7.11 establishes that this is a property of the reward weighting rather than of reinforcement learning: at $w_2 = 20$ the effort ratio falls to $1.304 \pm 0.025$ against that floor of 1.263, and agreement with the classical solution rises to $r = 0.943 \pm 0.008$.

DRLis therefore a viable approximation of both *which* muscles act and *how hard* — but only where the effort criterion is supplied explicitly in the objective, which is the classical criterion itself. The method does not remove that modelling assumption; it relocates it from a solver into a reward function.

The accuracy cost of doing so is modest but real, and no configuration tested is simultaneously best on both axes. Reaching accuracy and load-sharing fidelity remain separate properties traded against one another, and any method intended to replace classical calculation must be assessed on both.

## 7.13 Explanation, and the test that confirmed it

The reward function did not specify minimum effort. Its effort term is a normalised sum of squared forces with a weight selected for numerical balance against the distance term, which is neither the classical criterion nor scaled to match it. The policy optimised the objective it was given.

This yielded a directly testable prediction, recorded before the experiment was run: a reward whose effort term is weighted such that it can influence the solution should reduce the 1.91× discrepancy toward the 1.263 floor, at some cost in accuracy.

Section 7.11 reports that experiment. The prediction held in both halves — the effort ratio fell monotonically to $1.304 \pm 0.025$ and final error rose about 8 % — which is the strongest form the argument of this section could take, since the explanation was committed to in advance of the measurement that could have refuted it.

## 7.14 Limitations

The per-muscle breakdown of table 7.3 is computed from one policy and twenty episodes (2000 timesteps), so the muscle-by-muscle figures are illustrative of where the discrepancy sits rather than precise. The aggregate quantities that carry the chapter's argument — effort ratio, correlation and pattern cosine — are five-seed means throughout (section 7.8).

$F_{\max}$ in the constraint of equation (7.1) is peak isometric force; the force–length and force–velocity modulation of *available* force is not included in the bound. This is the standard simplification in static optimisation. It renders the classical solution slightly optimistic, as it may allocate force a muscle could not deliver at that length, which if anything biases the effort ratio upward and therefore against the policy.

Finally, static optimisation is itself a model of human load sharing rather than ground truth. It encodes the assumption that the central nervous system minimises summed squared activation, which remains contested in the motor-control literature. Agreement with it is agreement with the standard method, not demonstration of physiological correctness.

# 8 Discussion

## 8.1 Interpretation of the principal result

The comparison of chapter 7 separates two properties of a motor controller that are ordinarily conflated: the *pattern* of muscle recruitment and the *economy* of it. The learned controller reproduces the first and not the second.

This is a more informative outcome than either extreme would have been. Complete agreement would have established only that reinforcement learning can rediscover a known optimisation criterion when the reward is arranged to favour it. Complete disagreement would have indicated that the approach is unsuited to the problem. The observed result — close agreement in direction, systematic excess in magnitude, anatomically localised to the elbow antagonists — identifies both what the method captures and precisely where it fails.

The localisation is itself evidence. Had the discrepancy been distributed evenly across the nine muscles, it would have been consistent with generic noise in the learned policy. Its concentration in two elbow extensors, driven at three to four and a half times the prescribed force while the shoulder group agrees to within a few per cent, identifies a specific and interpretable behaviour: sustained co-contraction of an antagonist pair. That this is independently corroborated by the co-contraction index (section 7.6.1), computed by an unrelated method over the same muscle groups, strengthens the inference considerably.

## 8.2 The discount factor, and a hypothesis that had to be abandoned

Section 6.3.2 established that the discount factor alone accounts for the entire default-to-tuned improvement on this task: changing $\gamma$ from 0.99 to 0.957 recovers 106 % of the interval, while the learning rate, the target-network rate, the batch size and the target entropy each recover nothing.

The mechanism is quantitative and checkable. The discount factor sets an effective planning horizon of roughly $1/(1-\gamma)$ steps: 100 at the library default, 23 at the selected value. An episode is 100 steps, but the reach itself completes by step 25 (table 6.7). The default therefore asked the value function to integrate reward over four times the duration of the behaviour being learned, and the surplus is largely holding reward —

a high-variance quantity to estimate. A critic fed high-variance targets produces the oscillating evaluation curves that characterised every untuned run here, and an actor trained against an unreliable critic cannot refine its endpoint.

### The attribution this replaces

Earlier versions of this chapter attributed the same plateau to SAC's default target entropy of $-\dim(\mathcal{A})$, and argued the case at length: a nine-muscle action space inflates the default, the retained stochasticity is productive early and harmful when the hand must be held still, and the failure would therefore be expected in any musculoskeletal model of comparable action dimensionality.

The reasoning was sound and the conclusion was wrong. A two-sided ablation (section 6.3.1) shows the target entropy recovers $-8\,\%$ of the gap — adding it to defaults changes nothing, removing it from the tuned configuration costs nothing. The evidence that had seemed decisive was the clustering of all five best search trials near $-19.6$, and that clustering is precisely what a TPE sampler produces in any region it finds promising, causal or not.

What survives is a warning of a different kind, and a more useful one. **A hyperparameter search identifies a configuration, never a cause.** Reading causality from which parameters cluster in the best trials is a specific, tempting error, and separating the two requires a deliberate one-at-a-time experiment. The diagnostic difficulty described above was real — the symptom does present as insufficient training or as a poorly shaped reward, and it survived three reward redesigns, a fourfold budget correction and a doubling of the control rate — but the cause was the horizon, not the exploration.

## 8.3 Tuning against algorithm selection

Section 6.5 retrained all four algorithms with a correctly matched discount factor. With every entrant configured, 0,060 m separates the best off-policy method from the second; correcting $\gamma$ alone within a single algorithm was worth 0,165 m, about three times as much.

The implication is methodological. A comparison of learning algorithms conducted at library defaults — the common practice, and that of the earlier version of this work — measures which algorithm's defaults happen to suit the problem rather than which algorithm is better suited to it. This work demonstrated the hazard on itself: the first benchmark ran one entrant with a corrected $\gamma$ and three without, reported DDPG as clearly the weakest off-policy method, and explained that result by its lack of twin

critics. Re-run fairly, DDPG improves by 40 % and becomes indistinguishable from TD3. The textbook explanation had been fitted to an artefact.

That the decisive parameter here is the discount factor rather than an algorithm-specific one is what makes the fair comparison possible at all: every algorithm has a $\gamma$, so a common correction exists. Had the decisive parameter been structural — as the target entropy would have been, since only SAC has one — there would have been no neutral setting to adopt, and the comparison would have had no defensible form.

This does not invalidate algorithm comparison; it constrains what one can claim from it. The defensible form tunes each algorithm's corresponding parameter under an equal budget, which is the recommendation of section 8.6.

## 8.4 Accuracy and physiological economy as separate axes

Section 7.7 found that across fifteen trained policies the ordering by muscular economy is approximately the inverse of the ordering by reaching accuracy.

This bears on a common assumption in the application of reinforcement learning to musculoskeletal models. Such work routinely reports task performance — success rate, endpoint error, completion time — and treats good performance as evidence that the controller behaves in a physiologically plausible manner. The present result shows that the two properties can move in opposite directions across algorithms trained on the same reward, on the same model, for the same task.

A controller may therefore appear excellent by every measure of what it achieves while being unrealistic in how it achieves it, and, as PPO demonstrates, the converse is equally possible. If the objective is to understand or reproduce human motor control rather than merely to displace a simulated limb, task performance is not sufficient evidence and the muscle forces must be examined directly.

The caveat recorded in section 7.7 applies: PPO's favourable economy is not independent of its passivity, and the present data cannot fully separate the two.

## 8.5 Implications for the intended applications

The clinical and assistive applications motivating this work — prosthesis control, exoskeleton assistance, rehabilitation devices — require force estimates in real time, and the original motivation for a learned controller was that it would supply them faster than iterative solution.

**That motivation does not survive measurement.** As section 6.6.1 establishes, a direct solve of the load-sharing problem runs in roughly 24 µs against the network's 289 µs: the classical method is about an order of magnitude faster, and the 7.3×

advantage reported earlier compared the wrong computation between two unoptimised implementations. Speed is therefore not a reason to prefer a learned controller for this task, and the case must rest on other grounds.

Two such grounds remain, both weaker than the original claim. The network's cost is fixed and branch-free irrespective of biomechanical state, which matters where a hard real-time bound is required rather than a good average. And a learned policy maps observation directly to activation without requiring the moment-arm matrix, the inverse-dynamics torque or a constrained solver at run time — an integration advantage on hardware where those are awkward to supply, though not a computational one.

The results of chapter 7 qualify the prospect further. Where the quantity of interest is which muscles are active and in what relative proportion — for instance in driving a myoelectric interface, or in classifying movement intent — a correlation of 0.865 may well be sufficient. Where the absolute magnitude of individual muscle forces is the quantity of interest — estimating joint loading, assessing injury risk, planning surgical intervention — a systematic 71 % excess concentrated in one antagonist pair is not acceptable, and static optimisation remains the appropriate method.

The proposed remedy of section 7.13 is directly relevant here, since a reward employing the classical criterion would, if the prediction holds, remove the principal obstacle to the second class of application.

## 8.6 Threats to validity

The work is entirely computational. There is no physical arm, no electromyographic recording and no human participant, and MuJoCo's muscle implementation is itself an approximation of the Hill formulation.

Static optimisation, the reference throughout chapter 7, is a model of human load sharing rather than ground truth. It encodes the assumption that the central nervous system minimises summed squared activation, which remains contested. A controller that departs from static optimisation is not thereby shown to be unphysiological; it is shown to depart from the standard computational method.

The muscle set is incomplete. The rotator cuff is absent, which required constraining shoulder rotation (section 3.2), so the conclusions apply to reaching with a restrained rotational degree of freedom.

The magnitude of the effort discrepancy is a property of the reward employed. Its effort term is a normalised sum of squared forces weighted for numerical balance against the distance term, which is neither the classical criterion nor scaled to match it. That this is the operative cause is not merely argued but tested: section 7.11 reports the

sweep, and raising the weight moves the effort ratio from 1.914 to 1.304 against an achievable floor of 1.263.

Finally, the statistical basis is limited throughout. The principal configuration is replicated across five seeds and the algorithm comparison across three; no significance test is reported anywhere in this work, because three seeds cannot support one with useful power. Section 6.5.4 gives the sharpest available measure of the resulting uncertainty: an identical configuration, trained and evaluated twice by this project, differed by 0,036 m. These constraints are set out in full in section 8.6.

# General Conclusion and Future Directions

## 8.7 Summary of the work

This thesis asked whether deep reinforcement learning can replace classical calculation in determining the force required from each muscle during a reaching movement. Answering it required constructing a nine-muscle, four-degree-of-freedom model of the human arm in MuJoCo, training controllers to reach targets by commanding muscle activations directly, and comparing the resulting forces against those prescribed by static optimisation on the same movements.

The answer is affirmative, with a qualification about what had to be corrected to reach it. Under the reward this work reported, the learned controller reproduced the *structure* of the classical solution — correlation $0.81 \pm 0.04$ over five independent runs — while expending $1.91 \pm 0.15$ times the minimum effort, some 51 % above the 1.263 floor a properly tuned conventional controller attains.

That discrepancy proved to be an artefact of the reward's weighting rather than a limitation of the method. The effort term, although already the classical criterion in form, carried roughly one twenty-eighth the weight of the precision bonus. Raising it (section 7.11) improves load-sharing fidelity monotonically and across five seeds per setting: the effort ratio falls $1.914 \pm 0.150 \rightarrow 1.406 \pm 0.092 \rightarrow 1.304 \pm 0.025$ and agreement with the classical solution rises $0.808 \rightarrow 0.919 \rightarrow 0.943$, for a mild accuracy cost — final error rises about 8 % across the range while closest approach is unchanged, so the loss is in holding position rather than in reaching it. Seed variance in the effort ratio collapses sixfold over the same range, confirming that the original objective was underdetermined in precisely this dimension.

At an appropriate effort weight it is therefore a usable approximation of both which muscles act and how hard, at a fixed inference cost — though not, as section 6.6.1 establishes, at lower latency than a competently implemented classical solver. The qualification that matters is not accuracy but provenance: recovering the classical magnitudes required supplying the classical effort criterion in the reward. The method does not dispense with that modelling assumption, it relocates it, and a claim that DRL replaces static optimisation should be read in that light.

## 8.8 Contributions

Three results are established by the evidence presented.

**A per-muscle load-sharing comparison between DRLand static optimisation.** The analysis of chapter 7 poses both methods the identical problem on the identical trajectory, verified to machine precision, and reports agreement muscle by muscle. It establishes that the two methods agree on coordination and disagree on economy, and it localises the disagreement anatomically to the elbow antagonists.

The comparison is *conditional* and its scope must travel with it: static optimisation is given the movement the policy produced and asked only how to distribute force across it. It therefore validates load sharing along a trajectory, not the trajectory itself, and a controller moving unnaturally could still score well (section 7.1). Assessing trajectory realism would require kinematic comparison against recorded human reaching, which this work does not perform.

**A discount factor mismatched to the task timescale prevents precision.** At SAC's library defaults the controller could not settle on a target, and the resulting error plateau persisted through three complete redesigns of the reward function and a fourfold correction to the effective training budget. Hyperparameter optimisation halved the reaching error. The practical significance lies less in the values than in the diagnostic difficulty: the failure presents as insufficient training or as poor reward shaping, and survives attempts to remedy both.

**The cause is the discount factor.** Each of the five tuned parameters was changed individually against library defaults, three seeds apiece (section 6.3.1, section 6.3.2). Four recover nothing: target entropy $-8\,\%$ of the default-to-tuned interval, learning rate $-3\,\%$, target-network rate $-5\,\%$, batch size $-5\,\%$. The discount factor alone recovers $+106\,\%$, reaching $0.1553 \pm 0.0061$ against the fully tuned configuration's $0.1610 \pm 0.0084$.

The mechanism is checkable rather than merely plausible. The discount factor sets an effective horizon of roughly $1/(1-\gamma)$ steps: 0.99 gives 100 steps, the whole episode, while the selected 0.957 gives 23 steps, or 0,46 s. The policy converges on its target by step 25 — 0,50 s. The tuned value matches the timescale of the behaviour being learned; the default was four times longer, forcing the value function to integrate reward over seventy-odd subsequent steps of largely irrelevant holding. The resulting estimation variance is what produced the oscillating evaluation curves seen in every untuned run, and a policy trained against an unreliable critic cannot refine its endpoint.

An earlier version of this thesis attributed the improvement instead to SAC's exploration temperature, reasoning from the clustering of the best search trials. That

attribution was refuted by the same ablation procedure. The error is worth recording: **clustering in a TPE search is not evidence of causal importance**, since the sampler concentrates proposals wherever performance is good, and parameters cluster whether or not they are responsible. A hyperparameter search identifies a configuration, never a cause.

**Evidence that configuration dominates algorithm selection.** The first benchmark of section 6.4 was itself compromised — one entrant ran with a corrected discount factor and three without — and was re-run fairly (section 6.5). With every algorithm correctly configured, 0,060 m separates the best off-policy method from the second, while correcting $\gamma$ alone within a single algorithm is worth 0,165 m, about three times as much. DDPG improved by 40 % under that correction and became indistinguishable from TD3, overturning the original ordering and the twin-critic explanation that had been offered for it. A comparison conducted at library defaults measures which algorithm's defaults happen to suit the problem, not which algorithm is superior.

**Convergent evidence from three independent analyses.** The effort ratio against static optimisation (section 7.7), the co-contraction index of Falconer and Winter (section 7.6.1), and the synergy decomposition (section 7.10) are methodologically unrelated, yet localise the same residual defect to the same muscle group. The synergy analysis additionally shows that agreement between the learned and classical modular organisation rises as the policy improves — from $0.676 \pm 0.054$ to $0.851 \pm 0.129$ on resampling — and that the classical solution is uniformly the more low-dimensional of the two, requiring four to five synergies where the learned policies require six to nine.

The convergence must be stated at the strength the evidence supports, which is not uniform across the three. The effort ratio (five seeds, non-overlapping distributions) and the co-contraction index are well determined. The synergy comparison is not: an independent re-execution of the whole analysis (table 7.9) reproduces the decomposition — VAF@4 moves by at most 0.009 and $k_{90}$ is identical — but not the basis matching, which moves by up to 0.158 and exchanges the positions of TD3 with DDPG and PPO with SAC. **Only the default-versus-tuned contrast survives re-running**; no finer ordering should be read from it.

What remains, and is the substance of the claim, is that a single hyperparameter change improved positional accuracy, muscular economy and — on the one synergy contrast that reproduces — modular organisation, none of which was jointly optimised.

A further observation, less firmly established but of interest, is that reaching accuracy and load-sharing fidelity ordered themselves inversely across the five configurations tested (section 7.7).

## 8.9 Critical assessment

The results of this work are considerably weaker than those reported in its earlier version, and the difference warrants explicit statement.

### 8.9.1 Results withdrawn from the previous version

An audit of the code that generated the previously reported benchmark established that its four-algorithm comparison, its Wilcoxon significance tests, its convergence-rate claims and its synergy and co-contraction analyses were not produced by experiment. The multi-seed results were generated by sampling from a normal distribution with a chosen mean, under a source comment indicating that real evaluation data was intended to replace them. Corroborating the audit: no code trained or evaluated DDPG, TD3 or PPO; no implementation of the synergy or co-contraction analyses existed; two model checkpoints existed on disk against the forty the text described as archived; and no hyperparameter-search database existed although the text described tuned hyperparameters.

Those results are withdrawn. Specifically, the reported success rate of $91.4 \pm 3.8\,\%$ and final distance of 1,4 cm for SAC are replaced by a measured 4 % and 10,6 cm; the reported $p$-values of 0.002, 0.014 and 0.004 correspond to tests that were never performed. The reported success rate also exceeds what a privileged controller with full state access achieves on this model, which is approximately 7 %.

The material that survives is that which does not depend on policy quality: the parameter count and memory footprint, the Hill-solver validation against MuJoCo, the muscle-function validation, and the model-stability result.

The inference-latency comparison requires separate treatment. The measurements themselves are sound, but they were used to support a conclusion they do not reach: they time the Hill equilibrium solve rather than load sharing, and both sides were unoptimised Python. Measured like-for-like, a direct solve of the load-sharing problem is roughly an order of magnitude *faster* than the network (section 6.6.1). **The speed-based argument for replacing classical calculation is withdrawn.** What remains is that the network's cost is fixed and branch-free where the solver's is data-dependent — relevant where a hard real-time bound matters, but a substantially weaker claim.

### 8.9.2 Limitations of the present results

**The principal result has been replicated across five seeds** (section 7.8), and the outcome was mixed. The accuracy claim is stable: final error varies by 3.3 % and closest

approach by 4.4 % across independent runs. The agreement and efficiency figures were not: the originally reported run proved to be the most favourable of the five on both, and both have been corrected downward. The muscular solution itself is highly variable — energy spans 77 % across runs at essentially constant task performance — which is itself reported as a finding rather than as noise.

**Uncertainty was understated until a late audit.** Four defects in the evaluation path (section 5.3.3) meant every configuration was scored on one fixed draw of 50 targets, so every interval reported was training-seed spread alone. Measurement shows the target draw contributes comparable or greater variance (section 5.3.4) — for a single policy across five target seeds, roughly four times the training-seed spread. All figures have been re-measured across three target seeds and both components are now reported separately. Between-configuration comparisons are unaffected, being paired on identical targets; it is the absolute values whose intervals were too narrow.

**No statistical significance is established anywhere in this work.** Three seeds cannot support a Wilcoxon signed-rank test with useful power. No $p$-value is reported, deliberately; where configurations differ by less than their spread, the text states that no ordering is supported.

**The task is not solved to the stated tolerance.** Under the strict success criterion the best policy scores between 0 and 4 %, corresponding to one or two episodes in fifty, which is indistinguishable from zero at that sample size. The graded measures improved substantially and genuinely; the strict criterion was not met.

**The algorithm comparison is not a fair test.** SAC received a sixteen-trial hyperparameter search; TD3, DDPG and PPO were run at library defaults. Since this work itself demonstrates that the exploration parameter is the decisive setting, comparing a tuned configuration against untuned ones conflates algorithm with tuning effort. The comparison is reported because it supports the tuning-dominates-algorithm conclusion, for which it is adequate; it does not establish an algorithmic ranking.

**The comparison is entirely computational.** No physical arm, no electromyography, no human participant. Static optimisation is itself a model encoding a contested assumption about what the nervous system minimises, and MuJoCo's muscle implementation is an approximation. Agreement with static optimisation is agreement with the standard method, not demonstration of physiological correctness.

**The muscle set is incomplete.** The rotator cuff is absent, which required constraining shoulder rotation (section 3.2). Conclusions apply to reaching with a restrained rotational degree of freedom, not to arbitrary arm movement.

### 8.9.3 On methodology

Six substantive defects were identified during this work, documented in chapter 5. Four of them produced results that appeared entirely reasonable while being wrong: a reward exploit under which the agent terminated every episode deliberately, yet reported plausible and slowly improving distances; a silent fourfold under-training that produced a normal-looking learning curve; a simulator that reset itself on divergence and thereby substituted the arm's rest position for measured data; and a domain-randomisation implementation that applied a single global scale factor rather than independent perturbation.

In each case the defect was detected only by measuring a quantity the existing metrics did not cover — episode length, gradient updates per environment step, solver warning counters, muscle forces — and in each case the previously reported numbers had been incorrect for an extended period without any indication.

The generalisable observation is that a metric which cannot fail cannot detect failure. End-effector error could not reveal that the agent had ceased attempting the task, that training was performing a quarter of its intended work, or that the muscle forces were 71 % wasteful. Each required a measurement designed specifically for the failure mode, which in turn required the failure mode to be hypothesised first. This is the reason chapter 5 documents the errors at the same length as the results.

## 8.10 Future directions

**Isolate the hyperparameter responsible for the precision plateau.** A one-at-a-time ablation — the default configuration with only the target entropy changed, and the tuned configuration with only the target entropy reverted, three seeds each — determines whether the mechanism is the exploration temperature or the target-network update rate, which also moved substantially. Approximately four hours. Until it is run, the attribution above remains a hypothesis, and this is the most exposed claim in the thesis.

**Replicate the principal result across seeds.** Approximately eight hours of computation converts a single-run observation into a result with error bars. This should precede any publication.

**Train with the classical criterion as the effort term.** Section 7.13 predicts that a reward whose effort term is exactly $\sum_i (f_i / F_{\max,i})^2$ should close the 1.71× discrepancy. This distinguishes a limitation of reinforcement learning from a limitation of the reward chosen here — two conclusions with entirely different implications.

**Tune the exploration parameter of every algorithm on an equal budget.**

Target entropy for SAC, action-noise scale for TD3 and DDPG, entropy coefficient for PPO. This is the only route to a fair four-way comparison, given that this work identifies exploration as the decisive axis.

**Compare against electromyography.** Static optimisation is a model; measured muscle activity is closer to ground truth. This step would convert a computational comparison into a physiological claim, and is the natural extension toward the clinical applications that motivate the work.

**Repeat the synergy and co-contraction analyses on a valid policy.** The implementation now exists; the earlier results were computed on policies since established to be invalid.

## 8.11 Closing remark

The contribution of this thesis is narrower than initially envisaged but rests on evidence that can be inspected. Every quantitative statement traces to a named artifact produced by a documented procedure, and where a result is weak, uncertain or absent, the text records it as such.

That a learned controller recovers the coordination structure of a classical optimisation criterion without ever being shown it is a genuine and encouraging result. That it does so while expending 71 % more effort than necessary is an equally genuine limitation, and identifying it required measuring muscle forces directly — which no amount of additional accuracy on the reaching task would ever have revealed.

# Bibliographie


[1] Emanuel Todorov, Tom Erez, and Yuval Tassa. “MuJoCo: A Physics Engine for Model-Based Control”. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2012, pp. 5026–5033. DOI: 10.1109/IROS.2012.6386109.

[2] A. V. Hill. “The Heat of Shortening and the Dynamic Constants of Muscle”. In: *Proceedings of the Royal Society of London. Series B, Biological Sciences* 126.843 (1938), pp. 136–195. DOI: 10.1098/rspb.1938.0050.

[3] Felix E. Zajac. “Muscle and Tendon: Properties, Models, Scaling, and Application to Biomechanics and Motor Control”. In: *Critical Reviews in Biomedical Engineering* 17.4 (1989), pp. 359–411.

[4] Darryl G. Thelen. “Adjustment of Muscle Mechanics Model Parameters to Simulate Dynamic Contractions in Older Adults”. In: *Journal of Biomechanical Engineering* 125.1 (2003), pp. 70–77. DOI: 10.1115/1.1531112.

[5] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. “Stable-Baselines3: Reliable Reinforcement Learning Implementations”. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.

[6] Timothy P. Lillicrap et al. “Continuous Control with Deep Reinforcement Learning”. In: *International Conference on Learning Representations (ICLR)*. 2016.

[7] Scott Fujimoto, Herke van Hoof, and David Meger. “Addressing Function Approximation Error in Actor-Critic Methods”. In: *International Conference on Machine Learning (ICML)*. Vol. 80. 2018, pp. 1587–1596.

[8] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. In: *International Conference on Machine Learning (ICML)*. Vol. 80. 2018, pp. 1861–1870.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).

[10] Gert Kwakkel, Boudewijn J. Kollen, Jeroen van der Grond, and Arie J. H. Prevo. "Probability of Regaining Dexterity in the Flaccid Upper Limb: Impact of Severity of Paresis and Time Since Onset in Acute Stroke". In: *Stroke* 34.9 (2003), pp. 2181–2186. DOI: 10.1161/01.STR.0000087172.16305.CD.

[11] Scott L. Delp et al. "OpenSim: Open-Source Software to Create and Analyze Dynamic Simulations of Movement". In: *IEEE Transactions on Biomedical Engineering* 54.11 (2007), pp. 1940–1950. DOI: 10.1109/TBME.2007.901024.

[12] Volodymyr Mnih et al. "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236.

[13] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press, 2018. ISBN: 9780262039246.

[14] Vittorio Caggiano, Huawei Wang, Guillaume Durandau, Massimo Sartori, and Vikash Kumar. "MyoSuite: A Contact-Rich Simulation Suite for Musculoskeletal Motor Control". In: Proceedings of Machine Learning Research 168 (2022). L4DC 2022, pp. 492–507.

[15] Olivier Codol, Jonathan A. Michaels, Mehrdad Kashefi, J. Andrew Pruszynski, and Paul L. Gribble. "MotorNet: A Python Toolbox for Controlling Differentiable Biomechanical Effectors with Artificial Neural Networks". In: *eLife* 12 (2024), RP88591. DOI: 10.7554/eLife.88591.

[16] Lukasz Kidzinski et al. "Learning to Run Challenge Solutions: Adapting Reinforcement Learning Methods for Neuromusculoskeletal Environments". In: *The NIPS '17 Competition: Building Intelligent Systems*. Springer, 2018, pp. 121–153. DOI: 10.1007/978-3-319-94042-7_7.

[17] Valery L. Feigin et al. "World Stroke Organization (WSO): Global Stroke Fact Sheet 2022". In: *International Journal of Stroke* 17.1 (2022), pp. 18–29. DOI: 10.1177/17474930211065917.

[18] Albert C. Lo et al. "Robot-Assisted Therapy for Long-Term Upper-Limb Impairment after Stroke". In: *New England Journal of Medicine* 362.19 (2010), pp. 1772–1783. DOI: 10.1056/NEJMoa0911341.

[19] Katherine R. S. Holzbaur, Wendy M. Murray, and Scott L. Delp. "A Model of the Upper Extremity for Simulating Musculoskeletal Surgery and Analyzing Neuromuscular Control". In: *Annals of Biomedical Engineering* 33.6 (2005), pp. 829–840. DOI: 10.1007/s10439-005-3320-7.

[20] David A. Winter. *Biomechanics and Motor Control of Human Movement*. 4th ed. Hoboken, NJ: John Wiley & Sons, 2009. ISBN: 9780470398180.

[21] A. M. Gordon, A. F. Huxley, and F. J. Julian. “The Variation in Isometric Tension with Sarcomere Length in Vertebrate Muscle Fibres”. In: *The Journal of Physiology* 184.1 (1966), pp. 170–192. DOI: 10.1113/jphysiol.1966.sp007909.

[22] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. “Optuna: A Next-Generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701.

[23] K. Falconer and David A. Winter. “Quantitative Assessment of Co-Contraction at the Ankle Joint in Walking”. In: *Electromyography and Clinical Neurophysiology* 25.2–3 (1985), pp. 135–149.

[24] Lalit J. Bhargava, Marcus G. Pandy, and Frank C. Anderson. “A Phenomenological Model for Estimating Metabolic Energy Consumption in Muscle Contraction”. In: *Journal of Biomechanics* 37.1 (2004), pp. 81–88. DOI: 10.1016/S0021-9290(03)00239-2.

[25] Andrea d’Avella, Philippe Saltiel, and Emilio Bizzi. “Combinations of Muscle Synergies in the Construction of a Natural Motor Behavior”. In: *Nature Neuroscience* 6.3 (2003), pp. 300–308. DOI: 10.1038/nn1010.

[26] Emanuel Todorov. “Optimality Principles in Sensorimotor Control”. In: *Nature Neuroscience* 7.9 (2004), pp. 907–915. DOI: 10.1038/nn1309.

[27] Stephen H. Scott. “Optimal Feedback Control and the Neural Basis of Volitional Motor Control”. In: *Nature Reviews Neuroscience* 5.7 (2004), pp. 532–546. DOI: 10.1038/nrn1427.

[28] Tamar Flash and Neville Hogan. “The Coordination of Arm Movements: An Experimentally Confirmed Mathematical Model”. In: *The Journal of Neuroscience* 5.7 (1985), pp. 1688–1703. DOI: 10.1523/JNEUROSCI.05-07-01688.1985.

[29] Christopher M. Harris and Daniel M. Wolpert. “Signal-Dependent Noise Determines Motor Planning”. In: *Nature* 394.6695 (1998), pp. 780–784. DOI: 10.1038/29528.

[30] Emanuel Todorov and Michael I. Jordan. “Optimal Feedback Control as a Theory of Motor Coordination”. In: *Nature Neuroscience* 5.11 (2002), pp. 1226–1235. DOI: 10.1038/nn963.

[31] Daniel M. Wolpert and Mitsuo Kawato. “Multiple Paired Forward and Inverse Models for Motor Control”. In: *Neural Networks* 11.7–8 (1998), pp. 1317–1329. DOI: 10.1016/S0893-6080(98)00066-5.

[32] Lena H. Ting and J. Lucas McKay. “Neuromechanics of Muscle Synergies for Posture and Movement”. In: *Current Opinion in Neurobiology* 17.6 (2007), pp. 622–628. DOI: 10.1016/j.conb.2008.01.002.

[33] Emilio Bizzi and Vincent C. K. Cheung. “The Neural Origin of Muscle Synergies”. In: *Frontiers in Computational Neuroscience* 7 (2013), p. 51. DOI: 10.3389/fncom.2013.00051.

[34] Roy D. Crowninshield and Richard A. Brand. “A Physiologically Based Criterion of Muscle Force Prediction in Locomotion”. In: *Journal of Biomechanics* 14.11 (1981), pp. 793–801. DOI: 10.1016/0021-9290(81)90035-X.

[35] Frank C. Anderson and Marcus G. Pandy. “Static and Dynamic Optimization Solutions for Gait Are Practically Equivalent”. In: *Journal of Biomechanics* 34.2 (2001), pp. 153–161. DOI: 10.1016/S0021-9290(00)00155-X.

[36] Yuval Tassa et al. “dm_control: Software and Tasks for Continuous Control”. In: *Software Impacts* 6 (2020), p. 100022. DOI: 10.1016/j.simpa.2020.100022.

[37] Matthew Millard, Thomas Uchida, Ajay Seth, and Scott L. Delp. “Flexing Computational Muscle: Modeling and Simulation of Musculotendon Dynamics”. In: *Journal of Biomechanical Engineering* 135.2 (2013), p. 021005. DOI: 10.1115/1.4023390.

[38] Richard Bellman. *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.

[39] Vijay R. Konda and John N. Tsitsiklis. “Actor-Critic Algorithms”. In: *Advances in Neural Information Processing Systems* 12 (2000), pp. 1008–1014.

[40] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations (ICLR)*. 2016.

[41] Aravind Rajeswaran, Kendall Lowrey, Emanuel Todorov, and Sham Kakade. “Towards Generalization and Simplicity in Continuous Control”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.

[42] Seungmoon Song et al. “Deep Reinforcement Learning for Modeling Human Locomotion Control in Neuromechanical Simulation”. In: *Journal of NeuroEngineering and Rehabilitation* 18.1 (2021), p. 126. DOI: 10.1186/s12984-021-00919-y.

[43] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. "DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills". In: *ACM Transactions on Graphics* 37.4 (2018), 143:1–143:14. DOI: 10.1145/3197517.3201311.

[44] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World". In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133.

[45] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810. DOI: 10.1109/ICRA.2018.8460528.

[46] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. "Deep Reinforcement Learning That Matters". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018, pp. 3207–3214. DOI: 10.1609/aaai.v32i1.11694.

[47] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. "Deep Reinforcement Learning at the Edge of the Statistical Precipice". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 34. 2021, pp. 29304–29320.

[48] Wendy M. Murray, Scott L. Delp, and Thomas S. Buchanan. "Variation of Muscle Moment Arms with Elbow and Forearm Position". In: *Journal of Biomechanics* 28.5 (1995), pp. 513–525. DOI: 10.1016/0021-9290(94)00114-J.

[49] Reza Shadmehr and John W. Krakauer. "A Computational Neuroanatomy for Motor Control". In: *Experimental Brain Research* 185.3 (2008), pp. 359–381. DOI: 10.1007/s00221-008-1280-5.

[50] Xavier Glorot and Yoshua Bengio. "Understanding the Difficulty of Training Deep Feedforward Neural Networks". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Vol. 9. 2010, pp. 249–256.

[51] Joelle Pineau et al. "Improving Reproducibility in Machine Learning Research". In: *Journal of Machine Learning Research* 22.164 (2021), pp. 1–20.

[52] Andrea d'Avella, Alessandro Portone, Laure Fernandez, and Francesco Lacquaniti. "Control of Fast-Reaching Movements by Muscle Synergy Combinations". In: *The Journal of Neuroscience* 26.30 (2006), pp. 7791–7810. DOI: 10.1523/JNEUROSCI.0830-06.2006.

# A MJCF File of the Musculoskeletal Model

An excerpt from the MJCF file describing the nine-muscle arm. The muscle force scales follow [19]; the inertial parameters follow [20]. Table 3.2 lists the values as compiled. The complete file is in the Git repository accompanying this thesis.

```
<mujoco model="arm_9muscles">
  <option timestep="0.002" iterations="50" solver="Newton"
          integrator="implicit" cone="elliptic"/>

  <default>
    <muscle ctrllimited="true" ctrlrange="0 1"
            force="-1" scale="200" lmin="0.5" lmax="1.6"
            vmax="10" fpmax="1.5" fvmax="1.4"/>
  </default>

  <worldbody>
    <body name="humerus" pos="0 0 -0.3">
      <joint name="shoulder_flex" type="hinge" axis="0 1 0" range="
         -0.5 2.5" damping="0.1"/>
      <joint name="shoulder_abd"  type="hinge" axis="1 0 0" range="
         -0.5 2.5" damping="0.1"/>
      <joint name="shoulder_rot"  type="hinge" axis="0 0 1" range="
         -1.5 1.5" damping="0.1"/>
      <geom type="capsule" fromto="0 0 0 0 0 -0.3" size="0.04" mass
         ="1.98"/>

      <!-- Muscle origin sites (humerus) -->
      <site name="bic_origin_long" pos="0     0.01  0.05"/>
      <site name="bic_s_origin"    pos="0.01  0.01  0.03"/>
      <site name="bra_origin"      pos="0     0.00 -0.15"/>
      <site name="tri_l_origin"    pos="0    -0.01  0.02"/>
      <site name="delt_a_origin"   pos="0.05  0.00  0.05"/>
      <site name="delt_m_origin"   pos="0.00  0.05  0.05"/>
      <site name="delt_p_origin"   pos="-0.05 0.00  0.05"/>

      <body name="radius" pos="0 0 -0.3">
        <joint name="elbow_flex" type="hinge" axis="0 1 0" range="0
            2.7" damping="0.1"/>
```

```
29        <geom type="capsule" fromto="0 0 0 0 0 -0.27" size="0.03"
             mass="1.18"/>
30
31        <!-- Via-point and insertion sites (forearm) -->
32        <site name="bic_via_1"        pos="0     0.02 -0.02"/>
33        <site name="bic_insertion"    pos="0     0.02 -0.08"/>
34        <site name="bic_s_insertion"  pos="0     0.02 -0.08"/>
35        <site name="bra_insertion"    pos="0     0.01 -0.05"/>
36        <site name="tri_l_insertion"  pos="0    -0.01 -0.01"/>
37        <site name="tri_la_origin"    pos="0.01 -0.01 -0.10"/>
38        <site name="tri_la_insertion" pos="0    -0.01 -0.01"/>
39        <site name="tri_m_origin"     pos="-0.01 -0.01 -0.10"/>
40        <site name="tri_m_insertion"  pos="0    -0.01 -0.01"/>
41        <site name="delt_a_insertion" pos="0.03  0.00 -0.15"/>
42        <site name="delt_m_insertion" pos="0.00  0.03 -0.15"/>
43        <site name="delt_p_insertion" pos="-0.03 0.00 -0.15"/>
44
45        <body name="hand" pos="0 0 -0.27">
46          <geom type="sphere" size="0.04" mass="0.45"/>
47          <site name="end_effector" pos="0 0 0"/>
48        </body>
49      </body>
50    </body>
51  </worldbody>
52
53  <tendon>
54    <spatial name="biceps_long"  width="0.003">
55      <site site="bic_origin_long"/>
56      <site site="bic_via_1"/>
57      <site site="bic_insertion"/>
58    </spatial>
59    <spatial name="biceps_short" width="0.003">
60      <site site="bic_s_origin"/>
61      <site site="bic_s_insertion"/>
62    </spatial>
63    <spatial name="brachialis"   width="0.003">
64      <site site="bra_origin"/>
65      <site site="bra_insertion"/>
66    </spatial>
67    <spatial name="triceps_long" width="0.003">
68      <site site="tri_l_origin"/>
69      <site site="tri_l_insertion"/>
70    </spatial>
71    <spatial name="triceps_lat"  width="0.003">
72      <site site="tri_la_origin"/>
```

```
      <site site="tri_la_insertion"/>
    </spatial>
    <spatial name="triceps_med"  width="0.003">
      <site site="tri_m_origin"/>
      <site site="tri_m_insertion"/>
    </spatial>
    <spatial name="deltoid_ant"  width="0.003">
      <site site="delt_a_origin"/>
      <site site="delt_a_insertion"/>
    </spatial>
    <spatial name="deltoid_med"  width="0.003">
      <site site="delt_m_origin"/>
      <site site="delt_m_insertion"/>
    </spatial>
    <spatial name="deltoid_post" width="0.003">
      <site site="delt_p_origin"/>
      <site site="delt_p_insertion"/>
    </spatial>
  </tendon>

  <actuator>
    <muscle name="biceps_long"  tendon="biceps_long"  force="624"
       lengthrange="0.01 0.50"/>
    <muscle name="biceps_short" tendon="biceps_short" force="435"
       lengthrange="0.01 0.50"/>
    <muscle name="brachialis"   tendon="brachialis"   force="987"
       lengthrange="0.01 0.50"/>
    <muscle name="triceps_long" tendon="triceps_long" force="798"
       lengthrange="0.01 0.50"/>
    <muscle name="triceps_lat"  tendon="triceps_lat"  force="624"
       lengthrange="0.01 0.50"/>
    <muscle name="triceps_med"  tendon="triceps_med"  force="624"
       lengthrange="0.01 0.50"/>
    <muscle name="deltoid_ant"  tendon="deltoid_ant"  force="1142"
       lengthrange="0.01 0.50"/>
    <muscle name="deltoid_med"  tendon="deltoid_med"  force="1142"
       lengthrange="0.01 0.50"/>
    <muscle name="deltoid_post" tendon="deltoid_post" force="259"
       lengthrange="0.01 0.50"/>
  </actuator>

  <sensor>
    <actuatorfrc name="f_bl"  actuator="biceps_long"/>
    <actuatorfrc name="f_bs"  actuator="biceps_short"/>
    <actuatorfrc name="f_bra" actuator="brachialis"/>
```

```
109     <actuatorfrc name="f_tl"  actuator="triceps_long"/>
110     <actuatorfrc name="f_tla" actuator="triceps_lat"/>
111     <actuatorfrc name="f_tm"  actuator="triceps_med"/>
112     <actuatorfrc name="f_da"  actuator="deltoid_ant"/>
113     <actuatorfrc name="f_dm"  actuator="deltoid_med"/>
114     <actuatorfrc name="f_dp"  actuator="deltoid_post"/>
115     <jointpos name="shf_q" joint="shoulder_flex"/>
116     <jointvel name="shf_v" joint="shoulder_flex"/>
117   </sensor>
118 </mujoco>
```

**Listing A.1.** Excerpt from the MJCF model file (`arm.xml`).

# B Gymnasium Environment (Python)

The Gymnasium environment `ArmReachEnv`, which integrates domain randomisation and computes the reward of section 3.4.4.

```python
class ArmReachEnv(gym.Env):
    """Nine-muscle upper limb, 3D reaching task."""

    def __init__(self, domain_rand: bool = True, max_steps: int = None,
                 seed: int = None, reward_weights: dict = None, ...):
        self.model = mujoco.MjModel.from_xml_path(config.ARM_XML)
        self.data  = mujoco.MjData(self.model)

        # Nominal values kept so domain randomisation rescales from nominal
        # rather than compounding on the current value (see sec: defects).
        self._nominal_body_mass = self.model.body_mass.copy()
        self._nominal_gainprm   = self.model.actuator_gainprm.copy()
        self._nominal_damping   = self.model.dof_damping.copy()

        # Per-muscle peak isometric force, used to normalise the effort term.
        self._fmax = self._nominal_gainprm[:, 2].copy()

        self.action_space      = spaces.Box(0.0, 1.0, shape=(9,), dtype=np.float32)
        self.observation_space = spaces.Box(-np.inf, np.inf, shape=(38,),
                                            dtype=np.float32)

        # Reward weights come from config.REWARD_WEIGHTS -- the single source
        # of truth. They are hand-set, NOT searched. An override dict is used
```

```
        # only by the reward-ablation study, to zero individual
            terms.
        w = dict(config.REWARD_WEIGHTS)
        if reward_weights:
            w.update(reward_weights)
        self.w1, self.w2, self.w3 = w["w1"], w["w2"], w["w3"]
        self.w4, self.w5          = w["w4"], w["w5"]
        self.w6                   = w.get("w6", 0.0)
        self.w7                   = w.get("w7", 0.0)

    def step(self, action):
        ...
        reward  = self.w4 - (self.w1 * (err**2 + 0.5 * err)
                             + self.w2 * e_eff
                             + self.w3 * float(np.mean(d_act**2)))
        reward += self.w7 * np.exp(-err / config.PRECISION_SCALE)
```

**Listing B.1.** Excerpt from `rl/environment.py`, as used for every reported result.

An earlier version of this appendix printed a hand-written excerpt that had drifted from the code: it named the class `ArmMusculoEnv`, set `max_steps=1000` where the environment uses 100, gave the reward weights as $(1.0, 0.005, 0.01)$, and annotated them "optimised by Optuna". The weights are $(1.0, 1.0, 0.5)$ and were never searched — section 5.3.2 records that claim as withdrawn. The listing above is taken from the source that produced the results.

# C SAC Training Script

Training script using Stable-Baselines3 for SAC, with observation normalisation, periodic evaluation, and saving of the best-performing models.

```
import argparse, os, numpy as np
from stable_baselines3 import SAC
from stable_baselines3.common.vec_env import (
    SubprocVecEnv, VecNormalize)
from stable_baselines3.common.callbacks import EvalCallback
from arm_env import ArmMusculoEnv

def make_env(seed, domain_rand=True):
    def _init():
        env = ArmMusculoEnv(domain_rand=domain_rand, seed=seed)
        env.reset(seed=seed)
        return env
    return _init

def main(args):
    os.makedirs(args.logdir, exist_ok=True)

    # Environnements vectorises
    train_envs = SubprocVecEnv(
        [make_env(args.seed + i) for i in range(args.n_envs)])
    train_envs = VecNormalize(
        train_envs, norm_obs=True, norm_reward=True, clip_obs=10.0)

    eval_env = SubprocVecEnv([make_env(args.seed + 999)])
    eval_env = VecNormalize(
        eval_env, norm_obs=True, norm_reward=False,
        training=False)

    # SAC defaults as registered in rl/algorithms.py. The tuned run
        overrides
    # gamma (0.957), learning_rate (4.40e-4), batch_size (128) and
       tau (0.0188)
    # via the 'hyperparams' passthrough; see tab:hparams.
    #
    # gradient_steps is set by the caller to 2 * config.N_ENVS = 8,
        giving a
```

```
    # measured replay ratio of 1.867. Leaving it at 1 is the defect
        of
    # sec:replaybug and must not be copied from here.
    model = SAC(
        "MlpPolicy", train_envs,
        learning_rate=3e-4,
        buffer_size=300_000,
        batch_size=512,
        learning_starts=4_000,
        tau=0.005,
        gamma=0.99,
        train_freq=1,
        gradient_steps=2 * config.N_ENVS,
        ent_coef="auto",
        target_entropy="auto",
        policy_kwargs=dict(net_arch=[256, 256]),
        seed=args.seed,
        verbose=1)
        # TensorBoard logging is deliberately not enabled (sec.
            5.1).

    callback = EvalCallback(
        eval_env,
        best_model_save_path=args.logdir,
        log_path=args.logdir,
        eval_freq=5000,
        n_eval_episodes=20,
        deterministic=True)

    model.learn(total_timesteps=args.steps, callback=callback)
    model.save(os.path.join(args.logdir, "final_model"))
    train_envs.save(os.path.join(args.logdir, "vecnormalize.pkl"))

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--seed", type=int, required=True)
    parser.add_argument("--steps", type=int, default=2_000_000)
    parser.add_argument("--n_envs", type=int, default=4)
    parser.add_argument("--logdir", type=str, default="runs/sac")
    args = parser.parse_args()
    main(args)
```

**Listing C.1.** Excerpt from `rl/train.py`.

# D Hill Integration by Newton–Raphson

Numerical solution of the muscle equilibrium equation (2.9) by Newton–Raphson iterations, used at each simulation step to determine the fibre length $\ell_M$ from the musculotendon length $\ell_{MT}$ and the activation level $a$.

```
import numpy as np

def solve_muscle_equilibrium(L_MT, a, L_M0, L_S0, alpha0,
                             F_max, epsilon=1e-6, max_iter=50):
    """
    Resout f_CE + f_PEE - f_SEE = 0 pour L_M.
    Retourne (L_M, L_S, F_muscle) par iterations de Newton-Raphson.
    """
    # Initialisation : hypothese L_S = L_S0 (tendon a longueur
       nominale)
    L_M = L_MT - L_S0
    L_M = max(0.5 * L_M0, min(1.6 * L_M0, L_M))  # bornes

    for it in range(max_iter):
        # Angle de pennation
        sin_alpha = (L_M0 * np.sin(alpha0)) / L_M
        cos_alpha = np.sqrt(max(1.0 - sin_alpha**2, 1e-6))

        # Longueur du tendon
        L_S = L_MT - L_M * cos_alpha

        # Force active (gaussienne centree sur L_M0)
        f_l = np.exp(-((L_M / L_M0 - 1.0) / 0.45) ** 2)
        f_CE = a * f_l * F_max

        # Force passive (exponentielle)
        if L_M > L_M0:
            f_PEE = F_max * (np.exp(5.0 * (L_M / L_M0 - 1.0)) -
               1.0) \
                    / (np.exp(5.0) - 1.0)
        else:
            f_PEE = 0.0

```

```
        # Force tendineuse (quadratique en deformation)
        eps_s = (L_S - L_S0) / L_S0
        if eps_s > 0:
            f_SEE = F_max * 37.5 * eps_s ** 2
        else:
            f_SEE = 0.0

        # Equilibrium residual
        res = (f_CE + f_PEE) * cos_alpha - f_SEE
        if abs(res) < epsilon:
            break

        # Derivee numerique pour mise a jour
        dL = 1e-5 * L_M0
        L_M_p = L_M + dL
        # (recalcul rapide f_CE, f_PEE, f_SEE a L_M_p omis pour
            concision)
        # Puis : L_M -= res / (dRes / dL_M)
        L_M -= 0.5 * res / (F_max / L_M0)  # simplification pas
            adaptatif
        L_M = max(0.5 * L_M0, min(1.6 * L_M0, L_M))

    return L_M, L_S, (f_CE + f_PEE) * cos_alpha
```

**Listing D.1.** Musculotendon equilibrium solver.

# E Detailed Statistical Protocol

**This protocol was not applied to any result in this thesis, and no $p$-value, effect size or confidence interval appears anywhere in the text.** It is retained here because the implementation is complete and correct, and because the seed count — not the analysis — is what prevents its use.

With the three seeds the available hardware permitted (section 5.3.2), the two-sided Wilcoxon signed-rank test has a minimum attainable $p$-value of 0.25. Running it would produce numbers that cannot reach significance under any threshold while giving the appearance of inferential rigour. An earlier version of this work reported $p$-values from this protocol over a purported ten seeds; those analyses were never executed and are withdrawn (section 8.6).

The code below therefore documents what *should* be run once the replication recommended in section 8.10 is complete: Wilcoxon signed-rank test for paired samples, Bonferroni correction for multiple comparisons, BCa bootstrap confidence intervals, and rank-biserial effect size.

```
import numpy as np
from scipy import stats
from scipy.stats import wilcoxon

def rank_biserial(x, y):
    """Rank-biserial effect size for two paired samples."""
    diff = np.asarray(x) - np.asarray(y)
    pos = np.sum(diff > 0)
    neg = np.sum(diff < 0)
    return (pos - neg) / (pos + neg) if (pos + neg) > 0 else 0.0

def bootstrap_bca(data, stat_fn=np.mean, n_boot=10000,
                  alpha=0.05, rng=None):
    """BCa confidence interval (bias-corrected and accelerated)."""
    rng = np.random.default_rng(rng)
    data = np.asarray(data)
    n = len(data)
    boot_stats = np.array([
        stat_fn(rng.choice(data, size=n, replace=True))
        for _ in range(n_boot)])
    theta_hat = stat_fn(data)

```

```
    # Bias correction
    z0 = stats.norm.ppf(np.mean(boot_stats < theta_hat))
    # Acceleration via jackknife estimate
    jack = np.array([
        stat_fn(np.delete(data, i)) for i in range(n)])
    jack_mean = np.mean(jack)
    num = np.sum((jack_mean - jack) ** 3)
    den = 6.0 * (np.sum((jack_mean - jack) ** 2) ** 1.5 + 1e-12)
    a = num / den

    z_a = stats.norm.ppf(alpha / 2)
    z_1a = stats.norm.ppf(1 - alpha / 2)
    a1 = stats.norm.cdf(z0 + (z0 + z_a) / (1 - a * (z0 + z_a)))
    a2 = stats.norm.cdf(z0 + (z0 + z_1a) / (1 - a * (z0 + z_1a)))
    return (np.quantile(boot_stats, a1),
            np.quantile(boot_stats, a2))

def compare_algorithms(results):
    """results: dict {algo: array[10] de taux de succes}"""
    keys = list(results.keys())
    n_tests = len(keys) * (len(keys) - 1) // 2
    alpha_corrected = 0.05 / n_tests  # Bonferroni

    for i, a in enumerate(keys):
        for b in keys[i+1:]:
            stat, p = wilcoxon(results[a], results[b],
                               alternative='two-sided')
            rb = rank_biserial(results[a], results[b])
            ci_a = bootstrap_bca(results[a])
            print(f"{a} vs {b}: W={stat}, p={p:.4f}, "
                  f"rb={rb:.2f}, signif={p < alpha_corrected}")
```

**Listing E.1.** Inter-algorithm statistical analysis.